

多 Agent 协同小分子对接筛选系统

技术报告

本地部署版 · 自动准备 / 对接 / 评估 / 异常处理 / 报告

版本 v0.8.0

生成日期 2026-09-24

摘要 本系统在单机上把「小分子库 × 蛋白质受体」的真实分子对接筛选完整自动化——由 1 个主管 Agent 统筹 4 个专业子 Agent，用 RDKit / meeko / AutoDock Vina 等真实计算引擎产出可追溯的性质、对接能量、结合模式比较与固定 9 章报告，全部数值均可回溯到工具原始输出。

内容源：docs/技术报告.md

排版与插图：scripts/build_report_docx.py 自动生成

本报告全部数值来自仓库内代码、既有文档与实测运行；未实测项已明确标注。

目 录

汇报要点（12 张幻灯片提纲）	7
1. 项目概述.....	12
1.1 要解决的问题.....	12
1.2 六条能力目标与验收标准.....	12
1.3 明确的非目标.....	13
1.4 两条贯穿全局的硬约束.....	14
2. 总体架构.....	15
2.1 分层图.....	15
2.2 一次运行的端到端数据流（含 SSE 事件）	16
2.3 两条执行路径.....	17
3. Agent 体系.....	18
3.1 六个角色与工具清单.....	18
3.2 分发链路与边界校验.....	20
3.3 共享黑板（横向协作）	21
3.4 主管纪律条款摘要（config/agent_llm_config.json 的 sp，共 12 条）	22
4. 任务受理层（intake）	25
4.1 调用链.....	25
4.2 确定性规约字段（task_spec）	25
4.3 run / ask / reject 判定规则.....	26
4.4 分子抽取（三种写法）	27
4.5 多轮会话（conversation_id、prior_turns、_can_ask 例外）	28
4.6 受体三态纪律（核心）	29
5. 在线解析链（resolve）	32
5.1 名称归一化与中英映射.....	32
5.2 UniProt 多策略检索.....	33
5.3 打分规则（逐条列出）	33
5.4 RCSB → AlphaFold 结构获取链与溯源字段.....	35
5.5 choices 结构化选项协议.....	35
6. 受体与配体准备.....	38
6.1 受体准备：去水 / 杂原子溯源.....	38
6.2 keep_hetatm 与「已准备好的 PDBQT」	39
6.3 altloc 重试、meeko 准备链与内容寻址缓存.....	39
6.4 配体准备：3D 构象与化学体检.....	40
7. 对接盒（重点，含实测）	43
7.1 口袋检测：P2Rank 2.5.1 与内置几何法.....	43

7.2 盒子来源优先级.....	44
7.3 库级配体下限 (C 方案 B1)	45
7.4 超限分子分组 (C 方案 B2)	47
7.5 box_group / box_size 落盘与 CSV 列.....	48
7.6 一致性不变量.....	49
7.7 换盒子实测：分数最大差 1.34 kcal/mol (原样引用)	49
8. 对接计算与参数自动规划 (重点)	51
8.1 Vina 1.2.7 调用链.....	51
8.2 AutoDock4 回退.....	51
8.2b 外部引擎 (engine=external)	52
8.2c 库已知后补规划与两阶段漏斗.....	54
8.2d 阻断式询问 (共晶配体阳性对照)	54
8.2e 模型可见消息日志 (气泡可审计)	55
8.3 exhaustiveness 自动规划公式与全部默认值.....	55
8.4 两阶段漏斗.....	58
8.5 pilot 预算护栏 (精度优先降级)	59
8.6 并发规划表 (plan_concurrency)	60
8.7 内存护栏、LPT 调度与取消.....	61
8.8 可复现性 (seed / seed_policy 与「批次无关」实测)	62
9. 评估与报告.....	64
9.1 性质评估 (Lipinski 等)	64
9.2 结合模式 (Morgan + MACCS + SMARTS 药效团 + 性质差)	65
9.3 阳性对照基线.....	66
9.4 报告 9 章与 PDF 生成.....	67
9.5 图表与 CSV 产物.....	70
9.6 产物清单与下载命名规范.....	72
10. 工程实现与质量保障.....	74
10.1 分层依赖与不变量清单.....	74
10.2 测试与门禁全清单 (项数为本次实测)	75
10.3 真实性取证与反证控制.....	77
10.4 上下文预算 (大库不撑爆模型)	78
10.5 文档体系、并发隔离与静态规范.....	78
11. 可信度、局限与风险.....	81
11.1 打分函数与方法的固有边界.....	81
11.1b 质子化引擎 (v0.32 起：受体与配体都用专业工具)	82
11.2 盒子 / 参数效应 (引实测)	83
11.3 同阶段内可比性保证.....	83
11.4 运行级与工程级缺口 (可信度风险的直接来源)	84

11.5 存留与边界.....	85
12. 部署与运维.....	87
12.1 一键启动与自检.....	87
12.2 端口与监听.....	88
12.3 设置页优先级链与只读项.....	88
12.4 目录结构.....	90
12.5 日志、产物与残留进程清理.....	91
13. 实测数据汇总.....	93
13.1 表一：并发、上下文与吞吐.....	93
13.2 表二：盒子与参数效应（全部为实测）	94
13.3 表三：端到端示例与「零计算」对照.....	95
14. 后续路线.....	98
14.1 P0 / P1（按 docs/architecture.md § 15 原样归纳，标注状态）	98
14.2 P2（锦上添花）	100
14.4 「明确不建议做」（docs/architecture.md § 15.8，共 4 条）	100
14.5 规范清理计划（对应 docs/architecture.md § 15.9）	101
15. 附录.....	102
15.1 全部环境变量默认值总表.....	102
15.2 关键文件索引（哪个文件负责什么）	107
15.3 术语表.....	109

提示：本目录为 Word 目录域（TOC）的缓存结果；在 Word 中按 Ctrl+A → F9 或右键「更新域」可刷新页码。

一句话摘要：本系统在**单机**上把「小分子库 × 蛋白质受体」的真实分子对接筛选完整自动化——由 1 个主管 Agent 统筹 4 个专业子 Agent，用 RDKit / meeko / AutoDock Vina 等**真实计算引擎**产出可追溯的性质、对接能量、结合模式比较与固定 9 章报告，全部数值均可回溯到工具原始输出。

文档分工：本报告是**评审用完整叙述**（含实测数据、来源引用与方法局限），下表的规模/测试数字是 **报告日期当天的实测快照**，不随代码更新。工程细节与验收口径见 `[../README.md](../README.md)` 与 `[architecture.md](architecture.md)`，HTTP 契约见 `[api.md](api.md)`，读者向短版见 `[../../docs/技术文档.md](../../docs/技术文档.md)`，历史日志见 `[../CHANGELOG.md](../CHANGELOG.md)`。

表 1 文档基本信息

项	内容
报告日期	2026-09-16
系统版本	0.8.0 （ <code>src/docking_agent/__init__.py</code> ； <code>pyproject.toml</code> 的 <code>version</code> 同为 0.8.0）
代码规模（实测）	Python 源码 55 个文件 / 16,336 行（ <code>src/docking_agent/</code> ）；测试 21 个文件 / 6,253 行（ <code>tests/</code> ）；脚本 16 个 / 2,595 行（ <code>scripts/</code> ）；前端 10,676 行（ <code>web/app.js</code> 6,345 + <code>index.html</code> 886 + <code>styles.css</code> 3,445）
模块行数（实测）	顶层 3,804 行； <code>api/</code> 1,428 行； <code>agents/</code> 1,202 行； <code>core/</code> 4,587 行； <code>tools/</code> 2,332 行； <code>reporting/</code> 2,165 行； <code>runtime/</code> 818 行
自动化测试项（实测）	pytest 315 项通过 （147.19 s）； <code>smoke_test.py</code> 46/46 ； <code>verify_docking.py</code> 52/52 ； <code>check_web.py</code> 118/118 ； <code>ui_e2e.js</code> 116/116 ； <code>ui_shot.py</code> check 6/6 页 ； <code>intake_eval.py</code> 14 项 ； <code>lint_local.py</code> 通过（新增 0 项）； <code>snapshot_ids.py</code> diff 基线 213 → 当前 241，无 id 被移除
工具链（本次实测运行回显）	AutoDock Vina 1.2.7 / RDKit 2026.03.6 / meeko 0.8.0 / P2Rank 2.5.1 / Python 3.12.3 / matplotlib 3.11.1 （ <code>var/runs/20260915-174954-6983/report.md: 75</code> ）
部署形态	本机一键启动（ <code>bash start.sh</code> ），默认只监听 <code>127.0.0.1</code> ，无鉴权，单机多进程吃满多核

本报告的数据纪律：所有数值都标注来源（文件:行号 或 文档 § 小节）。仓库里没有实测记录的量一律写「未实测」，不做任何推算冒充实测。

汇报要点（12 张幻灯片提纲）

幻灯片 1 — 封面与一句话价值

- 标题：多 Agent 协作分子对接筛选工作台（本地部署 v0.8.0）。
- 一句话：把「小分子库 × 受体」的真实对接筛选全流程自动化，并保证**每个数字都能回溯**。
- 底座：RDKit / meeko / AutoDock Vina 1.2.7 / P2Rank 2.5.1，全部真实计算引擎（配图 `var/tmp/ui_shots/first.png`）。

幻灯片 2 — 要解决的问题与非目标

- 问题：小分子库规模大、受体准备繁琐、结果可解释性差、异常体系（金属/盐/辅因子）常被静默丢弃。
- 六条能力目标：自动准备 / 自动对接 / 自动评估 / 自动异常处理 / 总结报告 / 主管协调（`docs/architecture.md §1.1`）。
- 非目标：不做分子生成、不做分布式集群、不自研引擎、不做多用户 SaaS、不引入 RAG（`§1.2`）。
- 两条硬约束：**真实性**（数值只来自工具）与**不静默**（丢弃/改写/降级逐条可见，`§1.3`）。

幻灯片 3 — 总体架构与端到端数据流

- 分层：入口 → 受理（intake）→ 编排（coordinator / pipeline）→ 角色 Agent → 工具 → 核心计算 → 持久化/报告 → 前端。
- 依 赖 方 向 单 向 向 下 ， `core/*` 绝 不 `import tools/agents/api`（`docs/architecture.md §2.1`）。
- `decision=ask/reject` 时**零工具调用**；SSE 事件带 `ts`，前端据此画**实测时间轴**（重叠即并行）。
- 配 图 ： `var/tmp/ui_shots/check-runs.png` 或 运 行 产 物 `var/tmp/smoke/docking_chart.png`。

幻灯片 4 — 六个角色与两种执行路径

- 6 个模型实例：`intake/coordinator/property/pocket/docking/binding`，各自独立实例与登记（`runtime/llm.py:42`）。
- 4 个子 Agent 各有独立 `checkpoint`；`invoke_worker` 每次调用用全新 `thread_id`（`agents/workers.py:159`）。
- 主管绑 9 个工具、滑动窗口 40 条（`agents/coordinator.py:41`）；两条路径共用 `core/` 与报告模板。
- 配图：`var/diag/live_agent_running.png`（编排图与实时逐分子表）。

幻灯片 5 — 任务受理层：三态决策 + 多轮会话

- `run / ask / reject` 由受理层显式判定，编排层无条件服从（`intake.py:535`）。
- 必需信息不齐 → 只提问、零工具调用：受体与分子库是必需项（缺任一 → `decision=ask`，不调任何工具）；系统没有默认受体，分子也不会静默用示例库；对照/参数仍有默认值、位点由工具确定。
- 多轮会话：`conversation_id` 作 `thread_id`，确定性继承上一轮分子与受体（`intake.py:366`）。
- 分子抽取支持 名称:SMILES / 名称 空格 SMILES / 散文中夹带的 SMILES（`core/ligands.py:52`）；配图 `var/tmp/shot/chat-advanced.png`。

幻灯片 6 — 受体三态纪律（产品底线，代码强制）

- 没提受体 → ``decision=ask``、零对接计算：只提问（① PDB 编号 / UniProt accession；② 受体名称，中英文，在线解析；③ 上传结构文件），系统没有默认受体，不提供预置受体候选、不写「已使用系统默认…」。
- 点名可解析 → 第一步自动解析（UniProt 多策略 + RCSB/AlphaFold），报告写明 accession/物种/结构来源。
- 点名不可解析或歧义 → 列 `choices` 让用户选，零对接计算；代码护栏在任何引擎调用前拦下（`agents/dispatch.py:315`）。
- 实测：拟南芥 ROS1 → Q9SJQ6/7YHP 自动继续；只写「ROS1」→ 7 个候选并列 64.0 → 停下让用户选（`var/e2e/final_ambig.sse`）。

幻灯片 7 — 对接盒：工具定盒 + 一致性第一

- 来源优先级：显式指定 > 实验位点 > 工具预测（P2Rank/几何法）> 质心兜底（`core/pockets.py:867`）。
- 一个受体 → 一个盒子、全库共用；绝不逐分子换盒（实测换盒最大差 **1.34 kcal/mol** 且非单调）。
- 库级下限 **P95(库内配体 3D 跨度) + 10 Å**（默认开，抽样 $K=200$ ）；超限分子进 `box_group="large"`，同一中心、更大盒子、不跨组比较。
- 配图：`var/tmp/smoke/affinity_histogram.png`、`var/runs/20260915-174954-6983/report.pdf` 第 1 页。

幻灯片 8 — 对接参数自动规划：精度优先 + 预算护栏

- 公式：`exhaustiveness = clamp(round(base × f_rot × f_box), 2, 32)`（`core/params.py:347`）；`base` 筛选 16 / 姿态分析 16（`AUTO_PARAM_BASE_SCREENING / AUTO_PARAM_BASE_BINDING`，v0.25 起由 12 提到 16，与界面默认同档）。
- `f_rot = clamp(P90 可旋转键/5, 0.75, 2.5)`；`f_box = clamp((V/223)(1/3), 1.0, 2.0)`。
- 两阶段漏斗 $N \geq 500$ ；pilot 护栏（库 ≥ 50 ，最贵 3 个分子真实试跑外推，超预算先降头部再降强度）。
- 参数是运行级/阶段级：同一 pass 内 `exhaustiveness/box_size/box_center` 完全一致；配图同幻灯片 7 的 PDF「1.5」页。

幻灯片 9 — 评估、报告与产物

- 报告固定 9 章（含「方法与局限」「失败与跳过」）；`report.pdf` 有封面工具版本、章节分页、页脚 `run id · 页码 · 生成时间`、PNG 真实内嵌。
- 正文不含 URL：`_strip_urls` 把 URL 替换为「（外链已省略）」（`reporting/content.py:118`）。
- `ranking.csv` 32 列（含 `seed/seed_policy/box_group/box_size/status/error`）；下载命名 `dock_{run_id}_{受体}_{N}mols_{kind}.{ext}`。
- 配图：`var/tmp/shot/pdfcover-1.png`、`var/tmp/smoke/structure_grid.png`。

幻灯片 10 — 工程实现与质量保障

- 20 个 @tool，docstring 即接口契约；变更门禁全绿：pytest 315 / smoke 46 / verify_docking 52 / check_web 118 / ui_e2e 116 / ui_shot 6 页 / intake_eval 14 / lint_local 通过。
- 真实性取证：verify_docking.py 不 import 项目对接代码，独立用 vina API 从零复算并逐位比对。
- 反证控制：盒子平移 25 Å (-5.78→-5.12)、换受体 (-5.78 vs -6.55)、非法 SMILES 不出数值、engine=autodock 明确报错。
- 文档也进测试：tests/test_docs_consistency.py 机器判定文档与代码一致；配图 var/tmp/ui_shots/check-manual.png。

幻灯片 11 — 实测数据（并发 / 盒子 / 端到端）

- 96 分子 exh=1：串行 159.8 s → 8 进程 28.2 s (**5.7×**)，逐位一致；7 分子精算 **4.9×** (40.0→8.1 s)。
- 换盒子最大差 **1.34 kcal/mol** 且非单调；上下文压缩 10k 行 **4.69 MB → 3,881 B**。
- 端到端：植物 ROS1 → UniProt Q9SJK6 → RCSB 7YHP → Mancozeb 代表结构 EBDC → 亲和力 **-4.28 kcal/mol**。
- 配图：var/tmp/smoke/docking_chart.png、var/tmp/hist.png。

幻灯片 12 — 局限、风险与路线

- Vina 是**近似打分函数**；未做重打分/共识打分/RMSD 聚类；未保留结构水。质子化态方面：受体已按目标 pH 用 pdb2pqr+PROPKA 准备，配体已用专业 pKa 引擎在目标 pH 窗口内枚举微观态，但**只取一个形式对接**，未做微观态布居/能量比较与互变异构体枚举。
- apo/ 去辅因子条件下的结论需显式标注；AD4 与原引擎不可通约 (docs/architecture.md §15.8 明确不建议加码)。
- 内存 checkpointer 只有进程内记忆边界，服务重启后多轮上下文失效（可切 sqlite）。
- 路线：P0「PAINS 标记 / 失败清单进报告 / 运行级兜底 reaper+超时+可取消」、P1、P2 见 docs/architecture.md §15；配图 var/tmp/ui_shots/check-chat-narrow.png。

1. 项目概述

1.1 要解决的问题

结论：本系统解决的是「把一个小分子库在一个已知/可推断的结合位点上跑完真实对接，并给出可复算、可追溯的排序与报告」。

表 2 要解决的问题

痛点	现状做法	本系统做法
受体准备繁琐	手工去水、加氢、转 PDBQT、猜盒子坐标	上传 .pdb 即现场用 meeko 准备，盒子由口袋工具确定 (core/receptors.py:235、core/pockets.py:867)
盒子放错位置	直接对整个蛋白质求质心	优先级链 + 工具预测验证；apo 受体自动改用 P2Rank top-1 (README.md § 11.2b)
大库跑不完	全库高搜索强度硬跑	两阶段漏斗 + pilot 预算护栏 + 并发规划 (core/params.py:372、core/docking.py:86)
结果不可信	模型自己编数字	数值只来自 RDKit / Vina / P2Rank 工具返回；verify_docking.py 独立复算取证
特殊化学被静默丢弃	直接丢掉全部 HETATM、把盐丢给 Vina	逐条上报 dropped_hetatm/kept_hetatm/unreported_hetatm，配体拆分给 dock_smiles+removed_fragments (docs/api.md § 16)
意图含混时行为摇摆	提示词里两条规则互相冲突	受理层显式给 decision，编排层只执行 (docs/architecture.md § 4.1)

1.2 六条能力目标与验收标准

来源：docs/architecture.md § 1.1（原表照录）。

表 3 六条能力目标与验收标准

#	能力	验收标准（可观测）	现状落点
G1	自动准备	用户只给 PDB/小分子文件即可开跑；盒子由工具确定而非臆造	core/ receptors.py 、 core/ ligands.py 、 core/ pockets.py
G2	自动对接	全部候选分子都有真实引擎返回的能量；大库能在有限算力内跑完	core/ docking.py 、 agents/ coordinator 漏斗纪律
G3	自动评估	物化性质/类药性/结合模式与对照比较，全部来自工具真实计算	core/ chemistry.py 、 tools/ {properties,binding}.py
G4	自动异常处理	单分子失败不拖垮整批；失败有原因；特殊体系有事实披露与补救建议	runtime/ errors.py 、 core/ docking.py 的 note/降级 路径
G5	总结与报告	固定 9 章报告 + 机器可读产物（CSV/JSON/图/位姿），数值可回溯到工具输出	reporting/ 、 agents/ persistence.py、 runs.py
G6	主管协调	受理判定 run/ask/reject；分发、并行、漏斗、分片由主管决定；流程不中断	intake.py 、 agents/ coordinator.py 、 agents/ dispatch.py

1.3 明确的非目标

来源：docs/architecture.md §1.2。

- 不做分子生成 / 骨架跃迁 / QSAR 建模：本系统是「筛选与评估」而非「设计」。
- 不做分布式集群调度：目标是单机多进程把多核吃满，不引入 Redis/Celery/K8s。
- 不做自研对接引擎 / 打分函数：只做 AutoDock Vina / AutoDock4 的编排与取证。
- 不做多用户 SaaS：默认只听 127.0.0.1，无鉴权（config.py 与 web/app.js 均按本机单用户假设）。
- 不引入向量库 / RAG：任务输入是结构化的分子与受体，不需要语义检索。

1.4 两条贯穿全局的硬约束

表 4 两条贯穿全局的硬约束

约束	含义	代码落点举例
真实性	任何数值都必须来自工具真实返回；工具失败要如实报错，禁止编造、禁止「看起来跑通了」的假成功	<code>verify_docking.py</code> 有专门的反证用例（非法 SMILES 必须不出数值）
不静默	任何被丢弃/改写/降级的东西都必须在结果里逐条可见，并给出补救路径	<code>dropped_hetatm</code> / <code>removed_fragments</code> / <code>unsupported_hetatm</code> (docs/api.md § 16)

2. 总体架构

结论：分层单向向下，受理与编排分离，两条执行路径共用同一套计算核心与报告模板。

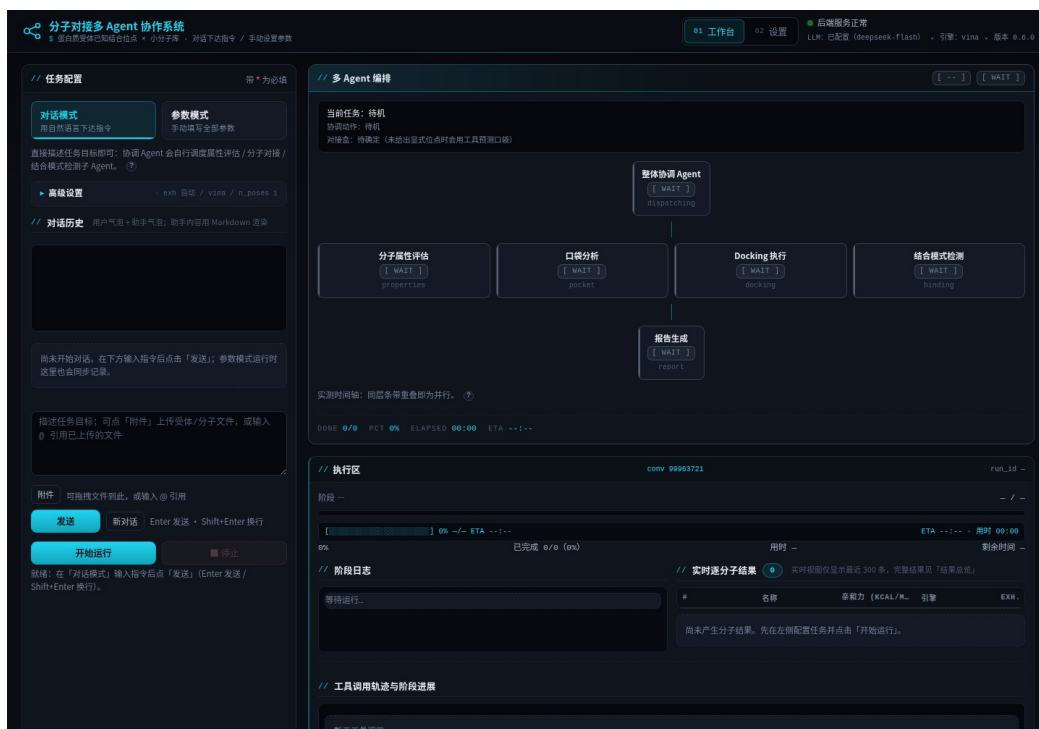


图 1 工作台全貌（对话模式首屏：左「任务配置」、中「多 Agent 编排」实时编排图、下「执行区」）

2.1 分层图

来源：[docs/architecture.md](#) §2.1（原图照录）。

入口层	cli.py / __main__.py (--check、pipeline、agent、runs、serve)
受理层	api/app.py (FastAPI, HTTP + SSE, 31 个路由) + api/schemas.py (pydantic 校验)
编排层	intake.py (用户指令 + 表单 → 结构化 task_spec; decision ∈ run/ask/reject) agents/coordinator.py (主管 Agent, LangGraph create_agent) agents/dispatch.py (发给子 Agent 的分发工具) + pipeline.py (确定性流水线, 零 LLM)
角色层	agents/workers.py (property / pocket / docking / binding, 各自独立模型实例与 checkpointner) agents/prompts.py (子 Agent 系统提示词)
工具层	tools/{docking,properties,binding,pockets,report,online,choices}.py (@tool 共 20 个, 模型唯一动作面)
核心计算	core/{docking,receptors,ligands,pockets,chemistry,ranking,params,resolve,files}.py (不依赖 LLM)
状态持久化	runtime/blackboard.py (运行内共享黑板) · runtime/tool_io.py (原始返回落盘 + 模型摘要视图) agents/persistence.py (唯一结果合并/落盘入口) · runs.py (运行目录、产物、分页、zip)
报告层	reporting/{report,charts,tables,content,pdf,artifacts,store}.py
运行时	runtime/{llm,streaming,errors,checkpoints,context,payload}.py settings.py (界面设置) · config.py (环境变量) · paths.py · cancellation.py · logging_setup.py

依赖方向单向向下：api/cli → intake/coordinator/pipeline → tools → core。
core/* 绝不 import tools/*、agents/*、api/*（唯一例外是

`core/receptors.py` 在函数内延迟导入 `core/pockets.py` 以避免循环依赖，`docs/architecture.md §2.1`）。实测模块行数分布见封面信息表。

2.2 一次运行的端到端数据流（含 SSE 事件）

来源：`docs/architecture.md §2.2`，SSE 事件类型取自 `docs/api.md §2/§7.3/§15`。

```

用户/前端 POST /api/agent/stream (或 /api/pipeline/stream)
▼ api/app.py 建 Run (runs.py: 运行目录 + run.json)
├─ intake.build_task_spec(req)          确定性规则（零模型）
│   └─ 需要时 intake.refine_task_spec() 仅「对话模式 + 自然语言」调用一次 LLM
│       decision=ask/reject → 直接回复，不调用任何工具（省算力、防胡说）
▼ coordinator (主管 Agent, LangGraph)
│   提示词注入 task_spec 只读摘要 (intake.render_agent_message)
│   同一轮多个分发工具调用 → ToolNode 并发执行 → 实测并行
├─ agents/dispatch.py: import_molecule_library / run_property_assessment /
│   run_pocket_analysis / run_docking / run_binding_mode_analysis
▼ @tool (tools/*.py)：大结果 tool_io.record() 全量落盘 + 只回 summary/top;
│   横向协作写 runtime/blackboard.py
▼ core/* (真实计算)：RDKit 构象、meeko PDBQT、Vina 对接、P2Rank/几何法口袋、排序
▼ agents/persistence.py (唯一落盘入口; _PROVENANCE_KEYS 回填模型漏掉的字段)
▼ runs.py 产物: molecules/properties/docking/binding/pockets/result.json + ranking.csv
│   + report.md (固定 9 章) + report.pdf + charts/*.png + poses/*
▼ SSE (runtime/streaming.py, 每个事件带 ts) → 前端 web/app.js 渲染卡片/表格/编排时间轴

```

SSE 事件类型 (`docs/api.md §2`、`runtime/streaming.py`)：`start` / `progress` / `stage` / `molecules` / `molecule`（小库单条） / `token` / `thinking` / `tool_call` / `tool_result` / `update` / `choices` / `final` / `done` / `error` / `cancelled`。其中 `thinking` 承载模型推理（`reasoning_content`、Responses 风格 `summary`、`<thinking>` 标签、content blocks 的 `thinking/reasoning` 块），经 `reasoning_of()` / `split_thinking()` 从正文里抽出后 **单独下发**（走 `custom` 帧，不混进 `messages/partial`），两套界面都把它折叠成气泡内默认收起的「思考 · N 字」块——正文只留结论，避免推理刷屏；助手回执与报告块则 **全文直接显示**（不再按行数/字数默认折叠）。其中 `stage ∈ import | properties | pocket | docking | binding | report | done` (`docs/api.md §10.4`)。所有事件都带服务端毫秒时间戳 `ts` (`docs/api.md §15.1`)，前端据此计算每个节点的**真实起止时间**。

表 5 实测并行样例 (docs/api.md § 15.2, 真实运行 chat+advanced)

节点	起止 (相对起点)	与相邻节点
口袋分析	2.2 → 11.7 s	串行
属性评估	12.9 → 17.4 s	与对接重叠 4.29 s (并行)
对接执行	13.1 → 23.1 s	同上
结合模式	23.9 → 29.2 s	串行
报告生成	30.7 → 31.6 s	串行

2.3 两条执行路径

来源: docs/architecture.md § 2.3。

表 6 两条执行路径

路径	入口	是否用 LLM	用途	注意
Agent 模式	/api/agent/stream、-m agent	是 (主管 + 子 Agent)	真实产品路径: 能理解自然语言、自主决策、可报告过程	提示词是产品行为的一部分, 改提示词等于改行为, 必须测试
流水线模式	/api/pipeline/stream、-m pipeline	否	回归/基准/取证: 确定性、可比、无密钥也能跑	两种模式共用 core/ 与报告模板, 改了 core 两条路径都会变

共用部分: core/* 全部计算、reporting/* 全部报告与产物、runs.py 全部落盘与下载规范。差异部分: 只有「结论与建议」一节的来源不同 (Agent 文字 vs 规则生成), 以及漏斗/并发的触发方式 (Agent 路径由受理层把规划结果作为建议参数写进指令, 流水线路径由 pipeline.run_pipeline 在 $N \geq \text{AGENT_FUNNEL_MIN}$ 时直接走 dock_library, docs/api.md § 7.6) 。

3. Agent 体系

结论：6 个角色各持独立模型实例；主管只编排不计算；子 Agent 通过共享黑板横向协作；分发层有代码级边界校验与受体护栏。

3.1 六个角色与工具清单

来源：runtime/llm.py:42 (ROLES)、settings.py:48、docs/architecture.md § 4.1、agents/prompts.py。

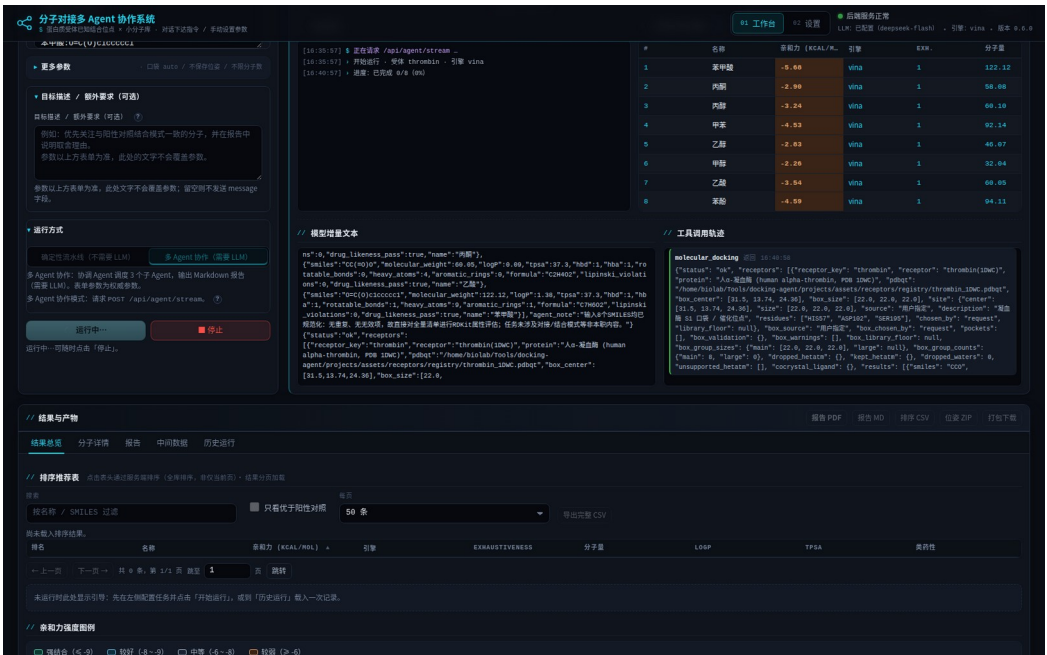


图 2 一次真实运行中的编排时间轴、实时分子结果与工具调用轨迹（SSE 推送渲染）

表 7 六个角色与工具清单

角色	作用	拥有的工具	提示词来源	默认温度
intake	任务受理：自然语言 → 结构化规约；不能给参数、不能编造分子	无（纯 LLM 补白名单字段）	intake.INTAKE_SYSTEM_PROMPT（intake.py:569）	0.0（config/agent_llm_config.json:24）
coordinator	主管：计划、分发、串并行、漏斗、汇总、推荐排行与理由、口袋说明与姿态结合分析、出报告	12 个（见下）	config/agent_llm_config.json 的 sp（全文 10,015 字符；按运行条件注入，常见小库运行 ≈5,763 字符）	0.2
property	分子属性评估	normalize_molecule_library、molecular_prope	agents/prompts.py:9	0.0

角色	作用	拥有的工具	提示词来源	默认温度
		rtty_assessment		
pocket	口袋分析：预测 → 与实验位点比对 → 提交盒子	predict_binding_pocket_s、compare_pocket_with_experiment、set_docking_site、list_pocket_engines	agents/prompts.py:97	0.0
docking	对接执行	molecular_docking、fetch_protein_structure	agents/prompts.py:31	0.1
binding	结合模式与对照比较	binding_mode_analysis、positive_control_similarity、check_binding_consistency	agents/prompts.py:71	0.0

主管绑定的 12 个工具（agents/coordinator.py）：
import_molecule_library、run_property_assessment、run_pocket_analysis、run_docking、run_binding_mode_analysis、recommend_compounds、submit_recommendations、customize_report、analyze_pose_pocket、generate_screening_report、fetch_protein_structure、fetch_molecule_record。（list_known_receptors 已从协调 Agent 工具面移除 —— 13 → 12；它现在是内部/诊断工具，没有任何 Agent 绑定。available_receptors 已删除。）

全部 @tool 共 23 个（实测：tools/ 下 @tool 装饰器计数）：dispatch.py 6、pockets.py 4、binding.py 3、properties.py 2、docking.py 2、online.py 2、report.py 1、recommend.py 2、pose.py 1。每个 `@tool` 的 docstring 就是给模型的说明书，因此 docstring 是接口契约的一部分（docs/architecture.md §3.4）。

条件纪律段（`agents/prompt\_blocks.py`）：系统提示词在每次模型调用上重发，而「执行纪律」占 sp 全文 55%，单次运行通常只用得到其中几条。因此 sp 用 <!-- block:KEY --> 标记出四段 —— 上传处理 / 受体纪律 3c / 两阶段漏斗 / 特殊体系 —— 协调 Agent 的 dynamic_prompt 中间件按本次运行事实抽掉不相关的段（事实来自 task_spec、param_plan 与 runtime/run_facts.py 记的对接观测）。安全方向写死：没有标记 → 全文、事实未知 → 注入、组装异常 → 全文。实测常见小库运行每次调用省 ≈ 2,549 tokens。提示词因此不再是常量，故每次注入清单随 result.prompt_blocks 落盘，可复盘「这次到底给了模型哪些纪律」。

为什么 intake 与 coordinator 分开：早期把「受理」和「执行」写在一个提示词里，「含糊就先问用户」与「分子非空就必须跑完」直接冲突，模型两头不讨好（docs/architecture.md § 4.1）。现在 decision 由受理层显式给出，编排层只执行。

3.2 分发链路与边界校验

来源：docs/architecture.md § 4.2、agents/dispatch.py。

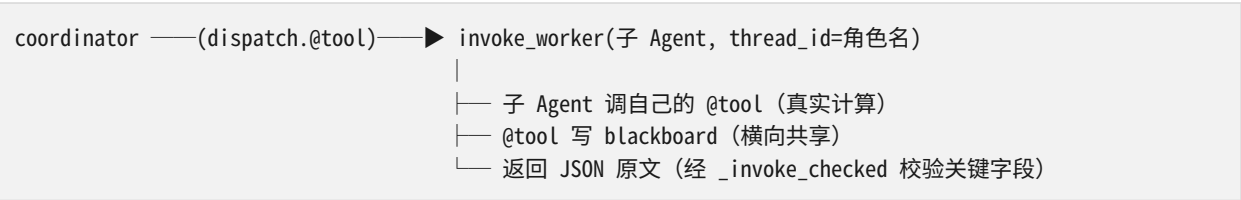


表 8 分发链路与边界校验

机制	实现	位置
每角色独立 LLM 实例	build_chat_llm(ctx, role=...) 每次都新建 ChatOpenAI 并登记 instance_id	agents/workers.py:48 、 runtime/llm.py:387
每子 Agent 独立 checkpointer	_checkpointer(role) 懒创建一个独立 MemorySaver	agents/workers.py:63
子 Agent 无状态	每次调用用 f"{thread_id}-{id(agent)}-{uuid4[:8]}" 作新 thread_id	agents/workers.py:159
必需字段校验	property→assessment ； pocket→pockets ； docking→receptors 或 summary （任一即可） ； binding→results	agents/dispatch.py:93-100
失败重试	带纠正提示自动重试一次（共 2 次尝试），纠正串在 dispatch.py:102	agents/dispatch.py:121-122
两次都失败的返回	{"status": "agent_output_invalid", "role": "...", "required_keys": [...], "raw": "<末尾 800 字符>"，不把垃圾文本喂给主管	agents/dispatch.py:137
同一轮多个分发调用	LangGraph ToolNode 并发执行 → 界面「实测并行」的来源	docs/architecture.md § 4.2
分片护栏	片数上限 AGENT_SHARD_MAX（默认 8），防止 LLM 往返开销反超对接本身	runtime/tool_io.py:147

表 9 代码级护栏（不靠模型自觉）

护栏	触发条件	行为	位置
受体未指定/未解析护栏	<code>task_spec.receptor.source ∈ {"unresolved", "default"}</code>	在调用任何子 Agent / 对接引擎之前返回 <code>{"status": "needs_user_input", ...}</code> ，并明确禁止改用任何默认受体	<code>agents/dispatch.py::unresolved_receptor_message</code> （ <code>run_docking / run_pocket_analysis</code> 均先调）
未指定受体不进入解析链	<code>receptor.source == "default"</code> 或受体参数为空	<code>molecular_docking / predict_binding_pockets</code> 在引擎调用前返回 <code>{"status": "needs_user_input", "missing": ["receptor"]}</code> ，不给预置受体候选； <code>resolve_receptor_specs</code> 对空参数直接 <code>raise ValueError</code> （不再有空值回退）	<code>tools/docking.py::_receptor_unspecified</code> 、 <code>tools/pockets.py</code> 、 <code>core/receptors.py::resolve_receptor_specs</code>
示例库不静默回退	未显式 <code>allow_example_fallback=True</code>	返回 <code>status="no_molecules"</code> ，不得擅自用示例库	<code>agents/dispatch.py:200</code>
分子库未指定	无 <code>molecules_json</code> 、无分子产物、黑板无分子	<code>run_property_assessment</code> 返回 <code>{"status": "needs_user_input", "missing": ["ligands"], ...}</code> ，不凭空评估	<code>agents/dispatch.py::run_property_assessment</code>

3.3 共享黑板（横向协作）

来源：`runtime/blackboard.py`、`docs/api.md` § 10.1。

表 10 共享黑板（横向协作）

区块	写入者	读取者	上限
<code>molecules</code> （规范化去重后）	分子属性评估 Agent	Docking 执行 Agent	快照保留 50 条
<code>properties</code>	分子属性评估 Agent	报告与复核	快照保留 50 条
<code>pockets / site</code>	口袋分析 Agent	Docking 执行 Agent（对接盒交接）	<code>pockets</code> 快照保留 20 条
<code>docking / receptor</code>	Docking 执行 Agent	结合模式检测 Agent（交叉核验）	快照保留 50 条

区块	写入者	读取者	上限
binding / positive_control	结合模式检测 Agent	复核	快照保留 50 条
notes	全部 Agent	界面展示（协作痕迹）	不设上限

- 快照上限是**字面量**（`blackboard.py:196` 的 `[:20]` 与 `blackboard.py:199-202` 的 `[:50]`），没有具名常量。
- `set_receptor` 不覆盖已固定的位点：`_site_pinned` 只在 `set_site` 里置真（`blackboard.py:89`），
`set_receptor` 仅在未固定时写 `site`（`blackboard.py:58`）。这就是「口袋 Agent 提交的盒子不会被受体默认位点覆盖」的机制。
- `stats()` 返回 `{"molecules","properties","docking","binding","pockets"}` 五个计数（`blackboard.py:187`），
即界面「协作记录」区展示的内容；`GET /api/runs/{id}` 的 `collaboration` 字段返回 `{"stats","notes"}`（`docs/api.md §10.1`）。
- 黑板是**进程内、单次运行隔离的**（`ContextVar`，`blackboard.py:207`）。

3.4 主管纪律条款摘要（`config/agent_llm_config.json` 的 `sp`，共 12 条）

来源：`config/agent_llm_config.json:28`（实测 `sp` 长 8,162 字符）。

表 11 主管纪律条款摘要（`config/agent_llm_config.json` 的 `sp`，共 12 条）

条目	要点	为什么
1	<code>decision=run</code> 时只要分子库非空就必须 完整执行 （导入→属性→对接→结合模式→报告），不得中途询问	产品底线
2	阳性对照是 可选项 ：只有明确给了 SMILES 才做对照对接与结合模式比较	修复「阳性对照总是使用默认值」
3	不得因参数缺失而中止；用任务参数默认值或注册位点	默认值体系
3b	位点盒必须由工具确定，不允许臆造坐标 ：没给坐标就先 <code>run_pocket_analysis</code>	修复「盒子放在蛋白中心」

条目	要点	为什么
3c	受体纪律 ：区分「没指定」(<code>decision=ask</code> ，只提问、零计算，系统没有默认受体) / 「点名了」(先自动解析) / 「解析失败或歧义」(列候选让用户选， 零计算)；点名但解析不了时 不得擅自 换任何默认受体	产品底线，代码级护栏兜底
4	对接必须覆盖分子库 全部 分子，不自行截断或抽样	完整性
5	最终必须调用 <code>generate_screening_report</code> 出报告	产物完整性
6	某一步失败要如实报告原因并继续其余可执行步骤	失败可见
7 / 9	有阳性对照时必须对对照也对接；对照对接由工具自动补入，不必单独再调	基线完整性
8	不要重复劳动：已返回过的结果不要再调一次	省算力
10 / 11	大库纪律 ： ≥ 500 走两阶段漏斗（全库粗筛 <code>exh=1</code> → 头部精算 <code>top_from_previous=200</code> , <code>exh=16</code> ）；子 Agent 工具一律留空分子参数；不按固定分子数切成很多片；分片不超过 <code>AGENT_SHARD_MAX</code> (8)	算力最优 + 防上下文爆炸
12	特殊体系（金属/辅因子/非标准配体）由 Agent 判断，工具只报事实 ：丢弃/保留/未保留都要说明；判断重要就 <code>keep_hetatm</code> 重跑；配体盐拆分与 <code>dock_smiles</code> 要写清	「不静默」硬约束

「位点盒由工具确定」「大库漏斗」「受体纪律 3c」「特殊体系交 Agent 判断」是四条最关键的纪律，其中 3c 与「未解析不计算」另有代码级护栏 (`agents/dispatch.py:47`)，不依赖模型是否照做。

4. 任务受理层 (intake)

结论：受理层是纯确定性规则（零网络、零模型即可跑），只在「对话模式 + 自然语言」时额外调用一次受理模型；它输出三态决策与结构化规约，编排层只读。

4.1 调用链

来源：docs/api.md §14.1。

```
POST /api/agent/stream
→ resolve 计数清零
→ thread_id = conversation_id or run_id
→ graph.aget_state(thread) 读最近 8 条 human/ai 作为 prior_turns (只读)
→ intake.build_task_spec(req, prior_turns=...) 确定性规则 (零模型；继承上一轮分子/受体)
→ [需要时] intake.refine_task_spec() 受理模型 (角色 intake, 温度 0; 上下文里带 prior_turns)
→ intake.render_agent_message(spec) 渲染成协调 Agent 的消息
→ run.data["task_spec"] = spec 写入运行记录 (可观测)
→ graph.astream(..., config={"configurable":{"thread_id": thread_id}}) 协调 Agent 按规约编排
```

4.2 确定性规约字段 (task_spec)

来源：intake.py:478-501、docs/api.md §14.2。

表 12 确定性规约字段 (task_spec)

字段	取值	说明
task_type	screening / properties_only / docking_only / binding_only / report_only / unknown	TASK_TYPES (intake.py:49)，由 _guess_task_type 从指令关键词判定 (intake.py:99)
goal / authority	用户目标或 DEFAULT_AGENT_TASK (intake.py:45) / manual • chat • chat+advanced	authority 由 mode + advanced 决定 (intake.py:347)
decision	run / ask / reject	由 _finalize 唯一推导 (intake.py:535)
receptor / site	{name, file, source} (source ∈ user/default/named/unresolved) / {center, size, source} (source ∈ user/tool)	见 §4.5；表单给了坐标就是 user，否则 tool (intake.py:455)
ligands / positive_control	{source, count, text, file,	见 §4.4；对照留空即跳过

字段	取值	说明
	molecules} (source ∈ text/file/message/library) / {smiles, provided_by} (provided_by ∈ user/none/skipped)	(intake.py:460)
params	engine/exhaustiveness/ n_poses/pocket_engine/ save_poses/max_ligands/ skip_positive_control	只透传这 7 个 (intake.py:488)
missing / assumptions / questions	记录用	`missing` 不构成阻断 (intake.py:538)
needs_llm / confidence / source	只有「对话模式 + 有自然语言」才为真 (intake.py:474) ; 规则路径 confidence=1.0, 需模型时初始 0.6 (intake.py:499) ; source ∈ rules/rules+llm	参数模式零额外调用

4.3 run / ask / reject 判定规则

来源：intake.py:510 (_can_ask) 、 intake.py:535 (_finalize) 。

表 13 run / ask / reject 判定规则

decision	成立条件	编排层行为
reject	out_of_scope == True (受理模型判定闲聊/无关)	友好说明能力范围， 不调用任何工具
ask	① receptor.source ∈ {"unresolved","default"} (点名未解析 / 未指定受体， 优先于多轮守卫 ；系统没有默认受体) ；或 ② 规约里没有任何可识别分子来源且未明确同意用示例库 (含上一轮继承)	只说明缺什么并提问， 不调用任何工具
run	其余全部情况 (受体与分子库都已指定)	必须完整跑完 ；唯一例外是 receptor.source == "named" —— 必须先自动解析

关键设计：必需信息不齐 → 只提问、零计算。受体与分子库是必需项（未指定受体 / 点名未解析 / 无分子来源都判 ask），系统**没有默认受体**，分子也不会静默用示例库；对照与对接参数仍有默认值，位点由口袋工具确定。这条规则来自实测：
`scripts/intake_eval.py --live` 曾抓到受理模型把 5 个本该 run 的用例判成 ask（`README.md §1`「任务受理层与编排层的分工」注）；现在反过来——必需项缺失必须 ask，回归用例两侧都看护。受理模型置的 `needs_user_input` **不会覆盖** 一个已完整指定（受体+分子库）的规约。

4.4 分子抽取（三种写法）

来源：`core/ligands.py:52`（`extract_smiles`）、`intake.py:289`、`docs/api.md §14.3b`。

表 14 分子抽取（三种写法）

写法	示例	处理
名称:SMILES	阿司匹林: <chem>CC(=O)Oc1ccccc1C(=O)O</chem>	直接切分（ <code>core/ligands.py:37 _split_candidates</code> ）
名称 空格 SMILES	华法林 <chem>CC(=O)CC(c1ccccc1)c1c(O)c2ccccc2oc1=O</chem>	按「含空白」的词法切分并回填名称
散文中夹带的分子	「帮我筛一下：咖啡因 <chem>Cn1cnc2c1c(=O)n(C)c(=O)n2C</chem> 」	抽取 SMILES 片段，散文被忽略

受理顺序（`intake.py:373-395`）：表单文本 → 上传/路径文件 → **指令中确定性抽取**（`source="message"`）→ 上一轮继承（`inherited="prior_turns"`）→ 最后才轮到示例库（`source="library"`，**仅在用户明确要求用时才真正加载**）。「**用户写了分子却对接了示例库**」是真实发生过的缺陷：旧分词只按 `,;\n` 切分，「名称 空格 SMILES」整句被当成一个 SMILES 解析失败 → 静默退回内置示例库，用户看到的是别人的分子被对接打分（`CHANGELOG.md` 第 24 条）。现已有回归用例看护。

4.5 多轮会话（`conversation_id`、`prior_turns`、`_can_ask` 例外）

来源：`docs/api.md` §2 「多轮会话」、`intake.py:303`、`intake.py:320`、`intake.py:510`。

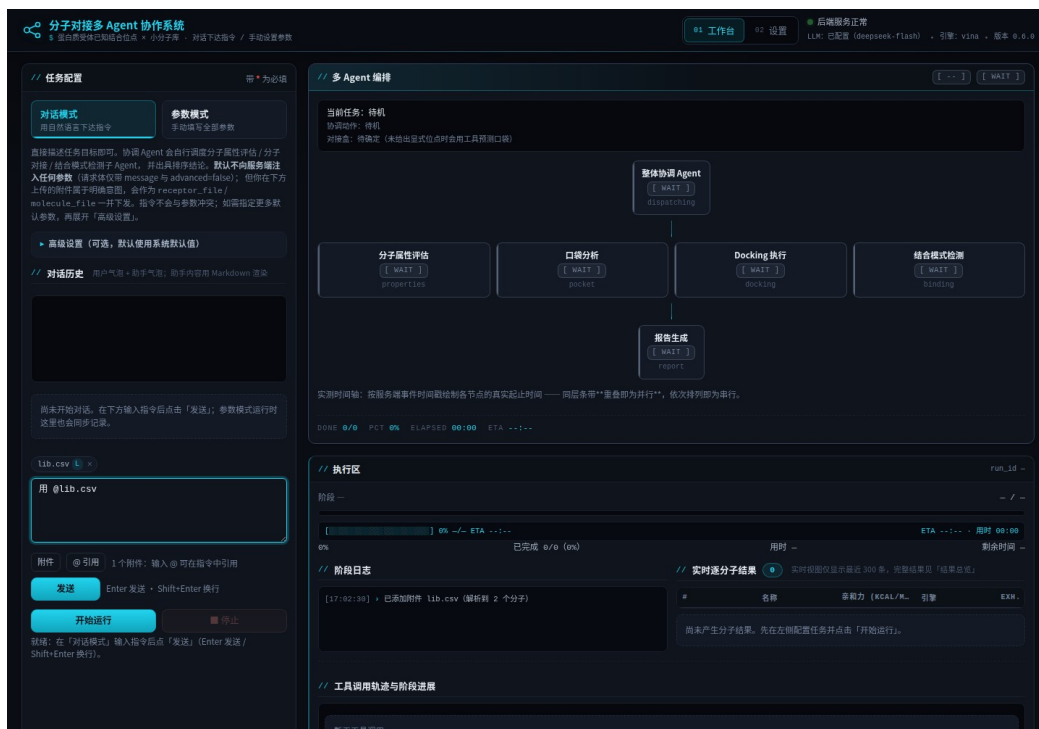


图 3 对话模式下的附件上传与 @ 引用（点选附件后以同一 `conversation_id` 续问）

表 15 多轮会话（`conversation_id`、`prior_turns`、`_can_ask` 例外）

机制	规则	位置
会话标识	<code>conversation_id</code> 直接当 LangGraph 的 <code>thread_id</code> ；留空则回落本次 <code>'run_id'</code> （每次独立，向后兼容）	<code>docs/api.md</code> §2
历史注入	读最近 8 条 human/ai 作为 <code>prior_turns</code> （只读）	<code>docs/api.md</code> §14.1
确定性继承（分子）	<code>_extract_req_molecules(prior_user_text)</code> or <code>_extract_req_molecules(prior_all_text)</code>	<code>intake.py:369</code>
确定性继承（受体）	先看本轮；再看上一轮提到的受体名；再看上一轮恰好只有一个已解析的 UniProt accession	<code>intake.py:426-442</code>
<code>prior_turn_count</code>	写入规约，供可观测	<code>intake.py:497</code>

机制	规则	位置
multi_turn_answer	本轮是在回答上一轮且不需要补充时置真	intake.py:560
`_can_ask` 的多轮例外	情形 ① (unresolved / default) 优先于多轮守卫 —— 哪怕本轮是在回答追问，也照样停下问，绝不能因为「上一轮问过」就把受体悄悄换成默认值或继续算	intake.py:528
多轮守卫（只对情形 ②）	有 prior_turns 且本轮 message 非空时不再判 ask，除非模型仍要求补充且两轮都拿不到任何分子来源	intake.py:522-526
持久化边界	默认 CHECKPOINT_BACKEND=memory，记忆只在进程内；服务重启后多轮上下文丢失（运行记录不受影响）	docs/api.md § 2
可观测	GET /api/runs/{id} 的 run.conversation_id / run.thread_id；SSE start 事件同带这两个字段	docs/api.md § 2

实测两轮样例 (var/e2e/)：第一轮 ros1-1 解析出 Q9SJQ6 并因分子是多组分而给出 choices；第二轮同一 conversation_id、消息只带分子代表结构，受体自动继承上一轮的 accession 并继续对接 (var/e2e/final_turn2.sse 中 llm_assumptions 明确写了「受体沿用上一轮已成功解析的结果」)。

4.6 受体三态纪律（核心）

来源：intake.py:397-453、intake.py:978（docs/api.md § 14.1b）、config/agent_llm_config.json 第 3c 条。

表 16 受体三态纪律（核心）

情形	receptor.source	判定	系统行为
完全没提受体	default	ask	零对接计算：只提问并给出三条出路（① PDB 编号 / UniProt accession；② 受体基

情形	receptor.source	判定	系统行为
			因/蛋白名（中英文，在线解析）；③上传受体结构文件）。提问文案里不给示例编号（助手回复中的示例编号曾被下一轮误当成用户指定）。系统没有默认受体（DEFAULT_RECEPTOR_NAME已删除），不给预置受体候选
PDB 号 / UniProt accession / 受体名称 / 上传受体文件	user	run	按用户指定的受体执行
点名了具体受体，但字面不是 PDB/accession（如「植物去甲基化酶 ROS1」「EGFR」）	named	run	第一步调用`fetch_protein_structure`自动解析；高置信唯一候选 → 继续完整流程，报告写明 accession/物种/蛋白名/结构来源/打分依据
检索到多个同样合理的候选，或单一候选置信度不足	unresolved（工具写回）	ask	零对接计算：把候选列成choices（accession / 物种 / 蛋白名 / 结构来源 / 打分），用户点选后以同一 conversation_id 继续
一个都没查到	unresolved（工具写回）	ask	零对接计算：提问并逐条列出已尝试的检索，请用户提供 PDB ID / accession / 文件

代码级兜底： unresolved_receptor_message(run) 会在任何子 Agent / 引擎工作之前 拦下 receptor.source == "unresolved" （返回 {"status":"needs_user_input","attempts":[...],"candidates":[...],"message":"..."} ） 与 receptor.source == "default" （返回 {"status":"needs_user_input","missing":["receptor"],...}，不给任何预置受体候选）两种情形，run_docking / run_pocket_analysis 都会先调它 —— 即使主管 Agent 没照提示停下来问，也**绝不可能**用默认受体把结果算出来。写回动作由 tools/choices.py::mark_receptor_unresolved 完成（tools/choices.py:29）。

表 17 实测对照 (var/e2e/evidence_resolve.out、var/e2e/final_ambig.sse)

输入	解析状态	结果
「植物去甲基化酶 ROS1」 (含物种线索)	resolved	Q9SJQ6 / Arabidopsis thaliana / DNA glycosylase-AP lyase ROS1 / 打分 114.0 → 自动继续
只写「ROS1」 (无物种线索)	ambiguous	7 个候选并列 64.0 (人源激酶 P08922、小鼠 Q78DX7、大鼠 Q63132 、 拟 南 芥 Q9SJQ6...) → 零对接，下发 4+ 个 choices
「不存在的蛋白 ZZZQQ9」	not_found	零对接，回执逐条列出 3 次已尝试检索，并给出三条出路

5. 在线解析链（resolve）

结论：名称归一化 → 物种线索 → UniProt 多策略检索 + 规则打分 → 三态判定 → RCSB/AlphaFold 结构获取 + 全字段溯源；所有「不确定」都变成结构化 `choices` 交回用户。

5.1 名称归一化与中英映射

来源：`core/resolve.py:179`（`normalize_molecule_name`）、`core/resolve.py:189`（`extract_gene_tokens`）、`core/resolve.py:212`（`map_family_terms`）、`core/resolve.py:228`（`normalize_query`）。

表 18 名称归一化与中英映射

步骤	做什么	例子
基因 token 抽取	从「植物去甲基化酶 ROS1」抽出 <code>ROS1</code>	<code>extract_gene_tokens</code> （ <code>core/resolve.py:189</code> ）
指示代词/动词剥离	剥掉「请把/这个/该」等	<code>intake.py:238</code> <code>_strip_demonstrative</code>
泛指排除	「未指定/默认/某个/其它/任意受体」不算点名	<code>intake.py:187</code> <code>_RECEPTOR_NON_NAME_MARKERS</code>
物种线索	中文/英文 → <code>taxid + is_plant</code>	<code>extract_species</code> （ <code>core/resolve.py:201</code> ）
中文家族词 → 英文	去甲基化酶 → <code>demethylase</code> / <code>DNA glycosylase</code> ；激酶 → <code>kinase</code> ；受体 → <code>receptor</code>	<code>map_family_terms</code> （ <code>core/resolve.py:212</code> ）
受体同义词	凝血酶 → <code>thrombin</code> 、人 α -凝血酶 → <code>thrombin</code> 等	<code>intake.py:180</code> <code>_RECEPTOR_SYNONYMS</code>

表 19 物种线索表（`core/resolve.py:92-114`，节选）

线索	物种	taxid	是否植物
植物（模式植物拟南芥）	<code>Arabidopsis thaliana</code>	3702	是
拟南芥 / 水稻 / 玉米 / 大豆 / 烟草 / 番茄	对应物种	3702 / 39947 / 4577 / 3847 / 4097 / 4081	是

线索	物种	taxid	是否植物
人类 / 小鼠 / 大鼠 / 酵母 / 大肠杆菌	对应物种	9606 / 10090 / 10116 / 559292 / 562	否

5.2 UniProt 多策略检索

来 源 ： `core/resolve.py:447` （ `build_uniprot_queries` ） 、 `docs/api.md` § 14.1c。

按精度从高到低依次构造检索式（去重后依序尝试）：

表 20 UniProt 多策略检索

顺序	策略名	检索式
0	<code>accession</code>	直接 accession / entry ID 直查 (有则最高优先)
1	<code>gene+species</code>	<code>gene_exact:<GENE></code> AND <code>organism_id:<taxid></code>
2	<code>gene+reviewed</code>	<code>gene_exact:<GENE></code> AND <code>reviewed:true</code>
3	<code>gene</code>	<code>gene_exact:<GENE></code>
4	<code>family+species</code>	(< 家 族 词 >) AND <code>(organism_id:<taxid>)</code>
5	<code>family+plant</code>	(< 家 族 词 >) AND (<code>plant</code> OR <code>Arabidopsis</code>)
6	<code>free-text</code>	英文自由文本（基因 + 家族词 + 物种名）

泛化家族词（`protein` / `enzyme` / `receptor` / `channel`）不用于构造检索式：`(receptor) AND reviewed:true` 会命中成百上千条（`core/resolve.py:160-162` `GENERIC_FAMILY_TERMS`）。

5.3 打分规则（逐条列出）

来 源 ： `core/resolve.py:338` （ `score_candidate` ） 与 模 块 `docstring`（`core/resolve.py:16-27`）。

表 21 打分规则（逐条列出）

项	分值	说明
Swiss-Prot 已审阅	+40	人工审阅条目优先于 TrEMBL
物种精确匹配	+30	候选 taxid == 线索 taxid（如拟南芥 3702）
同为植物	+18	线索是植物但候选是另一种植物
物种不符	-25	线索是植物却命中人源/鼠源等
蛋白名含映射家族词	+20	如 demethylase / DNA glycosylase / kinase
蛋白名不含家族词	-10	只在用户给了家族词时扣
基因名精确匹配	+15	gene_names 里逐字等于线索基因（ROS1 ≠ ROS1A）
注释完整度	+3×3	蛋白名 / 基因名 / 物种三项各+3

表 22 三态判定阈值与状态（core/resolve.py:60-63、decide_status core/resolve.py:378）

常量	值	作用
MIN_ACCEPT_SCORE	25.0	最高分低于此 → not_found
AMBIGUITY_MIN_SCORE AMBIGUITY_MARGIN	/ 45.0 / 8.0	次高分 ≥ 最高分 - 8 且 best ≥ 45 且两者物种不同 → ambiguous
LOW_CONFIDENCE_SCORE	55.0	单一候选低于此 → low_confidence；另要求已审阅、物种匹配、基因精确、家族词匹配

表 23 打分规则（逐条列出）

状态	含义	上层行为
resolved	唯一候选且所有用户给出的线索都强匹配（精确 accession 本身即权威，不再推断）	自动继续
ambiguous	多个分数接近、跨物种无法消歧	列候选让用户选
low_confidence	单一候选但置信度不足（未 reviewed / 物种不符 / 名称只部分匹配）	列出让用户确认

状态	含义	上层行为
not_found	一个都没查到	逐条列出已尝试检索

5.4 RCSB → AlphaFold 结构获取链与溯源字段

来源：`tools/online.py:324`（`fetch_protein_structure`）、`docs/api.md § 14.1c`。

```
名称(含中文) → normalize_query (基因 token / 物种线索 / 中文家族词→英文)
→ 多策略检索 UniProt (§5.2) → 打分 (§5.3) → 三态判定
→ 结构链: UniProt PDB 交叉引用 / RCSB Search 反查的实验结构优先;
    没有实验结构、或候选结构都准备失败 → AlphaFold DB (用接口返回的 pdbUrl, 模型版本不写死)
→ prepare_user_receptor 生成 PDBQT (含 altloc 重试与内容哈希缓存)
→ 溯源: accession / entry_id / 物种 / 蛋白名 / structure_source(rcsb|alphafold) / structure_url /
    下载文件与大小 / AlphaFold 模型版本 / 打分理由 / 已尝试检索
```

返回 JSON 关键字段（成功路径，`docs/api.md § 14.1c`）：
`receptor_file`、`box_center`、`box_size`、`structure_source`、`pdb_id`（RCSB）或 `alphafold`（预测模型）、`accession`、`organism`、`protein`、`score`、`score_reasons`、`why_selected`、`structure_url`、`structure_file`、`file_size`、`receptor_resolution.attempt_summary`、`candidates`、`provenance`。

候选结构准备失败会自动换下一个候选（`README.md § 9`），并在 `attempts` 里留痕。

实测（``var/e2e/evidence_resolve.out``）：`植物去甲基化酶 ROS1` 的 6 条检索依次命中 1 / 7 / 8 / 8 / 8 / 8 条，最终选中 Q9SJJQ6（打分 114.0 = 40 + 30 + 20 + 15 + 9），`pdb_ids` 为 7YH0 / 7YHP / 7YHQ；只写 `ROS1` 时前三条检索分别命中 7 / 8 / 8 条，7 个候选同为 64.0 → `ambiguous`。

5.5 choices 结构化选项协议

来源：`tools/choices.py:1-16`、`docs/api.md § 7.3`、`intake.py:146`。

选项结构： {id, kind, label, value, prompt, detail}, kind ∈ receptor | molecule。

表 24 choices 结构化选项协议

字段	含义
id	稳定标识，如 receptor:Q9SJQ6 / molecule:3034368:organic
kind	受体或分子
label	给人看的完整描述（accession / 物种 / 蛋白名 / 结构来源 / 打分）
value	机器可用值（accession / CID / SMILES）
prompt	点选后 原样发出 的下一轮追问
detail	结构化细节（物种、蛋白名、结构来源、pdb_ids、打分、理由）

- 下发通道：SSE choices 事件 + run.data.choices + GET /api/runs/{id} 的 run.choices，
同一次运行只发一次（心跳路径与 final 前各有一条触发路径，已去重，docs/api.md §7.3）。
- 前端渲染成助手气泡下方的可点按钮；点选后以同一 `conversation\_id` 发送 prompt。
- 高置信解析成功时**不会出现** choices（直接继续，不打扰用户）。
- 陈旧清理：clear_choices(kind) 只清**同类型**的旧候选项 —— 同一运行里「分子是混合物要选、受体已高置信解析」是正常组合
(tools/choices.py:92)。

分子多组分/混合物的处理（以 Mancozeb 为例）： fetch_molecule_record 命中多组分/配位聚合物时 **不臆造单一结构** (tools/choices.py:146 mixture_choices)，返回原始 SMILES、is_mixture、components、representative_smiles 与 mixture_note，并给出 4 个选项：

表 25 choices 结构化选项协议

选项	取法	值（实测， <code>var/e2e/final_turn1.sse</code> ）
① 原始多组分	PubChem 原始多组分结构，金属与配体片段原样保留	<code>C(CNC(=S)[S-])NC(=S)[S-].C(CNC(=S)[S-])NC(=S)[S-].[Mn+2].[Zn+2]</code>
② Zn-EBDC 单体	金属 + 最大有机片段	<code>S=C([S-])NCCNC(=S)[S-].[Zn+2]</code>
③ Mn-EBDC 单体	金属 + 最大有机片段	<code>S=C([S-])NCCNC(=S)[S-].[Mn+2]</code>
④ 最大有机片段 EBDC	无金属代表结构	<code>S=C([S-])NCCNC(=S)[S-]</code>

- 正确 CID 与 分子式：**PubChem CID 3034368** / **C₈H₁₂MnN₄S₈Zn** (`var/e2e/final_turn1.sse` 的 `choices[0].detail`)。
- 每个选项的 `detail.note` 都如实提示「对接引擎通常不支持金属配位，实际分数需谨慎解读」。
- 中文别名映射：代森猛锌 / 代森锰锌 → Mancozeb (`README.md § 9`)。

6. 受体与配体准备

结论：受体准备去水去杂原子但逐条溯源，`keep\_hetatm` 可要求保留并按最终 PDBQT 内容回填；配体准备用 ETKDGV3 + MMFF 生成 3D 构象，并在对接前做一次确定性「化学体检」。

6.1 受体准备：去水 / 杂原子溯源

来源：`core/receptors.py:178`（`_write_prep_pdb`）、`core/receptors.py:235`（`prepare_user_receptor`）、`docs/api.md` § 16.1。

表 26 受体准备：去水 / 杂原子溯源

字段	含义	实测样例
<code>dropped_hetatm / dropped_waters</code>	按标准流程剔除的非水杂原子计数（残基名 → 数量） / 剔除的水分子数	<code>{"5CM": 20}</code> 、 <code>{"HEM": 43, "S04": 10}</code> / 0
<code>kept_hetatm</code>	最终受体 PDBQT 里实际存在的白名单残基计数（以文件内容为准，不是「打算保留」）	<code>{"SF4": 8}</code>
<code>unsupported_hetatm</code> <code>cocrystal_ligand</code>	/ 要求保留但因缺少 meeko 模板未能进入受体的残基 / 用于定位位点盒的共晶配体（ <code>resname/n_atoms/center</code> ）	<code>[]</code> 或 <code>["HEM"] / 5CM</code>

- 实测（本次运行）：受体 7YHP 的 `dropped_hetatm = {"5CM": 20}`、`kept_hetatm = {"SF4": 8}`、
`unsupported_hetatm = []`（`var/runs/20260915-174954-6983/docking.json`）；
对应的 note 为「受体 7YHP 丢弃了非水杂原子：5CM×20（水 0 个）…」（`var/runs/20260915-174954-6983/run.json` 的 `notes`）。
- 实测（金属离子 vs 大辅因子）：`ZN/MG/CA/FE/MN/CL` 等金属离子可直接保留；
`HEM`、`NAD`、`GOL` 等有机残基需要 meeko 模板，否则计入 `unsupported_hetatm`（`docs/api.md` § 16.1）。

- 保留失败不会崩：实现会逐个剔除无法参数化的残基直到准备成功，
把剔除名单放进 `unsupported_hetatm`，并给出三条补救路径（去掉 `keep_hetatm` 重跑 / 提供模板 SDF / 直接给已备好的 PDBQT）（`core/receptors.py:291-325`）。
- ``kept_hetatm`` 以最终 PDBQT 内容为准：`core/receptors.py:338` 明确「绝不出现『说保留了、其实没进对接』的假信息」。
- 共晶配体识别：按（链:残基号:残基名）取最大的一团非水 HETATM，并排除离子与结晶添加剂
（SO4/GOL/EDO/PEG...）（`core/pockets.py:345 cocrystal_ligand`）。位点来源会写清是哪个配体，如「共晶配体(MIT)质心」。

6.2 keep_hetatm 与 「已准备好的 PDBQT」

来源：`core/receptors.py:537`（`_pdbqt_spec`）、`README.md §11.4c`。

表 27 keep_hetatm 与 「已准备好的 PDBQT」

场景	行为
受体是 <code>.pdb</code>	现场用 <code>meeko mk_prepare_receptor.py</code> 准备（ <code>core/receptors.py:206</code> ）
受体是 <code>.pdbqt</code> ，未请求保留杂原子	直接使用该 PDBQT
受体是 <code>.pdbqt</code> ，请求了 <code>keep_hetatm</code>	回到 <code>.site.json</code> sidecar 记录的来源 PDB 重新准备（ <code>core/receptors.py:552-562</code> ）
<code>.pdbqt</code> 且无来源 PDB	无法谈保留，如实说明（杂原子在上传准备时已被剔除）

实测：1A6M 上传的 PDBQT + `keep_hetatm="OXY"` 的分数从 **-6.97 变为 -7.15**，OXY 真的进入了受体（`README.md §11.4c`）。本次实测中，7YHP 的 Docking Agent 自动补做了一次 `keep_hetatm=SF4` 的重跑：保留 SF4 时 **-4.28**，未保留时为 **-4.25**，差异在噪声范围内（`var/e2e/final_turn2.sse` 的最终回复第 6 节）。注意：这一对比来自 Agent 运行内的两次真实对接，其「噪声范围内」的判断是模型叙述，报告层面只把两个数值当作实测分数引用。

6.3 altloc 重试、meeko 准备链与内容寻址缓存

来源：`core/receptors.py:172-176`、`core/receptors.py:148`、`core/receptors.py:206`、`core/receptors.py:269-290`。

表 28 altloc 重试、meeko 准备链与内容寻址缓存

项	内容
altloc 重试	尝试顺序 <code>[["--default_altloc","A"], ["--default_altloc","B"], ["--default_altloc","C"], []]</code> 。交替构象是晶体结构准备失败的主因，仅用 <code>-a</code> 不够，必须显式指定默认 altloc；最后一次不带该参数（针对干净结构）。来自真实缺陷修复（CHANGELOG.md 第 11 条）
准备脚本定位	<code>mk_prepare_receptor.py</code> （ <code>venv/bin</code> 与控制台脚本两种安装方式都能定位）
缺失模板残基检测	<code>_parse_unmatched_residues</code> （ <code>core/receptors.py:93</code> ）解析 meeko 输出得到 unmatched 残基名，并入 <code>unsupported_hetatm</code> ，逐个剔除后重试（ <code>core/receptors.py:291-325</code> ）
PDBQT 规格	<code>_pdbqt_spec</code> （ <code>core/receptors.py:537</code> ）产出 <code>{pdbqt, center, size, site, dropped_hetatm, kept_hetatm, dropped_waters, unsupported_hetatm, ...}</code>
缓存键	源文件内容 sha1 前 10 位 + <code>keep_hetatm</code> 集合的 sha1 前 6 位
判定依据	准备后结构的内容哈希（ <code>.sha1</code> sidecar），不是时间戳—— <code>prot.pdb</code> 每次都重写，用时间戳会导致缓存恒失效
内容寻址的意义	两个并发运行（或两个同名上传）不会互相覆盖产物；「我请求保留 ZN，拿回别人的 HEM+ZN」的静默串数据已被消除
回归用例	<code>tests/test_receptor_race.py</code> ：3 线程并发准备同名受体，断言各自结果与请求一致、产物文件名互不相同（2 个用例）

6.4 配体准备：3D 构象与化学体检

来源：`core/ligands.py:258`（`smiles_to_pdbqt`）、`core/ligands.py:176`（`describe_ligand`）、`docs/api.md` §16.2。

表 29 配体准备：3D 构象与化学体检

步骤	参数 / 规则	位置
3D 构象生成	<code>AllChem.ETKDGv3()</code> ， <code>params.randomSeed = seed</code> （默认 <code>seed=42</code> ）；ETKDGv3 不可用	<code>core/ligands.py:265-269</code>

步骤	参数 / 规则	位置
	时回退 ETKDv2	
力场优化	AllChem.MMFFOptimizeMolecule(mol, maxIters=500) ; 失败只 logger.warning 不中断	core/Ligands.py:273
PDBQT 转换	meeko 写出配体 PDBQT	core/Ligands.py:258
化学体检	每个分子对接前执行一次确定性 describe_ligand	core/docking.py:191

化学体检字段（`ligand\_facts`）：片段数、净电荷、是否含金属、MW、重原子数、可旋转键、HBD/HBA、TPSA、cLogP、未定义手性中心数（[README.md § 11.4c](#)）。

表 30 化学体检告警 (ligand_warnings)

情形	处置	位置
多片段（盐/反离子/溶剂）	按 最大有机片段 对接，写明移除了谁与判定（如 移除：[Na+](反离子)），给 dock_smiles + removed_fragments	core/Ligands.py:205-215
质子化态（运行级策略）	ph（默认，按目标 pH 7.4）/ neutralize / keep；逐分子记录 policy/applied/charge_before/charge_after/method、 专业引擎名与版本 、候选微观态与选定规则（engine/engine_version/variants/variant_rule）， 原始 SMILES 始终保留 ，理化性质与对接同口径；排序 CSV 带 protonation_policy/charge_input/charge_used	core/protonation.py（apply_protonation）、core/ligand_pka.py、core/chemistry.py、settings.py（docking.protonation）
目标 pH 质子化（ ph 策略，v0.32）	优先专业 pKa 引擎 ：Dimorphite-DL（RDKit pKa 规则库）按目标 pH（窗口 LIGAND_PKA_WINDOW ，默认 ±0.5）枚举可电离官能团微观态；窗口内多微观态时按「\	净电荷\
受体质子化（与配体同 pH）	pdb2pqr --ph-calc-method=propka --with-ph=<pH> → PQR → meeko --read_pqr → 受体 PDBQT；逐残基留痕（HIS HID/HIE/HIP、	core/receptor_ph.py、core/receptors.py

情形	处置	位置
	各可滴定残基质子化计数、边界残基、剔除的主链不全残基)；缓存按「文件内容 + pH」内容寻址；工具缺失/有机辅因子 → 回退标准流程并标 applied=false+原因。 PQR→meeko 前做固定列规范化 ：pdb2pqr 按 PDB 固定列写出，插入码 (36A) 与 4 位残基号 (‘GLU A1005’) 都会让白空格切分粘连，meeko 会把 x 坐标当残基号 (int('128.990')) 而整条失败 → parse_pqr_atom_line() 统一拆开 (core/receptor_ph.py，回归见 tests/test_receptor_ph_parsing.py)	
含金属原子	给出告警	core/ligands.py:224
净电荷非零	告警「请确认质子化态」， 不自动加/减氢	core/ligands.py:229
未定义手性中心	告警「3D 构象是其中随机一种」	core/ligands.py:249
偏大分子	MW > 800 或重原子 > 60 → 告警「Vina 采样可能不充分」	core/ligands.py:239
SMILES 无法解析	返回 ok=False + error + ligand_warnings， 不静默跳过、不编造数值	core/ligands.py:192

主键不变量：smiles 始终是原始输入（作为合并/去重主键），改写后的对接用 SMILES 放在 dock_smiles（core/docking.py:204-207、docs/api.md §16.2）。

实测（本次运行）：配体「代森锰锌-EBDC」ligand_span = [8.38, 2.76, 4.03]，ligand_warnings 含「配体带净电荷 -2：请确认质子化态是否符合目标 pH」（var/runs/20260915-174954-6983/）。

7. 对接盒（重点，含实测）

结论：盒子由工具确定、一个受体只定一次、全库共用；库级配体感知下限保证大配体不被欠采样；真正超限的分子用同一中心的更大盒子单独成组，绝不逐分子换盒。

7.1 口袋检测：P2Rank 2.5.1 与内置几何法

来源：`core/pockets.py:404`（`available_engines`）、`core/pockets.py:437`（`run_p2rank`）、`core/pockets.py:563`（`detect_geometric`）、`README.md § 11.2/ § 11.2b`。

表 31 口袋检测：P2Rank 2.5.1 与内置几何法

引擎	原理	参数	实测（凝血酶 1DWC）
P2Rank 2.5.1	随机森林 + 溶剂可及表面特征（Java 程序，本地部署）	<code>P2RANK_HOME</code> （留空自动探测 <code>assets/tools/p2rank*</code> ）、 <code>P2RANK_THREADS</code> （留空 <code>cpu//2</code> ）	top-1 口袋中心距共晶配体 4.54 Å （ <code>score</code> 7.70， <code>probability</code> 0.407）；top-2 6.02 Å；与实验位点共享 8 个残基；单受体耗时 ≈ 7 s （10 线程）
内置几何法	表面层网格 → 埋藏度/包封度/疏水接触三特征打分 → 贪心球聚类 + 有限合并	网格 <code>spacing</code> 默认 1.0 Å	top-1 口袋中心距共晶配体 2.6 Å ；top-2 2.97 Å；top-1 附近残基 <code>GLU192</code> <code>TRP215</code> <code>GLU217</code> <code>CYS191</code> <code>GLY216</code> <code>GLY219</code> ；单受体耗时 ≈ 1 s （2283 个重原子）
引擎选择 / 质量门	<code>POCKET_ENGINE ∈ auto / p2rank / geometric / known_site</code> （默认 <code>auto</code> ）； <code>tests/test_pockets.py</code> （28 个用例）要求 top-3 内命中实验位点（≤8 Å）	未安装 P2Rank 时自动回退几何法并在 <code>attempts/warnings</code> 说明， 不静默降级	<code>settings.py:177</code> 、 <code>README.md § 11.2/ § 11.3</code>

「工具真的在定盒子」的端到端证据（`README.md § 11.2b`）：把凝血酶 PDB 的共晶配体删掉（apo 受体，系统只能得到「蛋白质质心」这个不可靠兜底），系统自动改用 P2Rank 预测的 `pocket1`：

表 32 口袋检测：P2Rank 2.5.1 与内置几何法

	盒子中心	距真实活性位点
旧行为（蛋白质心兜底）	41.52 / 21.84 / 19.17	13.2 Å
现在（P2Rank 预测）	32.89 / 11.85 / 19.37	5.52 Å

本次 e2e 实测（拟南芥 ROS1 / 7YHP）：口袋引擎 **p2rank**，选中 **pocket1**（score **31.55**，probability **0.939**，中心 **130.6469 / 132.8673 / 123.2747**，extent **14.33 / 14.71 / 14.54**），pocket2 score 8.14、pocket3 score 5.85（**var/runs/20260915-174912-4159/pockets.json**）。

7.2 盒子来源优先级

来源：**core/pockets.py:867**（select_site）、**README.md §11.1**。

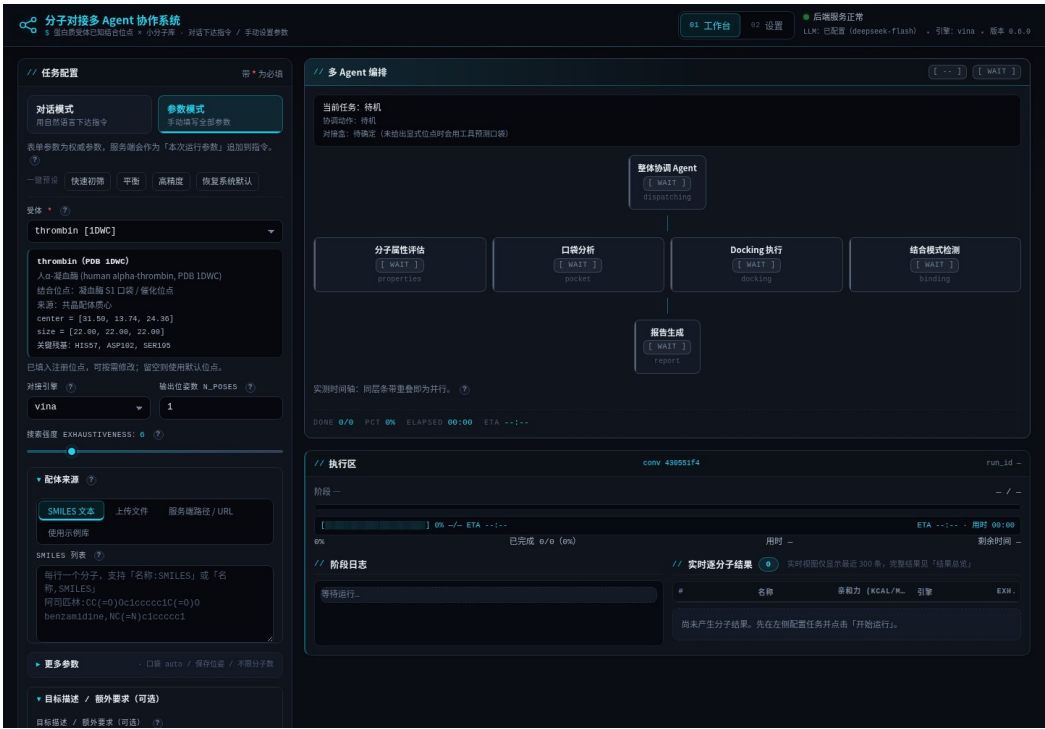


图 4 参数模式：受体与对接盒（center / size）的显式指定；留空则走工具定盒

表 33 盒子来源优先级

优先级	来源	说明	chosen_by
1	用户显式指定	表单中心/尺寸、指令里的坐标，或口袋分析 Agent 提交的盒子	request / pocket_agent
2	实验位点	注册表标注的已知位点，	experimental_site

优先级	来源	说明	chosen_by
		或用户上传受体里共晶配体的质心	
3	P2Rank 预测	成熟的口袋预测工具	pocket_prediction
4	内置几何法	无外部依赖的兜底	pocket_prediction
5	蛋白质心	以上都不可用时的最后兜底，box_source 明确标注「不可靠」	centroid

决策规则 (`core/pockets.py:918-963`) :

- 有实验位点时：以实验位点为准，同时用工具预测做**独立验证**——
一致（中心相距 $\leq \text{DEFAULT_AGREE_RADIUS} = 8.0 \text{ \AA}$ ，`core/pockets.py:46`）就写进溯源；**不一致则记录警告**并写入 `box_warnings`；
- 只有低可信兜底位点（蛋白质心，`trust != "experimental"`）时：**改用工具预测的 top 口袋**；
- 完全没有参考位点时：直接用工具预测的 top 口袋；
- 工具不可用则逐级回退，并把「实际使用的来源」如实写进 `box_source`。

盒子尺寸公式：`size_i = clamp(口袋 extent_i + 2 × padding, min_size, max_size)`，默认 `padding = 4 \AA`、`min_size = 18 \AA`、`max_size = 30 \AA`（`core/pockets.py:47-49`、`settings.py:185-190`）；Vina 网格间距 `0.375 \AA`（`docs/architecture.md §15.7b`）。

7.3 库级配体下限（C 方案 B1）

来源：`core/pockets.py:252`（`library_span_bound`）、`settings.py:191-198`、`docs/api.md §13.5`。

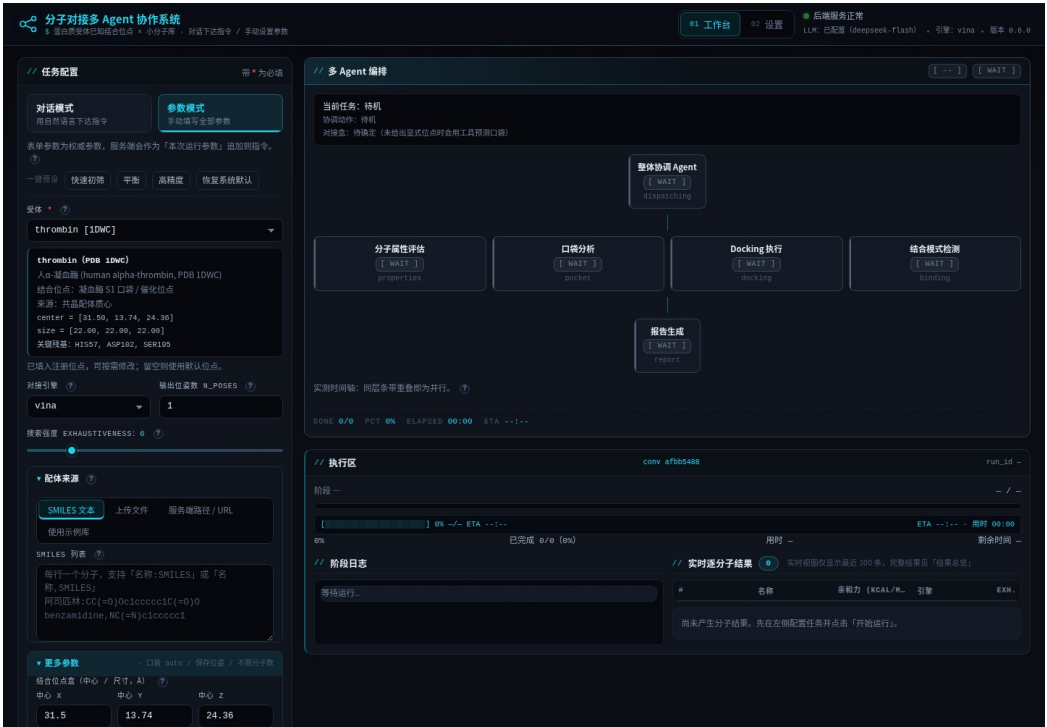


图 5 「更多参数」面板：结合位点中心 X/Y/Z 的显式输入（留空即由口袋工具确定）

表 34 库级配体下限（C 方案 B1）

项	规则	默认值
开关	<code>BOX_SPAN_ENABLED</code> ；off = 旧行为（纯口袋驱动）	on
抽样 K	按 2D 重原子数降序取前 K 个生成 3D；夹在 20–1000；库小于 K 时全算	200
统计量	P95(库内配体 3D 最大跨度)（最近秩法，抗单个异常值）	—
下限公式	$\text{lib_lower_bound} = \max(\text{POCKET_MIN_SIZE}, \text{round}(\text{P95} + 10, 1))$ ，其中 $+10 \text{ \AA} = 2 \times 5 \text{ \AA}$	—
与盒子的关系	$\text{size}_i = \text{clamp}(\text{pocket_extent}_i + 2 \times \text{padding}, \text{lib_lower_bound}, \text{max_size})$	—
缓存	按库 SMILES 的 sha1 前 12 位缓存到 <code>assets/cache/box_span/</code> ；缓存只存 P95，下限按当前 <code>min_size</code> 重算	<code>core/pockets.py:283-297</code>
降级	抽样/3D 生成失败一律退回	<code>core/pockets.py:312-318</code>

项	规则	默认值
	<code>POCKET_MIN_SIZE</code> + <code>logger.warning</code> ，绝不让对接因为算跨度而失败	
溯源	抬高时把 <code>库级下限 Y Å</code> 追加到 <code>box_source</code> ，并写入 <code>box_warnings</code> 与 <code>box_library_floor</code>	<code>core/pockets.py:980-987</code>
显式优先	用户/口袋 Agent 显式指定的盒子不受库级下限抬高	<code>core/pockets.py:883</code>

为什么是 P95 而不是 max：单个异常值（如一个超大分子）不应把主组盒子整体推到上限；真正超限的分子由 `large` 组用 **同一中心** 的更大盒子兜住（`docs/architecture.md §15.7b`）。

7.4 超限分子分组（C 方案 B2）

来源：`core/docking.py:706`（`_box_overflows`）、`core/docking.py:714`（`_large_group_box`）、`core/docking.py:889-940`、`docs/api.md §13.5`。

表 35 超限分子分组（C 方案 B2）

项	规则	默认值
超限判据	<code>any(span_i + BOX_GROUP_MARGIN > box_i)</code> ，任一维超限即算	<code>BOX_GROUP_MARGIN = 10 Å</code>
组盒子	<code>clamp(组内最大跨度 + BOX_LARGE_PADDING, 主盒, 大配体组上限)</code> ；上限允许超过 <code>POCKET_MAX_SIZE</code> （否则分组无意义），但受 <code>BOX_LARGE_MAX_SIZE = 60 Å</code> 硬约束	<code>BOX_LARGE_PADDING = 12 Å</code>
中心	与主组 完全相同 （中心永远不随分子变）	<code>core/docking.py:718</code>
实现	两遍对接 ：先用主盒跑一遍并扣住超限分子的结果（不上报、进度不重复计），主组即	<code>core/docking.py:889-899</code>

项	规则	默认值
	最终主组；随后 large 组用组盒重跑并上报	
开销	large 组为空时 零额外开销 ；代价是超限分子被对接两次（跨度直接取自第一遍本来就要生成的配体 PDBQT， 不为全库预生成 3D ）	docs/api.md §13.5
保底	若 large 组结果意外缺失，仍按主组结果上报（ 不丢数据 ）	core/docking.py:926-930

跨度字段来源： `ligand_span` 由 `core/pockets.py:157`（`ligand_span`，解析 PDBQT 坐标）产出，在 `_dock_vina` 里 `round(..., 2)` 写入结果行（`core/docking.py:264`）。

7.5 `box_group / box_size` 落盘与 CSV 列

来源：`agents/persistence.py:110`、`reporting/tables.py:13`、`docs/api.md §13.5`。

表 36 `box_group / box_size` 落盘与 CSV 列

位置	内容
结果行	<code>box_group</code> ∈ <code>main/large</code> 、 <code>box_size</code> （该分子实际使用的盒子）、 <code>ligand_span</code> ；large 组另有 <code>main_box_size</code> 与 改 写 过 的 <code>box_fit_warning</code> （「已单独分组重跑（盒子 X Å）」）
<code>ranking.csv</code>	在 <code>exhaustiveness</code> 之后新增 <code>box_group</code> 、 <code>box_size</code> 两列（ <code>box_size</code> 为 <code>22.0x22.0x22.0</code> 紧凑串）
<code>docking.json / result.json</code>	两组都在（行上带标记，不丢数据）
受体块	<code>box_group_sizes</code> = { <code>"main": [...]</code> , <code>"large": [...]</code> }、 <code>box_group_counts</code> = { <code>"main": N</code> , <code>"large": M</code> }、 <code>box_library_floor</code>
默认排序榜	<code>merge_and_rank</code> 只含 main 组；large 组在报告「2.2 大配体组（盒子不同，不跨组比较）」小节单独列出

实测落盘示例（`var/runs/20260915-174954-6983/docking.json`）：

`box_group_sizes` = `{"main": [22.0, 22.0, 22.0], "large": null}`、
`box_group_counts` = `{"main": 1, "large": 0}`、结果行
`box_group="main"、box_size=[22.0, 22.0, 22.0]`。

7.6 一致性不变量

表 37 一致性不变量

不变量	内容	守护
主组盒子唯一	<code>select_site</code> 每个受体只算一次盒； <code>dock_library</code> 让 main 组所有分子共用 <code>spec["center"]</code> / <code>spec["size"]</code>	<code>tests/test_box_sizing.py</code> 第 1 条（共 10 个用例）
参数与盒子阶段一致	同一 <code>pass</code> 内所有分子的 <code>exhaustiveness/box_size/box_center</code> 完全一致	<code>tests/test_param_plan.py</code> （8 个用例）
中心不随分子变	<code>large</code> 组中心 == 主组中心	<code>core/docking.py:718</code> 、 <code>tests/test_box_sizing.py</code>
显式优先	用户/口袋 Agent 显式指定的盒子不被库级下限抬高（仍会按超限规则分组）	<code>core/pockets.py:883</code> 、 <code>tests/test_box_sizing.py</code>
不跨组比较	<code>large</code> 组分数不与主组混排；报告有独立小节说明	<code>core/ranking.py</code> 、 <code>reporting/report.py:431</code>
盒子效应真实存在	见下表的 B0 实测；因此「每分子一个盒子」会把盒子效应混进排序	<code>docs/architecture.md §15.7b</code>

7.7 换盒子实测：分数最大差 1.34 kcal/mol（原样引用）

来源：`docs/architecture.md §15.7b` 的 B0 表。参数：2026-09-14，vina 1.2.7，凝血酶 1DWC 实验位点中心 31.5/13.74/24.36，`exhaustiveness=4`，`n_poses=1`，`seed=42`，`threads=8`；格式 分数 kcal/mol / 耗时 s。

表 38 换盒子实测：分数最大差 1.34 kcal/mol（原样引用）

配体（3D 跨度 max Å）	18 ³	22 ³	28 ³	34 ³
乙醇（2.4）	-2.83 / 0.5	-2.81 / 0.6	-2.81 / 1.0	-2.85 / 1.5
阿司匹林（5.4）	-5.87 / 0.8	-5.87 / 1.0	-5.86 / 1.3	-5.88 / 1.8
华法林（8.4）	-7.97 / 1.4	-7.75 / 1.6	-7.75 / 1.9	-7.75 / 2.5
三肽模拟（10.0）	-8.55 / 4.9	-7.47 / 4.9	-8.81 / 6.1	-7.60 / 5.9
六肽（16.3）	-7.27 / 12.6	-7.88 / 11.5	-8.08 / 12.6	-8.21 / 12.8

结论（照录）：① 分数对盒子大小敏感且非单调（三肽模拟 18³→22³ 反而差 1.08），所以「每个配体各自一个盒子」会直接把盒子效应混进排序；② 小分子耗时随盒子体积增长（0.5→1.5 s，约 3×），而柔性大配体的耗时主要由可旋转键决定（12.6→12.8 s 几乎不变）；③ 大配体在 22³ 下确实被欠采样（六肽 22³ -7.88 vs 34³ -8.21），因此需要「库级下限 + 超限分子分组」而不是逐分子自适应。

最大差值：同一配体在四个盒子下的最大分差 = 三肽模拟的 $|-8.81 - (-7.47)| = 1.34$ kcal/mol。

本次 e2e 中的盒子溯源实测（`var/runs/20260915-174954-6983/`）：`box_source` = "用户指定"、`box_chosen_by` = "request"、`box_center` = [129.6744, 134.11885, 125.12055]、`box_size` = [22.0, 22.0, 22.0]、`box_library_floor` = null（单分子库，未触发抬高）；报告元信息写作「对接盒：中心 129.67, 134.12, 125.12 Å；尺寸 22.0 × 22.0 × 22.0 Å」（`var/runs/20260915-174954-6983/report.md:50`）。

8. 对接计算与参数自动规划（重点）

结论：Vina 调用链固定为「建受体 → 建网格图一次 → 逐配体设配体/dock/取能量」；参数由`core/params.py`自动规划（精度优先），pilot 护栏保证不超时；并发按分子数自动规划进程 × 线程，并对内存与可复现性都有护栏。

8.1 Vina 1.2.7 调用链

来源：core/docking.py:142（DockingSession）、core/docking.py:214（_dock_vina）。

```
__init__: Vina(seed=42) → set_receptor(spec["pdbqt"]) → compute_vina_maps(center, box_size) # 网格图只算一次，整批复用
dock(): describe_ligand(smiles) → smiles_to_pdbqt(seed) → set_ligand_from_string(pdbqt)
      → v.dock(exhaustiveness, n_poses) → best = v.energies(n_poses)[0] # [total, inter, intra, torsional]
      → n_poses>1: v.write_poses(energy_range=3.0) / n_poses==1: v.write_pose(...)
```

表 39 Vina 1.2.7 调用链

能量项	结果字段	来源
总结合亲和力 / 分子间 / 分子内 / 扭转惩罚	affinity_kcal_mol / intermolecular_kcal_mol / intramolecular_kcal_mol / torsion_kcal_mol	best[0..3]，总分为 round(..., 2)（core/docking.py:255-258）

- 网格图复用：compute_vina_maps 与对接本身同量级开销，每个 worker 对「受体 + 盒子」只算一次（README.md §4）。
- 位姿语义：n_poses > 1 必须真的写出全部位姿（pose_count），不能只写最优一个——历史 bug（core/docking.py:228-232）。
- 盒子适配告警：span_i + BOX_GROUP_MARGIN > box_i 时写 box_fit_warning，口径与分组判据一致（core/docking.py:245-251）。

8.2 AutoDock4 回退

来源：core/docking.py:328（_autodock_bin）、core/docking.py:333（run_docking_autodock_internal）、core/docking.py:174-177。

表 40 AutoDock4 回退

项	规则
触发条件	<code>engine="auto"</code> （默认）且 Vina 导入/初始化失败 → 记 warning 后回退 AutoDock4 CPU
显式 <code>engine="vina"</code>	Vina 不可用直接抛异常（不回退），避免「以为在用 Vina」
显式 <code>engine="autodock"</code>	只用 AutoDock4； <code>autodock4/autogrid4</code> 未安装 → 明确报错
实测（反证）	<code>engine=autodock</code> 返回：所有对接引擎均失败 (<code>vina/autodock</code>): AutoDock CPU 引擎不可用: 未找到 <code>autodock4/autogrid4...</code> (<code>verify_docking.py</code> D 层)
参数	<code>ga_num_evals</code> 默认 25,000 (<code>core/docking.py:337</code>)
缺口（已知）	AD4 行里没有 <code>exhaustiveness/seed</code> ，与 Vina 结果不可通约； <code>docs/architecture.md</code> §15.2 第 2.6 条已登记为 P2

8.2b 外部引擎（`engine=external`）

来源：`core/external_tools.py`（登记与探测）、`core/external_run.py`（执行适配）、`core/docking.py::DockingSession._init_external/_external_maps/_dock_external`。

表 41 b 外部引擎（`engine=external`）

项	规则
启用方式	设置页「外部工具」或 <code>EXTERNAL_DOCKING_BIN</code> 登记可执行文件； <code>engine=external</code> （表单/工具参数/ <code>docking.engine</code> 默认值三者其一）才真正改由它执行
未登记 / 探测不通过	拒绝启动并给出补齐方法（ <code>ExternalEngineError</code> ），不静默改用内置 CPU —— 否则用户以为在用 GPU 却拿到 CPU 分数
已接入执行适配	<code>autodock-gpu</code> （格点图 + <code>--ffile/--lfile/--nrun/--resnam</code> ，解析 <code>.dlg</code> ）、 <code>vina-cpu</code> （ <code>--receptor/--ligand/--out + --score_only</code> 取能量分解）
仅登记不执行	<code>unidock / vina-gpu</code> ：本机没有可验证输出格式，执

项	规则
	行层直接报错，不猜参数（ <code>tests/test_external_run.py</code> ）
格点图	<code>autogrid4</code> 按「受体 + 盒子」在会话内生成一次并缓存（间距 0.375 Å、格点数为奇数）；配体类型取实测可用全集（含 meeko 必带的极性氢 HD），全集之外的类型（Cu/Hg/Se/Na/K 等参数库没有的） 明确报错 而非静默跳过该分子
设备号	项目内 <code>GPU_DEVICE</code> 是 0 基，AutoDock-GPU 的 <code>--devnum</code> 是 1 基，下发时 +1（0 基直传会被引擎拒绝，实测退出状态 255）
结果行	与 内 置 引 擎 同 形： <code>engine</code> （如 <code>autodock-gpu</code> ）、 <code>engine_version</code> （AutoDock-GPU version: v1.6-release）、 <code>affinity_kcal_mol</code> 与三项能量分解、 <code>pose_file</code> 留档
能量取值	<code>--nrun N</code> （由 <code>n_poses</code> 映射）时 DLG 内有多组结果，解析取 最低结合能那一组 并与 <code>CLUSTERING HISTOGRAM</code> 交叉校验，而不是文件里第一组
位姿格式	DLG 内含多组结果，落档时抽取 最优那一组 的 <code>DOCKED</code> ：载荷写成 PDBQT（ <code>pose_file</code> ，下游分析与报告按 PDBQT 读），原始 DLG 另存 <code>pose_raw</code> 。真实损失：此前只存 <code>.dlg</code> ，位姿分析 <code>read_pdbqt</code> 读不出 → 报告整段“未产生可读取的位姿文件”（一次运行 2845 个位姿白算）
大环配体	配体准备统一 <code>rigid_macrocycles=True</code> ：meeko 默认切环会插伪原子（元素 G，类型 <code>CG0/G0</code> ）， <code>autogrid4</code> 参数库没有这些类型（实测 <code>CG0/G0/G1/CG/W</code> 全部 unknown），AutoDock4/-GPU 必然失败。实测 116 条失败里 113 条 由此修好（其余 2 条 meeko 无法给 Se/Mn/Pt 定类型、1 条 3D 构象失败），代价是 7-33 元环不采样环构象（notes 与报告如实说明）
本机实测	RTX 5090（驱动 590.48.01，AutoDock-GPU v1.6 CUDA sm_120 构建）：凝血酶 1DWC × 苯甲脒 <code>NC(=N)c1ccccc1</code> → -6.15 kcal/mol （分子间 -6.44），乙醇 → -2.80 kcal/mol，单配体 < 1 s（ <code>engine=external</code> ， <code>tests/test_external_run.py::test_real_autodock_gpu_end_to_end</code> ）

8.2c 库已知后补规划与两阶段漏斗

来源：`tools/docking.py`（补规划与漏斗执行）、`core/params.py::plan_docking_params`、`agents/dispatch.py`。

表 42 c 库已知后补规划与两阶段漏斗

项	规则
触发条件	受理层规划时库还没解析出来（分子来自上传文件）→ <code>param_plan</code> 为空；对接工具在库、盒子都已知时补规划一次并写回 <code>run.data["param_plan"]</code> （报告 1.5 节与后续提示词都用它）
漏斗执行	规划判定两阶段（库 $\geq$ <code>AGENT_FUNNEL_MIN=500</code> ）且用户没显式给 <code>exhaustiveness</code> 时，在工具内确定性地跑两遍：全库按 <code>coarse=round(exh/4)</code> 粗筛 → 头部 <code>refine_top_n</code> （默认 200）按 <code>exh</code> 精算；精算行 <code>pass=fine</code> 并保留 <code>affinity_coarse</code> ， <code>notes</code> 如实说明"排序请只采用精算行或注明粗筛"
用户显式参数	用户给了 <code>exhaustiveness</code> 时不改写成粗筛值（规划只记录，不改用户参数）
真实缺口	2026-09-23 运行 20260923-224338-3779：2961 条库按默认 16 全量精算 40 分钟，粗筛/精算两阶段一次没跑（ <code>param_plan</code> 整份为空）

8.2d 阻断式询问（共晶配体阳性对照）

来源：`tools/choices.py::publish_choices(blocking=True)` / `blocking_choice_pending`、`tools/docking.py`、`agents/dispatch.py`、`api/routers/agent.py`。

表 43 d 阻断式询问（共晶配体阳性对照）

项	规则
语义	受体自带共晶配体且用户未指定阳性对照时先问再算：候选下发后本轮立即以 <code>needs_user_input</code> 收口，不开始对接
护栏	未回答前 <code>run_docking</code> / <code>molecular_docking</code> 直接返回 <code>needs_user_input</code> （连对接子 Agent 都不调度），杜绝"幂等标记放行 → 白跑几十分钟"

项	规则
前端	候选在 回合边界 （工具返回/节点推进/阶段变化）挂出，运行中禁用、跑完可点；点选后以同一会话发起新一轮并带上对照
真实故障	2026-09-23：问题 22:47 下发，运行 23:23 才结束才显示，用户回答时结果早已算完

8.2e 模型可见消息日志（气泡可审计）

来源：`agents/persistence.py::build_messages_log` → 运行产物 `messages_log.json`。

表 44 e 模型可见消息日志（气泡可审计）

项	规则
内容	逐条 <code>{role, tool?, tool_calls[<code>{name,args_chars}</code>], chars, head(≤400 字)}</code> ，最多保留最后 200 条 + 最终结论
为什么	用户反馈"气泡把大量数据对话了出去"时，运行产物里只有最终结论（ <code>agent_report.md</code> ，KB 级）→ 事后 无法回看 模型看到了/输出了什么。日志只存形状与开头，不落全量正文（8 MB 工具 JSON 不会因此再存一份）
配合	协调 Agent 提示词新增纪律：「正文只给结论，不复述工具 JSON / notes / 逐分子明细」；气泡正文超过 20000 字符（病态 dump）才折叠并指向报告页签

8.3 exhaustiveness 自动规划公式与全部默认值

来源：`core/params.py:248`（`plan_docking_params`）、`core/params.py:15-28`（模块 docstring 的规则表）、`settings.py:216-264`、`docs/api.md §7.6`。

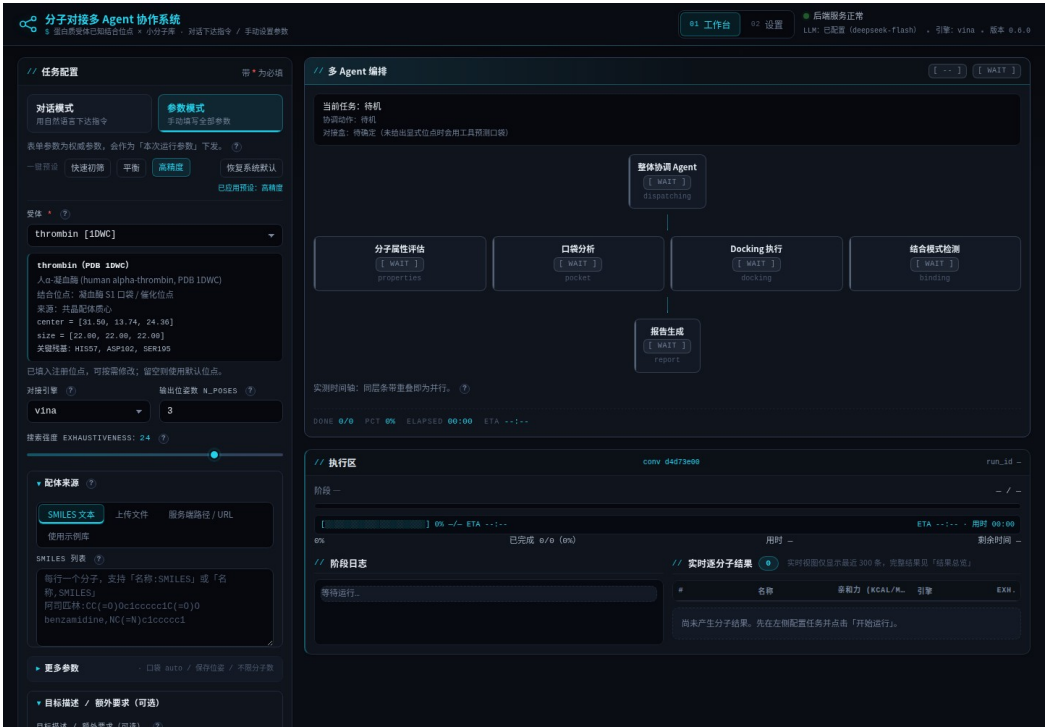


图 6 搜索强度一键预设（快速初筛 / 平衡 / 高精度 / 恢复系统默认）与自动规划出的 EXHAUSTIVENESS: 24

表 45 exhaustiveness 自动规划公式与全部默认值

规则	公式	默认参数
基准强度	screening → 12 ; binding_only/姿态分析 → 16; properties_only → 不规划	AUTO_PARAM_BASE_SCREENING=12、 AUTO_PARAM_BASE_BINDING=16
柔性系数	f_rot = clamp(P90(库内可旋转键) / 5, 0.75, 2.5) (2D 描述符, 大库抽样最重的 500 个)	AUTO_PARAM_FLEX_DIVISOR=5、 AUTO_PARAM_FLEX_MIN=0.75、 AUTO_PARAM_FLEX_MAX=2.5、 AUTO_PARAM_STATS_SAMPLE=500
盒体积系数	f_box = clamp((V_box / 22 ³)^(1/3), 1.0, 2.0) (保持单位体积采样密度)	AUTO_PARAM_BOX_REF=22、 AUTO_PARAM_BOX_FACTOR_MAX=2.0
搜索强度	exhaustiveness = clamp(round(base × f_rot × f_box), 2, 32)	AUTO_PARAM_EXH_MIN=2、 AUTO_PARAM_EXH_MAX=32
n_poses	筛选 1; 姿态/结合模式分析 3; 上限 5	AUTO_PARAM_N_POSES_BINDING=3、 AUTO_PARAM_N_POSES_MAX=5 (core/params.py:53-55)
引擎与种子	固定 engine=vina、seed=42	core/params.py:58-59 (PLAN_ENGINE / PLAN_SEED)
两阶段漏斗	N ≥ AGENT_FUNNEL_MIN → 粗筛 max(1, round(exh/4)) 全库 + 精	AGENT_FUNNEL_MIN=500、 AGENT_RE_FINE_TOP_N=200

规则	公式	默认参数
	算 exh 前 refine_top_n	
总开关	AUTO_PARAM_ENABLED=on ; off 时沿用旧默认 exhaustiveness=16 / n_poses=1	settings.py:217-220 、 core/params.py:49-50
用户显式参数	source="user", 自动规划 不改 写; 同时跳过 pilot 降级	core/params.py:351-358 、 core/params.py:389

表 46 AUTO_PARAM_* 全部默认值总表

环境变量	默认	含义	位置
AUTO_PARAM_ENABLED	on	自动规划总开关	settings.py:218
AUTO_PARAM_PILOT	on	pilot 预算护栏开关	settings.py:222
AUTO_PARAM_PILOT_N	3	pilot 试跑分子数 (夹 1-10)	settings.py:226
AUTO_PARAM_PILOT_MIN	50	触发 pilot 的最小库规模	settings.py:228
AUTO_PARAM_BUDGET_RATIO	0.6	预算 = 该比例 × RUN_TIMEOUT_SECONDS (0.6 × 900 = 540 s)	settings.py:230
AUTO_PARAM_EXH_MIN	2	exhaustiveness 下限	settings.py:233
AUTO_PARAM_EXH_MAX	32	exhaustiveness 上限	settings.py:235
AUTO_PARAM_BASE_SCREENING	12	筛选基准	settings.py:237
AUTO_PARAM_BASE_BINDING	16	姿态/结合模式基准	settings.py:239
AUTO_PARAM_STATS_SAMPLE	500	库统计抽样分子数 (夹 20-5000)	settings.py:241
AUTO_PARAM_FLEX_DIVISOR	5.0	柔性系数分母	settings.py:244
AUTO_PARAM_FLEX_MIN	0.75	柔性系数下限	settings.py:246
AUTO_PARAM_FLEX_MAX	2.5	柔性系数上限	settings.py:248
AUTO_PARAM_BOX_REF	22.0	盒体积系数参考边长 (Å)	settings.py:250
AUTO_PARAM_BOX_FACTOR_MAX	2.0	盒体积系数上限	settings.py:252
AUTO_PARAM_REFINE_TOP_MIN	100	超预算时精算头部下限	settings.py:254
AUTO_PARAM_REFINE_TOP_LARGE	300	大库精算头部上调值	settings.py:256
AUTO_PARAM_LARGE_LIB	5000	触发精算头部上调的库规模	settings.py:258
AUTO_PARAM_N_POSES_BINDING	3	姿态分析 n_poses	settings.py:262
AUTO_PARAM_N_POSES_MAX	5	n_poses 上限	settings.py:264

``f_box` 何时为 1.0`: 主盒尺寸尚未确定时（例如规划发生在口袋分析之前）体积系数按 1.0 并如实记录（`core/params.py:341-343`）。本次 e2e 实测就是这种情况：决策链第 3 条为「主盒尺寸尚未确定 → 体积系数按 1.0」（`var/runs/20260915-174954-6983/report.md:106`）。

实测的完整决策链（`AUTO_PARAM` 生效样例，`var/runs/20260915-174954-6983/report.md:104-109`；该次运行发生在 **base 仍为 12** 的版本，故下表第 1、4 行出现 12 / 9 —— 现版本基准为 16，同样输入会得到 `clamp(round(16 × 0.75 × 1.00), 2, 32) = 12`）：

表 47 exhaustiveness 自动规划公式与全部默认值

#	决策
1	任务类型 <code>docking_only</code> → 基准搜索强度 <code>base=12</code> （筛选）
2	P90 可旋转键 3（库 1 个，抽样 1/1 个最重分子）→ 柔性系数 0.75
3	主盒尺寸尚未确定 → 体积系数按 1.0
4	<code>exhaustiveness = clamp(round(12 × 0.75 × 1.00), 2, 32) = 9</code>
5	<code>n_poses=1</code> （筛选默认 1），上限 5
6	库 1 < 500 → 单阶段（ <code>exh=9</code> 跑全库）

8.4 两阶段漏斗

来源：`runtime/tool_io.py:141-162`（`funnel_settings` / `funnel_advice`）、`docs/architecture.md § 8.1`、`docs/api.md § 7.6`。

候选数 ≥ AGENT_FUNNEL_MIN(500)
① 全库粗筛 `run_docking(exhaustiveness=AGENT_COARSE_EXHAUSTIVENESS=1)`
② 头部精算 `run_docking(top_from_previous=AGENT_REFINE_TOP_N=200, exhaustiveness=AGENT_FINE_EXHAUSTIVENESS=16)`

表 48 两阶段漏斗

机制	规则	位置
精算清单来源	从黑板取，不经过模型上下文（ <code>top_from_previous</code> ）	<code>docs/architecture.md § 8.1</code>

机制	规则	位置
合并精度优先	<code>_dock_precision = (1 if pass=="fine" else 0, exhaustiveness)</code> ； 显 式 <code>pass=fine</code> 优先 ， 其次 比 <code>exhaustiveness</code> ，相等则保留先出现的	<code>agents/persistence.py:100-135</code>
粗筛分保留	被 替 换 时 旧 亲 和 力 写 进 <code>affinity_coarse</code> （ 若 新 行 没 有 ）	<code>agents/persistence.py:137-139</code>
去重主键	<code>(name, smiles)</code> ，不是 <code>smiles</code> 单独	<code>agents/persistence.py:127</code>
陷阱	<code>coarse_map</code> 必须在对接之前捕获（对接后黑板会被精算结果覆盖）	<code>docs/architecture.md §8.1</code>
流水线也走同一漏斗	<code>pipeline.run_pipeline</code> 在 <code>N ≥ AGENT_FUNNEL_MIN</code> 时 用 <code>dock_library</code> 跑「全库粗筛 → 头部精算」，精算沿用粗筛盒子	<code>docs/architecture.md §8.1</code>
分片护栏	不推荐按固定分子数切很多片（每次分发约 8-10 s LLM 往返，10k/200=50 片凭空多出约 7 分钟）；确需分片受 <code>AGENT_SHARD_MAX=8</code> 约束	<code>core/params.py:102-104</code> 、 <code>runtime/tool_io.py:147</code>
大库精算头部上调	<code>N ≥ AUTO_PARAM_LARGE_LIB(5000)</code> 且 预算允许 → 精算头部从 200 上调到 300	<code>core/params.py:440-454</code>

8.5 pilot 预算护栏（精度优先降级）

来源：`core/params.py:383-485`。

表 49 pilot 预算护栏（精度优先降级）

项	规则	默认 / 位置
触发	库 $\geq$ <code>AUTO_PARAM_PILOT_MIN</code> ，且 注入了 <code>pilot</code> 回调，且 <code>AUTO_PARAM_PILOT=on</code> ，且用户未	50

项	规则	默认 / 位置
	显式指定参数	
选取 / 试跑	库统计里最贵的`AUTO_PARAM_PILOT_N`个分子（cost = 可旋转键×4 + 重原子），以 exhaustiveness=1 真实对接取平均秒/分子	3
外推 / 预算	两阶段 mean × N × coarse + mean × refine_top_n × exh；单阶段 mean × N × exh。预算 = AUTO_PARAM_BUDGET_RATIO × RUN_TIMEOUT_SECONDS = 0.6 × 900 = 540 s	core/params.py:220-226、core/params.py:211-217
降级顺序	① 先降精算头部到 AUTO_PARAM_REFINE_TOP_MIN(100)；② 仍超才降搜索强度，不低于 base/2；③ 到下限仍超则按精度优先保留当前强度，只记 warning 与决策说明	core/params.py:456-485
pilot 只测量	不写位姿、不进 ranking/黑板；抛异常/返回 None 一律退回静态规划（只 warning）；用户显式参数时跳过降级	core/params.py:26、core/params.py:389-425
库统计缓存	按库内容 + 抽样规模的 sha1 缓存到 assets/cache/param_stats/	core/params.py:110-118

8.6 并发规划表（plan_concurrency）

来源：core/docking.py:86（plan_concurrency）、README.md §11.4b。

表 50 并发规划表（plan_concurrency）

分子数	进程 × 线程	理由
1	1 × min(可用核 , DOCKING_SERIAL_THREADS)	单分子把核压在搜索上
1-7	1 × 8（默认）	少量分子用线程吃满核，避免多进程各自建图的固定开销
8	min(可用核 , 24) × max(1, min(thread_cap, 可用	进程与线程混合

分子数	进程 × 线程	理由
	核/workers))	
≥ 50	最多 24 × 1	大库靠进程并行；受 DOCKING_WORKERS 与内存护栏约束

- 可用核 = CPU 数 - 1（留一核给主进程/服务，core/docking.py:68）。
- thread_cap = min(DOCKING_SERIAL_THREADS, 16)，默认 8（core/docking.py:101）。
- DOCKING_WORKERS 显式给出时覆盖自动规划，但仍被 min(total, 内存上限) 夹紧（core/docking.py:104-109）。
- 实测对照（README.md § 11.4b）：7 分子精算 40.0 s → 8.1 s（4.9×）。
- 实测并发（本次 e2e）：单分子库的 note 为「对接并发：1 进程 × 8 线程」（var/runs/20260915-174954-6983/run.json 的 notes）。
- 实测并发（大库）：96 分子 exhaustiveness=1，串行 159.8 s → 8 进程 28.2 s（5.7×），结果逐位一致（README.md § 4）。

8.7 内存护栏、LPT 调度与取消

表 51 内存护栏、LPT 调度与取消

项	内容	位置
内存护栏	per_mb = max(64, DOCKING_WORKER_MEM_MB)（默认 400 MB）；cap = max(1, int(MemAvailable_MB × 0.7) // per_mb)（留 30% 余量）；读不到 /proc/meminfo 时返回保守值 64；per_mb 下限 64 防除零	core/docking.py:72-83、在 plan_concurrency 里与分子数/可用核一起夹紧（core/docking.py:109）
LPT 调度	成本 cost = 可旋转键 × 4 + 重原子（RDKit 计算，解析失败返回 0），按成本从大到小提交，减少尾部拖尾；实测 7 分子 4.9× 加速	core/docking.py:119、core/docking.py:535、README.md § 11.4b
取消	协作式：线程 Event + 跨进	cancellation.py、docs/api.md

项	内容	位置
	程 ； 对 接 循 环 检 查 <code>cancel_event</code> ，取消时父线程 <code>_terminate_pool</code> 。 不 能 用 <code>asyncio.Task.cancel()</code> （只取消协程，Vina 仍在跑）；实测取消后 1.5 s 内结束（否则需 30 s+），状态标 <code>cancelled</code> 且保留已完成数据	§ 8.1、CHANGELOG.md 第 12 条
已知缺口	Agent 模式的 <code>cancel_event</code> 未传入对接工具（P0，见 § 11.4 第 4.1 条）	docs/architecture.md § 15.4

8.8 可复现性（seed / seed_policy 与「批次无关」实测）

来源：`core/docking.py:142-162`、`docs/architecture.md § 8.2`。

表 52 可复现性（seed / seed_policy 与「批次无关」实测）

项	内容
默认种子	<code>seed = 42</code> ； 结 果 行 带 <code>seed</code> 与 <code>seed_policy="session"</code>
<code>seed_policy</code> 语义	只有 "session" 一种：Vina 每次 <code>dock()</code> 都按构造时的种子重新开始采样，因此 不需要也没有 per-molecule 的收益（ <code>core/docking.py:160-162</code> ）
批次无关性	同一分子的分数与批次组成、提交顺序、进程数 / 分片方式 无关 —— 有永久回归测试 <code>tests/test_agent_context.py::test_scores_are_independent_of_batch_and_workers</code> （8 分子并行批里某分子的分数 == 它单独对接的分数，整批重跑也逐位一致）
已验证并否决的做法	不要 为了「隔离」给每个分子换种子：那样只会改变分数而不带来收益（ <code>docs/architecture.md § 8.2</code> ）
独立复算取证	<code>verify_docking.py</code> B 层用 vina API 从零重写对接，与工具输出 逐位一致 ；配体 PDBQT sha256 相同；位姿用全新实例重打分一致（ <code>README.md § 13</code> ）
参数敏感性实测	<code>exhaustiveness ∈ {1, 3, 6, 16} → 分数 {1:</code>

项	内容
	-5.84, 3: -5.87, 6: -5.78, 16: -5.85}, 极差 0.09 kcal/mol (本次实跑 verify_docking.py C3 层)
能量恒等式	total = intermolecular + torsional 对全部 8 分子成立 (最大偏差 0.01, README.md §13 C 层)

注意：不同搜索强度会得到不同分数（如 -5.78 @6 vs -5.87 @3），这是**参数真实参与计算**的表现，因此排序 CSV 与 docking.json 都带 engine / exhaustiveness / seed / seed_policy 四列 (README.md §12)。

9. 评估与报告

结论：性质、指纹、药效团、结合模式全部是 RDKit 真实计算；报告是固定 9 章模板（两种运行模式共用），PDF 版有封面工具版本、分页与页脚、PNG 真实内嵌，正文不含 URL。

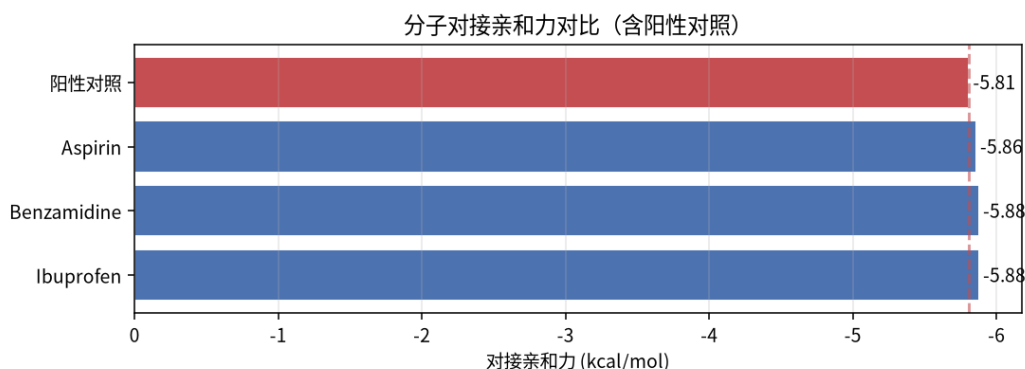


图 7 对接亲和力对比图（含阳性对照；来自实测运行 20260916-085944-4688 的报告产物）

9.1 性质评估（Lipinski 等）

来源：`core/chemistry.py:48 (compute_properties)`。

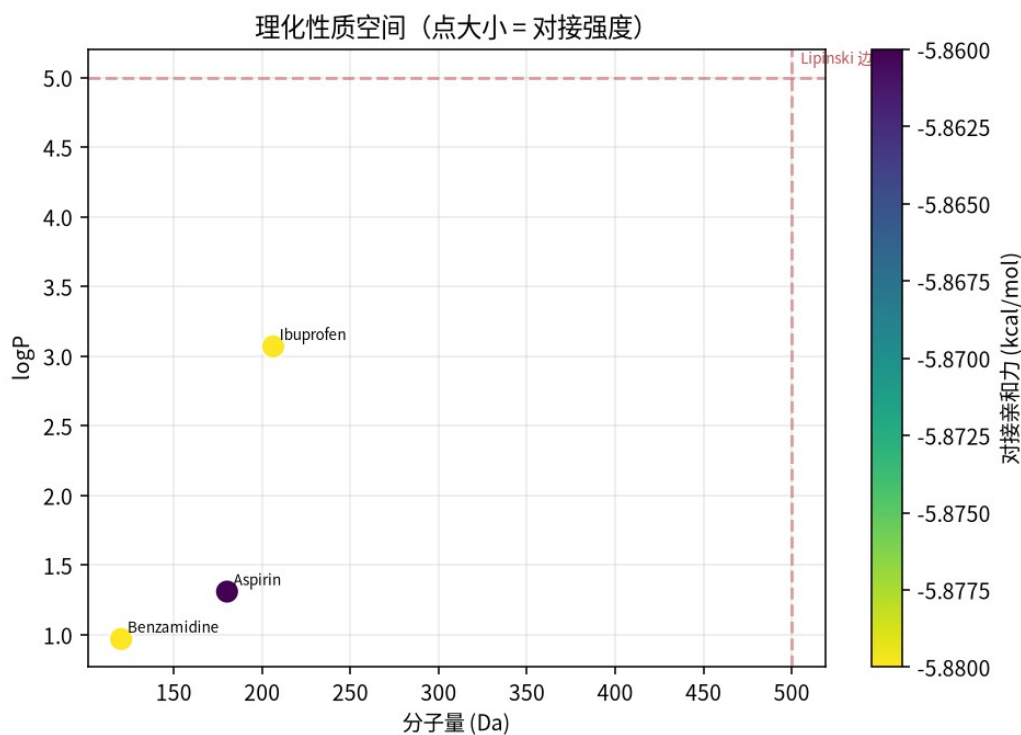


图 8 理化性质空间（横轴分子量、纵轴 logP、颜色/点大小 = 对接强度，虚线为 Lipinski 边界）

表 53 性质评估（Lipinski 等）

字段	计算方式
molecular_weight	Descriptors.MolWt（2 位小数）
logP	Crippen.MolLogP（2 位小数）
tpsa	rdMolDescriptors.CalcTPSA（2 位小数）
hbd / hba	CalcNumHBD / CalcNumHBA
rotatable_bonds	CalcNumRotatableBonds
heavy_atoms / aromatic_rings / formula	GetNumHeavyAtoms / CalcNumAromaticRings / CalcMolFormula
lipinski_violations	MW>500、logP>5、HBD>5、HBA>10 各 +1； rotB>10 也 +1（仅参考提示）
drug_likeness_pass	violations <= 1

9.2 结合模式（Morgan + MACCS + SMARTS 药效团 + 性质差）

来 源 ： core/chemistry.py:160 （ binding_mode_profile ） 、 core/chemistry.py:91-135、tools/binding.py:1-15、CHANGELOG.md 第 7 条。

表 54 结合模式（Morgan + MACCS + SMARTS 药效团 + 性质差）

项	实现
双指纹相似度	Morgan(radius=2, 2048bit) 与 MACCS keys(166 位) 的 Tanimoto 相似度，两者互补用于交叉验证
药效团锚定基团（SMARTS）	5 类真实子结构匹配及原文：amidine [NX3][CX3]=[NX2] 、 guanidine [NX3][CX3]=[NX2])[NX3] 、 carboxylic_acid [CX3](=O)[OX2H1] 、 sulfonamide [SX4](=O)(=O)[NX3]、aromatic_ring c1ccccc1
结构一致性 / 性质差异 / 提示	_consistency：combined ≥ 0.7 或 morgan ≥ 0.85 → high，combined ≥ 0.4 或 morgan ≥ 0.5 → medium，否则 low；_delta 给出与阳性对照的逐项性质差；_hint（core/chemistry.py:146）给出结合模式提示
输出字段	similarity_to_positive_control（= Morgan，主字段，图 表 /CSV 沿用）、maccs_tanimoto、combined_similarity、structural_consistency、anchor_match、binding_mode_hint、vs_positive_control_kcal_mol

项	实现
两个工具口径一致	<code>binding_mode_analysis</code> （完整）与 <code>positive_control_similarity</code> （轻量，只返回相似度字段子集）共用同一数据源 <code>core.chemistry.compute_binding_report</code> ，因此不会出现「方法学缩水」
跨 Agent 核验	<code>check_binding_consistency</code> （ <code>tools/binding.py:141</code> ）：核对对接与结合模式是否齐全、是否出现「亲和力强但相似度低」/「相似但对接弱」等不一致，输出 <code>flags{consistent, affinity_strong_low_similarity, similar_but_weak_docking, missing_docking, missing_binding, control}</code>

9.3 阳性对照基线

来源：`tools/docking.py:169-188`、`CHANGELOG.md` 第 13/17 条、`docs/api.md` § 7.1。

表 55 阳性对照基线

规则	内容
可选性与显式跳过	只有用户提供了阳性对照 SMILES 才执行对照对接与结合模式比较；留空即跳过； <code>skip_positive_control=true</code> 即使提供了也跳过
跳过时的如实标注	<code>result.positive_control</code> 为 <code>{}</code> 、 <code>result.binding</code> 为空、相似度字段为 <code>null</code> ，并在 <code>run.notes</code> 说明；图表不画对照线；示意图显示 [SKIP]
自动补入与排序	对接清单里由工具自动补入对照分子（内部标记 <code>__positive_control__</code> ，对外展示为「阳性对照」），Agent 不必单独再调；对照行在报告中单独一行， <code>ranking.csv</code> 里 <code>rank=PC</code> 、 <code>status=control</code> 、 <code>similarity_to_positive_control=1.0</code>
默认对照文件与实测示例	<code>assets/libraries/positive_control.csv</code> （ <code>benzamidine</code> ， <code>agents/dispatch.py:142-153</code> ）；示例库 8 分子运行时对照亲和力 -5.78 kcal/mol （ <code>docs/api.md</code> § 3 的响应示例）

9.4 报告 9 章与 PDF 生成

来源：[reporting/report.py](#)、[reporting/pdf.py](#)、[README.md](#)（报告小节）。

分子对接筛选报告

运行 20260914-165554-8574 · 受体 thrombin(1DMC) · 1 个候选分子 · 确定性流水线 (无 LLM)

工具版本

vina | 1.2.7
rdkit | 2026.09.6
neeko | 0.8.0
p2rank | p2rank_2.5.1
python | 3.12.3
matplotlib | 3.11.1

生成时间: 2026-09-14T16:55:54

项目	内容
运行编号	20260914-165554-8574
运行模式	确定性流水线 (无 LLM)
生成时间	2026-09-14T16:55:54
受体	thrombin(1DMC)
对接盒	中心 [31.5, 13.74, 24.36], 尺寸 [22.0, 22.0, 22.0] Å / 来源: 实验位点 (共晶配体质心) · p2rank 预测-
对接引擎	vina
搜索强度	1
候选分子数	1
阳性对照	未提供 (未做对照分析)
任务受理	screening · 权威=pipeline · 决策=run · 来源=rules / 假设: 确定性流水线 (零模型) : 参数由请求与默认-

1. 任务与参数

- 随机种子: seed=42 / seed_policy=session (相同参数与随机种子可复算对接结果)
- 对接并发: 1 进程 × 8 线程 (1 个分子; 大分子优先调度以减少拖尾)

结合口袋预测 (真实工具输出)

提示: 引擎: p2rank; 下表为工具预测的候选口袋, 实际使用的对接盒见上方元数据「对接盒」一行。

#	口袋	score	中心 (Å)	范围 (Å)	附近残基	选择
1	pocket1	10.58	[32.8846, 11.8534, 19.3657]	[12.79, 15.04, 12.8]	H_191 H_192 H_193 H_195 H_2~	← 采用
2	pocket2	6.04	[33.089, 19.1875, 24.7724]	[13.76, 12.49, 14.59]	H_189 H_190 H_191 H_192 H_1~	
3	pocket3	3.94	[57.1272, 22.7571, 18.9268]	[13.31, 14.89, 14.37]	H_120 H_121 H_122 H_123 H_1~	
4	pocket4	3.41	[38.648, 31.8768, 37.2898]	[14.81, 14.44, 12.23]	H_133 H_135 H_161 H_162 H_1~	
5	pocket5	3.29	[45.796, 21.0061, 35.3138]	[14.23, 15.43, 11.92]	H_130 H_132 H_162 H_163 H_1~	
6	pocket6	2.32	[35.9974, 35.7811, 23.468]	[12.41, 13.32, 15.01]	H_157 H_158 H_159 H_18 H_18~	
7	pocket7	1.92	[42.0477, 31.984, 10.3205]	[14.11, 13.65, 13.67]	H_117 H_23 H_24 H_25 H_27 H~	
8	pocket8	1.52	[51.0702, 41.508, 22.1225]	[11.79, 11.4, 14.05]	H_202 H_205 L_11 L_12 L_13 ~	

2. 结果排序

提示: 按对接亲和力和升序 (数值越低结合越强)。

排名	分子	亲和力 (kcal/mol)	Δ vs 对照	引擎	搜索强度	分子量	logP	TPSA	类药性	相似度
1	乙醇	-2.83	—	vina	1	46.1	-0.00	20.2	通过	—

对接亲和力对比图

run 20260914-165554-8574 · 第 1 页

图 9 系统自动生成的 PDF 报告第 1 页：封面、工具版本、任务与参数（真实运行产物）

表 56 固定 9 章（章节标题硬编码，表/图号由运行时计数器生成；产物缺失的图不占号）

#	章节标题	位置
(首)	报告信息表	reporting/report.py:270
1	## 1. 任务与参数：1.1 任务来源与受理 · 1.2 受体、位点与对接盒 · 1.3 对接参数（同一阶段内参数一致） · 1.4 工具	reporting/report.py:275 / 282 / 319 / 335 / 367 ；1.5 标题在 reporting/content.py:283 （触

#	章节标题	位置
	与版本 · 1.5 参数自动规划 · 1.6 结合口袋预测（有口袋数据才出现）	发点 reporting/report.py:374) ； 1.6 在 reporting/report.py:210
2	## 2. 结果排序（2.1 主组排序；2.2 大配体组（盒子不同，不跨组比较））	reporting/report.py:382 / 404 / 431
3-5	## 3. 推荐分子（3.1 推荐化合物排行：综合分 + Agent 理由 + 筛选建议；3.2 优于阳性对照） · ## 4. 理化性质 · ## 5. 结合模式与阳性对照比较	reporting/ report.py 、 reporting/ recommend.py
6-9	## 6. 方法与局限 · ## 7. 失败与跳过 · ## 8. 结论与建议 · ## 9. 数据与产物	reporting/report.py:522 / 571 / 603 / 633

文案原则（面向交付）：报告是交付文档，不是过程日志 —— 正文不出现「以下是简短总结 / 明细见报告 / 本节是…」这类过程性元话，不写「真实计算、未编造」这类自我声明（溯源信息在第 9 节产物表与 `result.json` 里），同一件事（质子化口径、同阶段参数一致、盒子一致性）只在最相关的一节写一次；协调 Agent 的结论文本进第 8 节前会先剥掉过程性句子与小节标题，只保留**结论/建议/风险**类小节。

- 受体来源（1.2 节）：**由 `fetch_protein_structure` 把 `provenance`（数据库 / UniProt accession / 物种 / PDB 号 / 实验方法与分辨率）记为 **运行事实**（`run.data["receptor_provenance"]` → `result.json`），报告 § 1.2 直接渲染成一行「受体来源：UniProt `Q9SJQ6`（...，*Arabidopsis thaliana*）；结构 RCSB PDB `7YHP`（EM，3.1 Å）」。**审计背景：**这份溯源原先只存在于工具返回与协调 Agent 的「实际执行」小节里，而结论节按「只留结论/风险」精简后它会消失 —— 溯源必须由报告自己给出。

关键小节说明：

- 盒子一致性（1.2 节）：**无 large 组时写「所有参与排序的分子共用同一对接盒，盒子效应对相对排序无影响」；

有 large 组时写「主组共用同一对接盒；另有 N 个分子因 3D 跨度超出主盒，划入 large 组、以同一中心更大盒子单独重跑，两组分数不可跨组比较」。

- **第 6 节方法学** (`reporting/report.py:532-536`)：写明盒边长 18/22/28/34 Å 下最大差 1.34 kcal/mol 且非单调。
- **失败与跳过** (第 7 节)：按原因分组列出失败分子数、占比与示例；原因归并分类为

「配体化学解析失败 (SMILES / 3D 生成)」 「位姿写入失败」 「运行被取消」 「对接超时」 「对接失败 (引擎 / 受体 / 参数)」 (`reporting/content.py:181-197`)，另有一组「未进入对接 (在解析/准备阶段被跳过)」。

- **参数自动规划** (第 1.5 节)：把 `param_plan.decisions` 的理由链成表展示，并在表里给出 pilot 状态 (`reporting/content.py:278-333`)。

表 57 PDF 生成 (`reporting/pdf.py`, `matplotlib PdfPages`, 零新增运行时依赖)

项	内容	位置
封面	分子对接筛选报告 + 副标题 (模式 / run id / 受体 / 分子数) + 工具版本表 + 生成时间；版本取值顺序为模块 <code>__version__</code> → 发行版元数据 → 已知常量 → 未知，P2Rank 用本地部署目录名 (如 <code>p2rank_2.5.1</code>)	<code>pdf.py:391-405</code> 、 <code>reporting/content.py:137-170</code>
页脚 / 页码 / 分页	页脚 <code>run {run_id} · 第 {N} 页 · {created_at}</code> (等宽字体)；页码从 1 开始按 <code>_new_page()</code> 自增 (无「共 M 页」)；空间不足自动 <code>savefig</code> + 新建页	<code>pdf.py:185-206</code>
图片内嵌	Markdown 图片块 → 按 <code>CHART_FILES</code> 反查产物名 → <code>mpl_image.imread</code> 真实贴图；按页宽等比缩放、超高再缩；找不到时写「(图表 X 未找到对应产物文件...)」，不静默丢图	<code>pdf.py:278-301</code> / <code>365-378</code> / <code>444-453</code>
无网址	<code>pdf.py</code> 全文不含协议链接；正	<code>reporting/content.py:118-</code>

项	内容	位置
	文来自 Markdown，固定模板路径由 <code>_strip_urls</code> 清洗	121、reporting/report.py:650
落盘	两条路径（流水线 / 多 Agent）都会生成；失败只 warning 不影响主流程	CHANGELOG.md 第 27 条

URL 清洗规则（reporting/content.py:37-38、118-121）：`http/https` 形式的链接 → （外链已省略）；`/files/...` 或 `/api/...` 绝对路径 → （路径已省略）。**图片相对路径不受影响**（charts/xxx.png 仍原样内嵌）。

边界说明：`build_report_pdf(markdown=...)` 若被外部传入未清洗的 Markdown，`pdf.py` 自身不做剥离（pdf.py:468）；固定模板路径不存在这个问题。

实测（本次运行）：`var/runs/20260915-174954-6983/report.md` 实际包含 # 分子对接筛选报告 + 9 个 ## 章节，第 1.4 节工具版本表为 `vina 1.2.7 / rdkit 2026.03.6 / meeko 0.8.0 / p2rank p2rank_2.5.1 / python 3.12.3 / matplotlib 3.11.1`（`var/runs/20260915-174954-6983/report.md:75`）；`report.pdf` 大小 476,915 字节。

9.5 图表与 CSV 产物

来源：`reporting/charts.py`、`reporting/report.py:63-73`、`reporting/tables.py:13`。

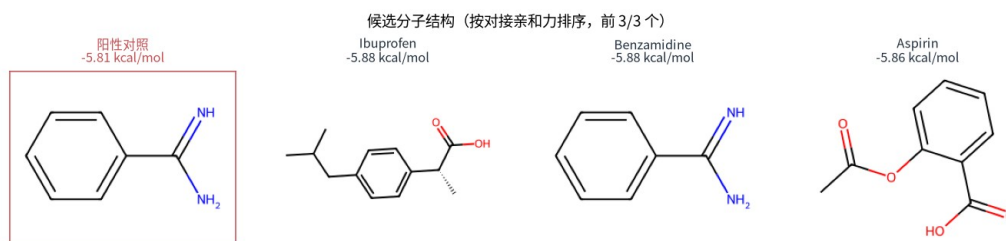


图 10 候选分子结构网格（按对接亲和力排序，左一为阳性对照）

表 58 图表与 CSV 产物

产物名	文件名	生成函数	参数
docking_chart similarity_chart	/ charts/ docking_chart.png	/ docking_bar_chart similarity_chart	/ 分子数 > CHART_BAR_MAX(50) 时改画

产物名	文件名	生成函数	参数
	charts/similarity_chart.png		Top-CHART_TOP_N(20)（相似度图按相似度降序），否则全画；对接图带对照参考线
affinity_histogram	charts/affinity_histogram.png	affinity_histogram	bin 数 $\max(10, \min(60, \text{int}(\sqrt{N}) * 2))$ ；均值/中位数/对照三条参考线
property_chart structure_grid binding_scatter	/ charts/property_chart.png / / structure_grid.png / binding_scatter.png	同名函数	性质图：横轴 MW、纵轴 logP、点大小 $\max(30, \text{abs}(\text{aff}) \times 22)$ 、Lipinski 边界（500 Da / logP 5）；结构网格默认 top_n=12, cols=4、单分子 260×200；结合模式散点默认 top_n=300，无相似度数据时不生成（返回 None）

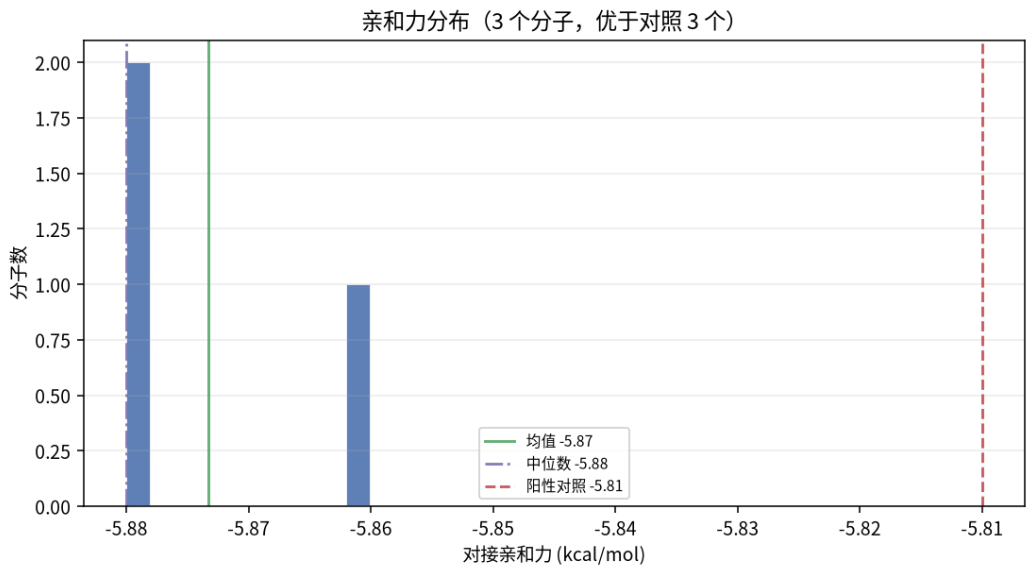


图 11 亲和力分布直方图（绿色 / 紫色 / 红色虚线分别为均值、中位数与阳性对照）

- PNG 统一 dpi=140, bbox_inches="tight" (charts.py:52)；中文字体自动挑选系统 CJK 字体并修正负号 (charts.py:27-42)。
- `ranking.csv` 共 32 列 (tables.py:13-24, 顺序照录)：

rank, name, smiles, affinity_kcal_mol, engine, exhaustiveness, box_group, box_size, seed, seed_policy, intermolecular_kcal_mol,

intramolecular_kcal_mol, torsion_kcal_mol, molecular_weight, logP, tpsa, hbd, hba, rotatable_bonds, aromatic_rings, formula, lipinski_violations, drug_likeness_pass, similarity_to_positive_control, maccs_tanimoto, combined_similarity, structural_consistency, anchor_match, binding_mode_hint, vs_positive_control_kcal_mol, status, error。

- 成功行 rank=1..N、status=ok；失败行 rank=FAIL、status=error；对照行 rank=PC、status=control (tables.py:89-121)。

9.6 产物清单与下载命名规范

来源：runs.py:56-111、docs/api.md § 3.1、docs/architecture.md § 13。

```
dock_{run_id}_{receptor}_{N}mols_{kind}.{ext}
例（分子数已知）：dock_20260914-151116-3404_thrombin_8mols_report.pdf
例（分子数未知）：dock_20260914-151116-3404_thrombin_report.pdf
```

表 59 产物清单与下载命名规范

规则	内容	位置
允许字符集	[A-Za-z0-9._-]，其余连续字符 段整体替换为单个 _	runs.py:53
截断与 fallback	受体名截断 24 字符并两端 strip("._-")（空则用 receptor）；run_id 截断 64 (fallback run)	runs.py:72-73
分子数	仅在 isinstance(molecules, int) and molecules > 0 时追加 _{N}mols；未知时 整段省略 （不 写 namols）	runs.py:75-76
kind.ext	report.pdf / report.md / ranking.csv / data.zip / poses.zip	runs.py:105-109
唯一实现	命名只在 runs.download_name() 一处实现；经 GET /api/runs/{id} 的 downloads 字 段下发给界面， 前端不自己拼	runs.py:94-111 、 docs/api.md § 3
路径与读回	产物路径一律 相对运行目录 登 记；对外 URL 为	runs.py:118/129/499、reporting /artifacts.py:74

规则	内容	位置
	<code>/api/runs/{id}/artifacts/{name}</code> ; 读回用 <code>Path(run.dir) / art["path"]</code> 并拒绝 <code>..</code> 与前导 <code>/</code>	

``ARTIFACT_BASE_URL`` 的实际作用域（重要澄清）：run 级产物走相对 URL，不经过 `ARTIFACT_BASE_URL`；该变量只影响旧的扁平接口 `GET /files/{key}`（`reporting/store.py:42-49`、`reporting/store.py:61`）。其默认值由协议头、`ARTIFACT_HOST`（默认 `127.0.0.1`）与端口拼成。

运行目录产物清单（`README.md §7`，实测目录 `var/runs/20260915-174954-6983/`）：
`run.json`、`request.json`、`molecules.json`、`properties.json`、`docking.json`、`binding.json`（有对照时）、`pockets.json`、`ranking.csv`、`ranking.json`、`report.md`、`report.pdf`、`result.json`、`charts/`（6 个 PNG）、`poses/`，以及各工具的原始返回 `*_tool.json`。

10. 工程实现与质量保障

结论：分层依赖与不变量有明确清单；门禁全绿且项数是本次实测；真实性取证脚本不信任自己的封装，用独立复算与反证控制看护「不编造」这条产品底线。

10.1 分层依赖与不变量清单

来源：docs/architecture.md §5。

表 60 分层依赖与不变量清单

抽象	位置	不变量
task_spec	intake.build_task_spec run.data["task_spec"]	→ 只读契约：编排层不得修改； decision ∈ {run, ask, reject}；missing 不构成阻断
blackboard	runtime/blackboard.py	进程内、单次运行隔离；快照 上限 20 (pockets) / 50 (其余 四类)；set_receptor 不覆盖已 固定的位点
tool_io	runtime/tool_io.py	工具原始返回全量落盘 (*_tool.json)，给模型的只 是视图；AGENT_TOOL_TOP_N 是视 图大小，不是输入上限
runs 产物	runs.py	任何数值都能在产物里找到出 处；result.json 是落盘白名 单；产物清单随运行增长要限 流
persistence	agents/persistence.py	唯一的合并/落盘入口；同工具 多次调用按 (name, smiles) 去 重、精度优先； _PROVENANCE_KEYS 用工具原始输 出回填模型漏掉的字段
绝对/相对路径	runs.py、reporting/ artifacts.py	产物登记一律相对运行目录
SSE 事件	runtime/streaming.py	每个事件带服务端 ts
缓存	core/receptors.py、core/ pockets.py、core/params.py	按内容哈希判定（不是时间 戳）；缓存键必须包含影响结 果的输入（如 keep_hetatm）

抽象	位置	不变量
取消	<code>cancellation.py</code>	协作式；取消是正常终态 (<code>status=cancelled</code>)
<code>param_plan</code>	<code>core/params.py</code> <code>run.data["param_plan"]</code> →	运行级/阶段级；用户显式参数冻结 (<code>source="user"</code>)；每条决策都有 <code>decisions</code> 理由
运行记录	<code>runs.Run</code>	<code>run.data</code> 存 <code>request/task_spec/created_at/agent_models/completeness/live_progress</code> ； <code>calls</code> 计数每次运行开始时重置

``\_PROVENANCE\_KEYS`` 回填清单 (`agents/persistence.py:40-46`，16 个键)： `box_source`，`box_chosen_by`，`box_validation`，`pockets`，`box_warnings`，`site`，`pdabt`，`protein`，`box_library_floor`，`box_group_sizes`，`box_group_counts`，`dropped_hetatm`，`kept_hetatm`，`dropped_waters`，`cocrystal_ligand`，`unsupported_hetatm`。模型转述时漏掉这些字段也会被工具原始输出补回（数据正确性不依赖模型）。

10.2 测试与门禁全清单（项数为本次实测）

表 61 测试与门禁全清单（项数为本次实测）

门禁	本次实测项数	作用	阻断性
<code>.venv/bin/python -m pytest tests -q</code>	315 passed (2 warnings, 147.19 s)	核心算法、API、设置、受理、协作、口袋、盒子一致性/库级下限/超限分组、特殊化学、可复现性、文档一致性、并发隔离	阻断
<code>scripts/smoke_test.py</code>	46/46 通过	Fake LLM 驱动真实多 Agent + 真实 Vina + 运行目录检查（不消耗模型额度）	阻断（动提示词/编排必跑）
<code>scripts/verify_docking.py</code>	52/52 通过	对接真实性取证：独立复算、参数敏感性、反证控制	动 <code>core/docking.py</code> 等必跑
<code>scripts/check_web.py</code>	118/118 通过	前端静态：语法、离线可跑、id 对应、结构配平、	动前端必跑

门禁	本次实测项数	作用	阻断性
		布局健壮性、设计令牌、导出条与动效、对话附件契约、多轮会话、工具提示与高级设置栅格	
scripts/snapshot_ids.py diff	基 线 213 → 当前 241（新增 28，无 id 被移除）	防止误删动态引用的 DOM id	动 index.html 必跑
node scripts/ui_e2e.js <base>	116/116 通过	jsdom 真加载页面 + 真点按钮 + 真读接口 + 合成 SSE 并行场景	可选（需起服务）
scripts/ui_shot.py check --base <url>	6/6 页通过	真 浏 览 器（Chromium）：控制台错误 / 横向溢出 / 图片加载失败	动前端必跑
scripts/intake_eval.py	14 个用例（确定性耗时合计 97.5 ms，平均 6.97 ms/次，零模型）	受理层决策（run/ask/reject）用例集	动 intake/提示词必跑
scripts/lint_local.py	通过（新增 0 项；存量豁免 249 项；文件 85 个 / 函数 1,090 个；静默 except 0、裸 getenv 0、print 0、缺 future 注解 0、超长文件 0；缺返回类型 251、函数内 import 457 为存量）	静 态 规 范：静 默 except、print、裸 getenv、future 注解、返回类型、超长文件	阻断（秒级）
bash start.sh --check	通过（真实跑一次对接 + 环境自检，输出「自检完成。」）	部署前环境自检	部署前

文档里的项数与实测有出入，以实测为准。举例：README.md §13 写 pytest 251 项、verify_docking 52 项、check_web 118 项、ui_e2e 104 项；docs/architecture.md §10.2 写 pytest 243 项、verify_docking 47 项、check_web 88 项、ui_e2e 70 项。本次实跑结果为 **pytest 315 / smoke 46 / verify_docking 52 / check_web 118 / ui_e2e 116**。这本身就是「文档漂移」风险的一个实例，tests/test_docs_consistency.py（6 条断言）正是为此设立。

测试文件分布（实测用例函数个数，共 20 个测试文件）：**test_intake** 35、**test_core** 33、**test_settings** 28、**test_pockets** 28、**test_agent_context** 23、**test_ligand_receptor_chemistry** 19、**test_unresolved_receptor** 16、**test_resolve** 15、**test_report_pdf** 13、**test_resilience** 12、**test_api** 11、**test_conversation**

10 、 test_box_sizing 10 、 test_collaboration 9 、 test_param_plan 8、 test_upload_report 7、 test_online_resolution 7、 test_docs_consistency 6、 test_receptor_race 2、 test_live_molecules 1（pytest 收集到的用例数 315 含参数化展开）。

10.3 真实性取证与反证控制

来源：scripts/verify_docking.py（模块 docstring）、README.md § 13。

表 62 真实性取证与反证控制

层	内容
A 引擎真实性	vina 为原生扩展；受体为真实蛋白（2771 原子、含催化三联体 HIS57/Asp102/Ser195、已去水）；对接代码剔除注释后无 mock/fake/硬编码分数
B 独立复算	用 vina API 从零重写对接（仅硬编码受体路径与盒子），与工具输出逐位一致；配体 PDBQT sha256 相同；位姿用全新实例重打分一致
C 工具交叉验证	无 LLM 直接调用 @tool：能量恒等式 total = intermolecular + torsional 对全部 8 分子成立（最大偏差 0.01）、重复调用逐位一致、不同分子分数各异
C2/C3 产物与参数	报告产物结构（真实 CSV 字节、行数、升序、合法 PNG）；参数真实透传与敏感性
D 反证控制	盒子平移 25 Å / 换受体 → 分数改变；非法 SMILES → 不出数值；engine=autodock（未安装）→ 明确报错；BOX_GROUP_MARGIN=100 → 所有分子回到同一盒子（分组判据真实生效）
E agent 闭环	同一次真实 LLM 运行内三方对齐：LLM 转交报告工具的数值 == 对接工具原始返回；相同参数重放 == 当时返回；落盘 CSV == 工具返回

本次实跑 D 层的真实数字（与 § 13.2 表二为同一批实测，此处不重复展开）：盒子中心平移 25 Å → -5.78 → -5.12；换受体 → thrombin -5.78 vs trypsin -6.55；非法 SMILES → 该行只有 error="配体化学解析失败：SMILES 无法解析：这不是 SMILES"、无亲和力数值；engine=autodock → 所有对接引擎均失败(vina/autodock)：

AutoDock CPU 引擎不可用：未找到 autodock4/autogrid4...； BOX_GROUP_MARGIN=100
→ box_size=[(30.83, 30.83, 30.83)] / box_group=['large']；搜索强度 {1,3,6,16}
→ {-5.84, -5.87, -5.78, -5.85}。

10.4 上下文预算（大库不撑爆模型）

来源：docs/architecture.md § 8.3、runtime/tool_io.py:58。

表 63 上下文预算（大库不撑爆模型）

机制	位置	作用
工具视图截断	tool_io.summary_limit() = max(1, min(500, AGENT_TOOL_TOP_N)), 默认 20	实测 10k 行结果 4.69 MB → 模型只见 3,881 B (summary + top)
分子库离线化	intake._render_big_library (in take.py:967)	大清单写 ligands_input.csv，指令里只给条数 + 预览 + 路径
结果内联截断 / 工具返回截断	RESULT_INLINE_LIMIT （默认 200） / STREAM_TOOL_RESULT_CHARS （默认 2000）	HTTP 详情不整包内联；SSE 不回传超长工具原文（本次实测 fetch_protein_structure 返回被截到 2,017 字符，原文 9,826 字符）
分片护栏 / 记忆窗口	AGENT_SHARD_MAX （默认 8） / MAX_MESSAGES = 40	禁止把库切成几十片逐片调度；主管消息滑窗 (agents/coordinator.py:41)

大库实测（`README.md § 4`，本机 32 核）：96 分子 exh=1 串行 159.8 s → 8 进程 28.2 s (5.7×)，结果逐位一致；126 分子 HTTP 全流程 9.4 s；1 万行分页查询 3–6 ms (22 KB/100 行)；1 万分子运行的 GET /api/runs/{id} 138.8 KB；1 万行 CSV 0.74 MB / 59 ms。（与 § 13.1 表重复列出，便于本节自洽阅读。）

10.5 文档体系、并发隔离与静态规范

表 64 文档体系、并发隔离与静态规范

文档	定位
仓库根 README.md	门面：这是什么、一分钟怎么跑起来
../docs/技术文档.md	读者向概览：框架 / 原理 / 使用说明（短版，与本文同主题）

文档	定位
README.md（工程主文档）	安装、配置、界面、接口速查、运行记录格式、门禁与验收
docs/architecture.md / docs/api.md	开发者视角总图（分层、不变量、扩展步骤、门禁、陷阱、路线图） / HTTP/SSE 契约与各子系统细节
CHANGELOG.md	历史修复日志（62 条，按时间编号）

文档分工（2026-09 文档收敛）：本文是**评审用完整叙述**，含实测数据与来源引用；读者向短版是 `../docs/技术文档.md`；工程细节与验收在 `README.md` 与 `docs/architecture.md`；历史日志在 `CHANGELOG.md`。四者不重复叙述同一件事，需要细节时互相指路。

文档也进测试：`tests/test_docs_consistency.py`（6 条断言）用机器判定「文档说的」与「代码做的」是否一致——已删除的角色不许复活、角色名必须来自 `ROLES`、代码读到的环境变量必须在 `.env.example` 登记、`README` 必须指向开发手册、承诺的门禁脚本必须存在、不得再承诺已移除的「自动补齐」。这是 2026-09-14 文档审计（30 处不一致）的直接产物：**错误文档比没有文档更糟**。

表 65 并发隔离与缓存正确性

机制	实现	守护
受体准备产物内容寻址	文件名带源内容哈希 + keep 集哈希，展示名与文件名分离	<code>tests/test_receptor_race.py</code> （3 线程并发，2 个用例）
缓存键含影响结果的全部输入	受体准备 <code>.sha1</code> sidecar；库级跨度缓存键含库 SMILES sha1 + K；库统计缓存键含库内容 + 抽样规模	<code>core/receptors.py:269</code> 、 <code>core/pockets.py:248</code> 、 <code>core/params.py:110</code>
子 Agent 记忆隔离	每角色独立 checkpointer + 每次调用新 <code>thread_id</code>	<code>agents/workers.py:63/159</code>
计数与测试隔离	每次运行开始 <code>reset_registry_counters()</code> ； <code>tests/conftest.py</code> 把 <code>LOCAL_SETTINGS_PATH</code> 指到临时路径	<code>CHANGELOG.md</code> 第 20 条、 <code>README.md §15</code>

静态规范与增量门禁（`scripts/lint_local.py`，本次实测存量）：静默 `except: pass 0`、`src/` 非 CLI 的 `print 0`、裸 `os.getenv 0`、缺 `from __future__ import`

annotations 0、>700 行文件 0（7 处已登记豁免）；公共函数缺返回类型 **251**、函数内 import 未注明原因 **457**（多为重依赖与 2 处破坏，允许存量）。
scripts/lint_baseline.json 只允许收窄（修好一条就 `--update-baseline`）。**分层依赖检查**：`core/*` 不 import `tools/*`、`agents/*`、`api/*`；唯一例外是 `core/receptors.py` 在函数内延迟导入 `core/pockets.py` 以避免循环依赖（`docs/architecture.md` §2.1）。

11. 可信度、局限与风险

结论：计算链路与「准备/对接/评估/报告」四段是真实且有测试看护的；「主管协调」「自动异常处理」「结果可信度」三段目前主要是提示词纪律 + 工具事实上报，缺少代码级强制与兜底 —— 这是下一阶段的主战场（`docs/architecture.md § 15` 总判断）。

11.1 打分函数与方法的固有边界

表 66 打分函数与方法的固有边界

局限	说明	证据
Vina 打分函数是近似	系统只编排引擎、不自研打分；分数高低是同一打分函数下的相对量，不等同实验亲和力	§ 1.2、§ 15.3 第 3.2 条
受体质子化态（已实现，v0.23 起；v0.32 修复解析缺陷）	<code>pdb2pqr --ph-calc-method=propka --with-ph=<pH> + meeko</code> 按目标 pH 准备受体，逐残基 pKa/HIS 状态留痕。 v0.32 修复 ： <code>pdb2pqr</code> 固定列下 4 位残基号（ <code>GLU A1005</code> ）会让 <code>meeko</code> 把 x 坐标当残基号（ <code>int('128.990')</code> ）导致四档全败并静默回退；现按「最后 5 个 token = x y z charge radius」反推残基键， <code>opt</code> 首档即成功	<code>core/receptor_ph.py</code> 、 <code>tests/test_receptor_ph_parsing.py</code>
结构水全部剔除；缺失残基 / loop、点突变、二硫键、金属配位几何只检测不建模	无法保留桥接水/配位水；断链口袋变形时结论不可用	§ 15.1 第 1.8 / 1.2 条（ 不建议 做建模，§ 15.8 第 1 条）
配体互变异构/质子化态： 微观态已在目标 pH 窗口内枚举，但只取一个形式对接 ；单一 3D 构象；立体化学只告警不处理	Dimorphite-DL 给出窗口内多个微观态后按固定规则取一个（未做布居/能量比较）；柔性分子可能落在差的局部极小 → 假阴性；手性药物对接的是随机对映体	§ 15.1 第 1.3 / 1.4 / 1.5 条（1.3 部分完成）
无重打分 / 共识打分 / 位姿 RMSD / 聚类	单一 Vina 分直接决定结论， 无不确定性度量	§ 15.3 第 3.2 条（P0）
无相互作用指纹 / 接触图 / 残基能量分解；无富集 / 显著性（ROC/AUC/EF/p）；ADMET 仅	无法解释盐桥/氢键网络；无法判断「优于对照」是信号还是噪声；类药性判断偏窄；无法估计	§ 15.3 第 3.3 / 3.4 / 3.5 / 3.6 条

局限	说明	证据
Lipinski; 无阴性对照/诱饵集	假阳性率	
PAINS / 反应性基团 / 聚集剂过滤 全缺	榜单头部可能是泛检测干扰物 → 湿实验浪费 (RDKit <code>FilterCatalog</code> 已带目录, 成本极低)	§ 15.3 第 3.1 条 (P0 + S)

上表「证据」列的 § 均指 docs/architecture.md。

11.1b 质子化引擎 (v0.32 起: 受体与配体都用专业工具)

受体与配体按**同一目标 pH** 处理; 两侧各用专业工具, 且在报告与产物里逐条留痕:

表 67 b 质子化引擎 (v0.32 起: 受体与配体都用专业工具)

侧	引擎	版本溯源	回退
受体	pdb2pqr (<code>--ph-calc-method=propka</code> , 即 PROPKA 逐残基 pKa) → PQR → meeko PDBQT	运行记录 <code>receptor_protonation.tool</code> / <code>pdb2pqr_version</code> 、PROPKA 摘要随产物交付	找不到 pdb2pqr / 需保留有机辅因子 → 回退标准模板准备并标 <code>applied=false</code> + 原因
配体	Dimorphite-DL (RDKit pKa 规则库), <code>LIGAND_PKA_ENGINE=auto</code> 默认	<code>ligand_facts.protonation.engine / engine_version</code> ; 报告 § 1.3 显示「质子化引擎: dimorphite-dl 2.0.2 (N 个分子)」	未安装/异常 → 回退内置 pKa 规则表 (近似), 记 <code>engine_fallback_reason</code> + 安装提示

- **多微观态的取舍**: Dimorphite 在 pH 窗口 (`LIGAND_PKA_WINDOW`, 默认 ± 0.5) 内会返回多个

微观态且不给布居数; 系统按「**|净电荷|** 最小 → 带电原子最少 → 字典序」取一个形式对接, **全部候选与规则** 写进 `ligand_facts.protonation.variants / variant_rule`, 报告 § 6 同步声明「未做布居与能量比较」。需要逐态枚举时用 SDF 提供已准备形式并选 **keep**。

- **安装 (重要)**: Dimorphite-DL 2.0.2 的元数据把 `rdkit` 钉在 **<2026**, 而本项目要求 `rdkit>=2026.3.6`; 直接安装会**降级 RDKit**。用 `--no-deps` 安装 (实测 2.0.2 与 2026.3.6 兼容):

```
uv pip install loguru && uv pip install --no-deps dimorphite-dl
```

uv pip check 会因此报一条**预期内**的 `dimorphite-dl requires rdkit<2026` 提示，可忽略。

- **探测**：`bash scripts/doctor.sh`、`POST /api/tools/probe`（字段 `pka`）、设置页「检测」三处同源。

11.2 盒子 / 参数效应（引实测）

表 68 盒子 / 参数效应（引实测）

风险	实测依据	系统对策
换盒子会改变分数且 非单调	同一配体在 18 ³ /22 ³ /28 ³ /34 ³ 下最大差 1.34 kcal/mol (<code>docs/architecture.md § 15.7b B0 表</code>)	主组盒子唯一 ：一个受体只定一次盒，全库共用
大配体在固定小盒下被欠采样	六肽 22 ³ -7.88 vs 34 ³ -8.21 (同上表)	库级下限 P95 + 10 Å + 超限分子分组（同一中心）
采样强度同样影响分数	<code>exhaustiveness ∈ {1,3,6,16}</code> → <code>{-5.84, -5.87, -5.78, -5.85}</code> (<code>verify_docking.py C3 层实测</code>)	参数是运行级/阶段级，同一 pass 内完全一致
小分子白吃大盒算力	小分子耗时随盒体积增长：0.5 s → 1.5 s（约 3×） (<code>§ 15.7b B0 表</code>)	<code>min_size=18 Å</code> 下限 + 体积系数 <code>f_box</code> 抬高采样强度

11.3 同阶段内可比性保证

表 69 同阶段内可比性保证

保证	内容
同阶段参数一致	同一 pass 内 <code>exhaustiveness / box_size / box_center</code> 完全一致（回归测试 <code>tests/test_param_plan.py</code> ）
同阶段盒子一致	主组共用一个盒子；报告写明「盒子效应对相对排序无影响」
不跨组比较	<code>large</code> 组单独成节，默认排序榜只含 <code>main</code> 组
参数逐行留痕	<code>ranking.csv</code> 带 <code>engine / exhaustiveness / seed /</code>

保证	内容
	seed_policy / box_group / box_size
批次无关	同一分子分数与批次组成/进程数无关（永久回归测试）

11.4 运行级与工程级缺口（可信度风险的直接来源）

来源：docs/architecture.md § 15.1– § 15.6。以下为待做项，标注优先级与工作量（S ≤ 半天，M ≤ 2 天，L > 2 天）。

表 70 运行级与工程级缺口（可信度风险的直接来源）

优先级	缺口（编号）	影响	工作量
P0	Agent 模式无法取消对接（4.1, cancel_event 未传入工具）；崩溃/断线无兜底（4.2，无 reaper/resume/retention，run.json 永久 running)	点「停止」后 Vina 进程池继续吃满 CPU；前端断流后线程与进程池继续跑且再也取消不了	S / M
P0	失败清单不归因、不进报告（4.4 + 5.2，逐分子 error 只在 JSON)	用户读到「全成功」的假象；审计列为「下一步最高优先」	S
P0	无动态重规划（6.3）；失败升级只到提示词，_completeness 明确「只作信息展示，不接管控制」（6.4)	子 Agent 失败后主管可能仍宣告成功；「保证项目正常工作」的兜底不存在	M
P1	漏斗参数只在提示词不强制（2.1）；单分子失败不重试/不降级（2.4）；无磁盘护栏与位姿清点（2.5）；无端到端墙钟超时（4.3)	LLM 不照做可能一次性 exh=6 跑完万级库；一个分子准备失败即失去该分子；万级库可写数万小文件；卡死运行永久占资源	M/M/S/S
P1	多受体结果被坍塌成第一个受体（5.3, merge_and_rank 以 smiles 为键）；两轮漏斗 pass/affinity_coarse 不进 CSV/报告（5.4)	声明的「蛋白质库」能力拿不到并列对比；报告无法说明粗筛/精算范围	M/S
P1	计划不是显式产物（6.1）；预算与步数控制	运行不可重放、不可断言「计划被遵守」；一次运	M/M/M/S

优先级	缺口（编号）	影响	工作量
	制形同虚设（6.2，只有 <code>recursion_limit</code> ）；运行图/时间轴不落盘、usage 硬编码 0（6.5）；前端未展示 <code>task_spec/completeness/blackboard_stats</code> （6.6）	行可能烧掉任意额度；事后无法审计	
P2	SSE 无重连回放/keepalive、队列无界（6.7）；无运行保留期/磁盘配额（4.8）	网络抖动即丢事件、无背压；长期无人值守必然积压	M/M
P2	<code>pass="coarse"</code> 标记丢失（2.2）； <code>top_from_previous</code> 兜底来源键名错误（2.3）；AD4 行里无 <code>exhaustiveness/seed</code> （2.6）	轮次不可区分；黑板为空时报 <code>no_previous</code> ；回退分数混进同一张排序表	S/S/S

11.5 存留与边界

表 71 存留与边界

项	边界	处置
多轮会话记忆 / 运行记录	默认 <code>CHECKPOINT_BACKEND=memory</code> 只在 进程内 、 服务重启即失效 ；但 <code>var/runs/<run_id>/</code> 始终 落盘 ，重启后仍可在「历史运行」查看	要跨重启保留对话设 <code>sqlite</code> （写 <code>var/checkpoints.sqlite</code> ，需装 <code>langgraph-checkpoint-sqlite</code> ）；产物无自动清理/保留期（缺口 4.8）
并发用户	子 Agent 在工具内同步调用，单进程串行；多用户高并发建议多进程部署（每进程独立 <code>checkpointer</code> ）	<code>README.md §15</code>
孤儿对接进程	服务被强杀（Ctrl-C 多次、 <code>kill -9</code> 、断线）后 worker 可能被 <code>init</code> 收养继续吃满 CPU	用 <code>scripts/kill_leftovers.sh</code> 清理， 必须在宿主 shell 运行
无鉴权	设置接口能改 LLM 端点，连通性测试会带当前密钥发请求	默认只听 <code>127.0.0.1</code> ；确需局域网访问时自行加反向代理鉴权/防火墙

12. 部署与运维

结论：一键启动、默认只听本机、设置优先级链明确（角色环境变量最优先）、部署保护项只读、残留进程有专用清理脚本且必须在宿主 shell 运行。

12.1 一键启动与自检

来源：start.sh、README.md §1。

表 72 一键启动与自检

命令	作用
bash start.sh	创建虚拟环境 → 安装依赖（首次）→ 加载 .env → 选择可用端口 → 启动服务 → 自动打开浏览器
bash start.sh --port 8000	指定端口（默认 5000，被占用时自动顺延）
bash start.sh --no-browser	不自动打开浏览器
bash start.sh --check	只做环境自检（跑一次真实对接，不启动服务）；本次实测通过，输出「自检完成。」
bash start.sh --setup	强制重装依赖
bash scripts/local_run.sh -m http -p 5000	等价的脚本入口
bash scripts/local_run.sh -m pipeline -i "阿司匹林:CC(=O)Oc1ccccc1C(=O)O" --receptor trypsin	命令行跑确定性流水线
bash scripts/local_run.sh -m agent --message "用示例库做一次完整筛选"	流式多 Agent（终端实时输出）
bash scripts/local_run.sh -m runs --list 10 / -m receptors	查看历史运行 / 查看受体与已知位点

CLI 常用参数（README.md §5）：-i、--molecule-file、--receptor、--site-center 31.5,13.74,24.36 --site-size 22,22,22 、 --positive-control 、 --exhaustiveness N、--n-poses N、--engine auto|vina|autodock|external、--max-ligands N、--no-poses。

12.2 端口与监听

表 73 端口与监听

项	默认	说明	位置
PORT	5000	被占用时 <code>start.sh</code> 自动顺延； <code>--port</code> 优先	<code>.env.example:47</code> 、 <code>setting s.py:318</code>
HOST / ARTIFACT_HOST	127.0.0.1 / 127.0.0.1	默认只监听本机；服务本身无鉴权；ARTIFACT_HOST 是旧扁平产物 URL 的主机名	<code>.env.example:49/164</code>
UVICORN_LOG_LEVEL	info	uvicorn 自身日志级别	<code>.env.example:166</code>
局域网访问	需 <code>python -m docking_agent -m http --host 0.0.0.0</code> ，并自行加反向代理鉴权/防火墙	安全提示	<code>README.md § 2.1</code>

本次实测：以 `-p 5110` 启动后 `GET /api/health` 返回 `{"status":"ok","version":"0.8.0","llm_configured":true,"engine_available":{"vina":true,"autodock":true,"external":true,"external_flavor":"autodock-gpu"}, ...}`，`roles` 字段返回 6 个角色最终生效配置（`intake/property/pocket/binding` 温度 0.0、`coordinator` 0.2、`docking` 0.1）。

12.3 设置页优先级链与只读项

来源：`settings.py:10-26`、`README.md § 2.1`、`docs/api.md § 12`。

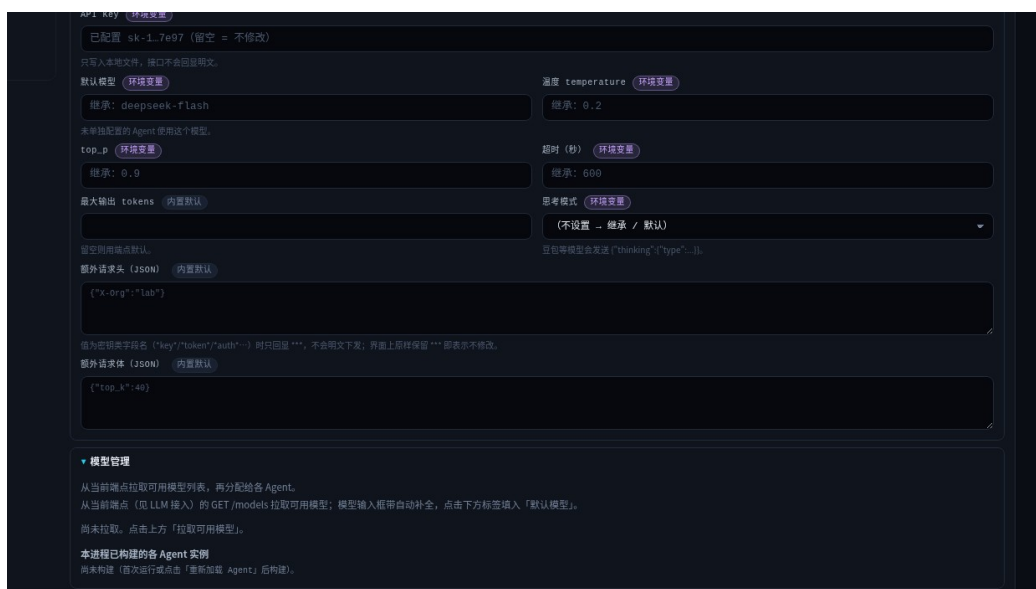


图 12 设置页：模型与部署级参数（局部视图；含 API 地址与密钥的字段已裁去）

LLM 字段（越具体越优先）：

角色环境变量 LLM_<字段>_<角色>

- > 界面设置（该 Agent） config/local_settings.json → roles.<角色>
- > 内置角色默认 config/agent_llm_config.json → roles.<角色>
- > 界面设置（全局） config/local_settings.json → llm
- > .env 全局 LLM_<字段>
- > 内置默认 config/agent_llm_config.json → config

实现上这 6 层由 `runtime/llm.py:140-154` (`_layers`) 按从低到高顺序叠加，每个字段的来源标签在 `resolve_role_config` 里一并返回（`runtime/llm.py:157-183`）。

运行类字段： 界面设置 > .env > 内置默认（`settings.py:20-23`）。
`apply_runtime_env()` 把运行类参数写进 `os.environ`，并记录注入基线（`injected_env_keys`），回滚时只删自己注入的键（历史 bug：`reset` 误删 shell/CI 注入的变量）。

只读（部署保护）项： `PORT`、`DOCKING_MAX_LIGANDS`、`UPLOAD_MAX_MB`（`settings.py:317-326`，`env_priority=True`）—— .env 优先，界面只读；
`reject_readonly()` 会整包拒绝提交（前端也已跳过只读字段，双重保护）。

需要重启的项：
`CHECKPOINT_BACKEND`（`settings.py:310`）、`ARTIFACT_BASE_URL`（`settings.py:312`）（`needs_restart=True`）。其余保存后在新的运行里立即生效。

密钥三条约定（[README.md § 2.1](#)）：[LLM_API_KEY](#) 与各角色 [api_key](#) 只写不读（界面只显示「已配置 + 首尾 4 位掩码」）；**额外请求头** 里名字含 [key/token/auth/secret](#) 的项只回显 *******（原样保留 ******* 提交 = 不修改）；上游失败的错误信息**脱敏**（[sk-...](#)、[Bearer ...](#) 等被替换）。

设置文件：[config/local_settings.json](#)（已 gitignore 与打包排除）；不存在时行为与之前完全一致；损坏或结构不对时**自动忽略并回退内置默认**，不影响启动（[settings.py:345-362](#)、[settings.py:365-385](#)）。

表 74 设置页优先级链与只读项

接口	作用
GET / PUT /api/settings	下发字段表（ specs ，前端不硬编码字段）/ 保存（出错字段标红并聚焦）
POST /api/settings/reload • POST /api/settings/reset	丢弃已构建的 Agent 与模型实例（模型/端点改动立即生效）• 删除设置文件回到 .env + 内置默认
GET /api/models • POST /api/settings/test	调端点拉取可用模型列表 • 真发一次最小请求测试连通性（只临时接管该角色并还原）

接口数量实测：[api/app.py](#) 有 **31 个路由装饰器**（[docs/architecture.md § 3.5](#) 写「~22 条」为旧值）。

12.4 目录结构

来源：[README.md § 6](#)、[paths.py](#)。

```
projects/
├── start.sh / pyproject.toml / requirements-local.txt / .env(.example)  # 启动、包元数据、依赖、本地配置
├── docs/                      # api.md / architecture.md / 技术报告.md
├── config/                    # agent_llm_config.json (内置模型参数 + 主管 sp)
│                               # local_settings.json (界面设置覆盖, gitignore, 可含 Key)
│                               # receptors.json (受体注册表与【已知结合位点】)
├── assets/                    # tools/ (可选外部工具, p2rank_2.5.1 解压即被识别)
│                               # libraries/ (示例库、阳性对照) receptors/ (registry/、structures/)
│                               # cache/ (box_span、param_stats、matplotlib) uploads/
├── src/docking_agent/         # ★ 单一 Python 包 (55 个文件 / 16,336 行)
├── web/                       # 前端 (index.html 886 / app.js 6,345 / styles.css 3,445 行)
├── scripts/                   # setup / local_run / http_run / smoke_test / verify_docking / check_web /
│                               # snapshot_ids / ui_e2e.js / ui_shot.py / shot.py / intake_eval /
│                               # lint_local / kill_leftovers / pack
├── tests/                     # pytest (20 个测试文件 / 6,253 行)
└── var/                       # 运行数据 (runs/ logs/ outputs/ cache/ diag/ e2e/ tmp/, 已 gitignore)
```

内置受体注册表：**thrombin**(1DWC，位点中心 **31.5/13.74/24.36**，尺寸 22³，来源「共晶配体质心」) 与 **trypsin**(1PTU，位点中心 **55.91/10.1/45.96**，尺寸 22³)；别名含 **1dwc→thrombin**、**1ptu→trypsin**；新增靶点无需改代码 (config/receptors.json、README.md §9)。

12.5 日志、产物与残留进程清理

表 75 日志、产物与残留进程清理

目录 / 命令	内容
var/runs/<run_id>/	每次运行的全部中间数据与产物（详见 §9.6）
var/logs/ · var/outputs/	服务日志（logging_setup.py 初始化）· 旧扁平产物目录（配合 GET /files/{key}）
var/cache/matplotlib	必须设 MPLCONFIGDIR 到此（HOME/~/.cache 只读）
var/diag/ · var/e2e/ · var/tmp/ · var/verify/	证据、端到端 SSE、临时目录、取证输出（/tmp 在两次 bash 调用之间不持久，必须用 var/tmp/）
LOG_LEVEL	默认 INFO（可选 DEBUG/INFO/WARNING/ERROR，settings.py:307）；src/ 非 CLI 代码禁止 `print`（实测 0）
bash scripts/kill_leftovers.sh --dry-run	只列出将被清理的进程，不做任何事
bash scripts/kill_leftovers.sh	清理工作进程，保留本项目 http 服务
bash scripts/kill_leftovers.sh --include-server	连服务进程一起停掉（--help 打印说明）

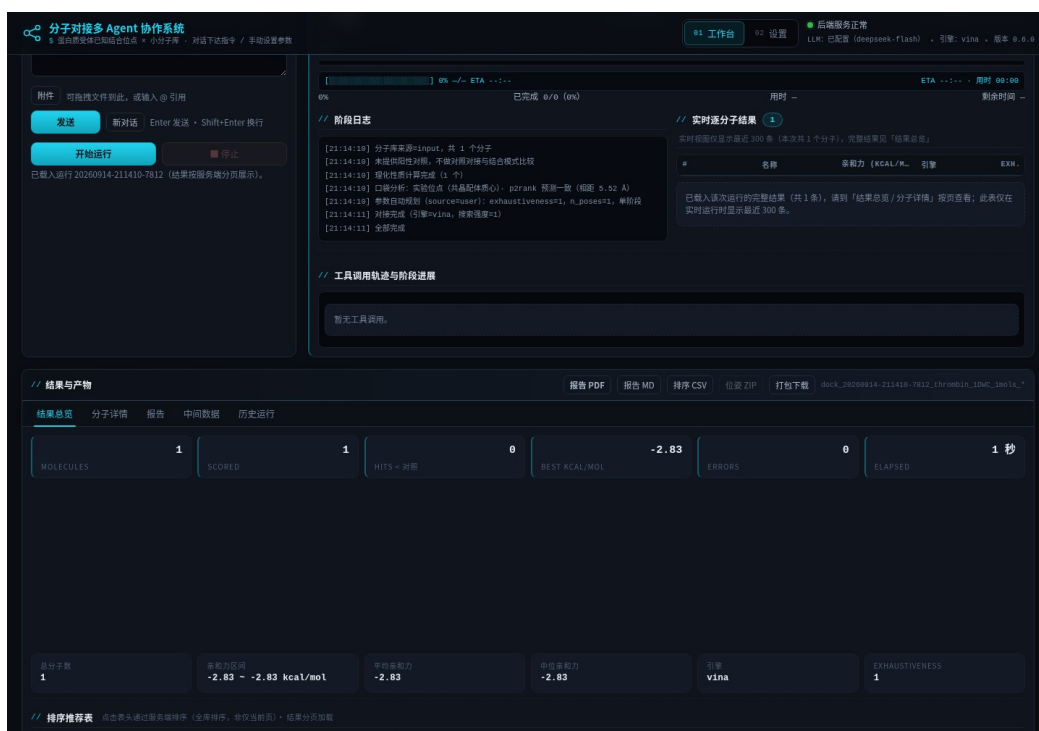


图 13 运行详情页：阶段日志、结果总览与产物下载（报告 PDF / MD / CSV / 位姿 ZIP）

识别规则（只处理当前用户且属于本项目的进程）：`/proc/PID/exe` 指向 `projects/.venv/bin/python`；或 `cwd` 在 `projects/` 下且 `cmdline` 含 `multiprocessing`；或 `cmdline` 含 `vina / mk_prepare_receptor / prank`。默认保护本项目 http 服务（`cmdline` 含 `docking_agent -m http`）与脚本自身。

必须在宿主 shell 里运行：容器/PID 命名空间内只能看到自己命名空间的进程，触达不到孤儿。不要用 ``pkill -f <关键字>``：会匹配到自己的 shell，命令自杀（exit 143）—— 这是 `docs/architecture.md §13` 明确记录的陷阱。

13. 实测数据汇总

结论：本章把散落各处的真实实测值集中成三张表。所有单元格都是仓库里已记录的实测数字或本次实跑结果，未实测的写「未实测」。

13.1 表一：并发、上下文与吞吐

表 76 表一：并发、上下文与吞吐

指标	数值	条件 / 来源
96 分子真实对接	串行 159.8 s → 8 进程 28.2 s (5.7×) ，结果逐位一致	<code>exhaustiveness=1</code> ，本机 32 核 (README.md § 4)
7 分子精算	40.0 s → 8.1 s (4.9×)	并发规划改为 1 进程 × 8 线程 (README.md § 11.4b)
126 分子 HTTP 全流程	9.4 s (含属性/对接/结合模式/图表/报告)	真实 HTTP 请求 (README.md § 4)
上下文压缩	10k 行结果 4.69 MB → 3,881 B (模型视图)	<code>tool_io.summary_limit</code> + <code>top</code> (docs/architecture.md § 8.3)
万级结果分页 / 详情 / CSV	分页查询 3-6 ms (22 KB/100 行)；详情响应 138.8 KB (内联前 200 条)；CSV 0.74 MB / 59 ms	<code>GET /api/runs/{id}/ranking / {id} / export.csv</code> (README.md § 4)
口袋单受体耗时	P2Rank $\approx$ 7 s (10 线程)；内置几何法 $\approx$ 1 s (2283 重原子, spacing 1.0 Å)	凝血酶 1DWC (README.md § 11.2/ § 11.2b)
取消响应时间	1.5 s 内结束 (否则需 30 s+)	协作式取消 (CHANGELOG.md 第 12 条)
受理层耗时	参数模式 0.02 ms (纯规则, intake 实例不构建)；对话模式 $\approx$ 2.5 s (1 次角色 <code>intake</code> 调用)	README.md § 1
<code>intake_eval.py</code> 确定性耗时	合计 97.5 ms (平均 6.97 ms/次, 14 个用例, 零模型)	本次实跑
端到端多 Agent 运行墙钟	73.26 s (1 分子 + 受体在线解析 + 准备 + 对接 + 报告 + 1 次 <code>keep_hetatm</code> 重跑；7YHP .pdb 现场准备为 PDBQT)	<code>run</code> 20260915-174954-6983 (<code>run.json</code>)

指标	数值	条件 / 来源
多 Agent 实测并行	属性评估 12.9→17.4 s 与对接 13.1→23.1 s 重叠 4.29 s ，微标 [并行 ×2]	真 实 运 行 chat+advanced (docs/api.md § 15.2)

13.2 表二：盒子与参数效应（全部为实测）

表 77 表二：盒子与参数效应（全部为实测）

实验	条件	结果	来源
换盒子大小	vina 1.2.7, 1DWC, 中心 31.5/13.74/24.36 , exh=4, n_poses=1, seed=42, threads=8	乙醇 -2.83/-2.81/-2.81/-2.85 ; 阿 司 匹 林 -5.87/-5.87/-5.86/-5.88 ; 华 法 林 - 7.97 /-7.75/-7.75/-7.75 5 ; 三 肽 模 拟 - 8.55/ -7.47 /-8.81/-7.60 ; 六 肽 - 7.27/-7.88/-8.08/ -8.21 (顺 序 18 ³ /22 ³ /28 ³ /34 ³)	docs/architecture.md § 15.7b B0 表
换盒子最大分差	同上	1.34 kcal/mol (三肽模拟 28 ³ -8.81 vs 22 ³ -7.47) , 且 非单调	同上
小分子耗时随盒体积增长	同上	乙醇 0.5 s → 1.5 s (约 3×)	同上
柔性大配体耗时	同上	六肽 12.6 s → 12.8 s (几乎不变, 主要由可旋转键决定)	同上
搜索强度敏感性	本次实跑 thrombin	exhaustiveness ∈ {1,3,6,16} → {-5.84, -5.87, -5.78, -5.85}, 极差 0.09 kcal/mol	本 次 verify_docking.py C3
盒子平移 25 Å / 换受体	本次实跑	原始盒 -5.78 vs 平移盒 -5.12 ; thrombin -5.78 vs trypsin -6.55 kcal/mol	本次 verify_docking.py D
BOX_GROUP_MARGIN=100	本次实跑	同一次调用里所有分子回到 同 一 盒 子 : box_size=[(30.83,30.83,30.83)]、box_group=['large']	本次 verify_docking.py D

实验	条件	结果	来源
手动指定 vs 默认对照	本次实验	阳性对照 benzamidine : - 5.78 @6 vs -5.87 @3 (README 文档值)	README.md § 12
保留 OXY / 保留 SF4 前后	1A6M 的 PDBQT + keep_hetatm="OXY" / 7YHP 本次 e2e	-6.97 → -7.15; SF4 保留 - 4.28 vs 未保留 -4.25	README.md § 11.4c、var/e2e/final_turn2.sse
内置几何法质量	1DWC, spacing 1.0 Å / 质量门用 1.5 Å 粗网格	top-1 距共晶配体 2.6 Å (top-2 2.97 Å); 粗网格 top-1 3.60 Å、top-2 22.8 Å, 因此门限是「top-3 内 ≤ 8 Å」	README.md § 11.2
P2Rank 质量	1DWC, 10 线程	top-1 4.54 Å (score 7.70, prob 0.407); top-2 6.02 Å; 共享 8 个残基	README.md § 11.2b
apo 受体自动换盒	删除凝血酶共晶配体	盒心从 41.52/21.84/19.17 (距真实位点 13.2 Å) 改为 32.89/11.85/19.37 (5.52 Å)	README.md § 11.2b
本次 e2e 盒子与口袋	7YHP	盒中心 129.6744/134.11885/125.12055、尺寸 22 ³ 、box_source="用户指定"、chosen_by="request"、box_library_floor=null; pocket1 score 31.55 / prob 0.939 (pocket2 8.14、pocket3 5.85)	docking.json、pockets.json

13.3 表三：端到端示例与「零计算」对照

示例 A（成功链路）：植物 ROS1 → Q9SJQ6 → 7YHP → EBDC 代表结构 → 亲和力 -4.28 kcal/mol。

表 78 表三：端到端示例与「零计算」对照

环节	实测值	来源
受理判定	task_type=docking_only、authority=chat、decision=run、receptor	var/e2e/final_turn1.sse

环节	实测值	来源
	<code>.source="named"、confidence=0.9</code>	
物种线索 / 检索	「植物」→ Arabidopsis thaliana / taxid 3702 / <code>is_plant=true</code> ；6 条检索命中数为 1/7/8/8/8/8	同上
选中受体与理由	Q9SJQ6 (<code>ROS1_ARATH</code> ，「DNA glycosylase/AP lyase ROS1」，gene <code>ROS1/DML1</code>)，打分 114.0 = 已审阅 +40 / 物种精确 +30 / 家族词 +20 / 基因精确 +15 / 注释 +9	同上
结构来源	RCSB 实验结构 7YHP （候选 PDB: 7YHO / 7YHP / 7YHQ）	同上
分子侧与第一轮	命中多组分聚合物 → CID 3034368 ， <code>C8H12MnN4S8Zn</code> ， <code>is_mixture=true</code> ；工具下发 4 个代表结构 <code>choices</code> ， 零对接	同上 (<code>choices</code> 事件)
第二轮（同 <code>conversation_id</code> ）	用户选定「最大有机片段 EBDC」→ 受体 自动继承 上一轮 accession	<code>var/e2e/final_turn2.sse</code>
对接结果	亲和力 - 4.28 kcal/mol ； <code>engine=vina、exhaustiveness=9、seed=42、seed_policy=session、box_group=main、box_size=[22,22,22]、ligand_span=[8.38,2.76,4.03]、pose_count=1</code>	<code>var/runs/20260915-174954-6983/docking.json</code>
自动规划	<code>exhaustiveness = clamp(round(12 × 0.75 × 1.00), 2, 32) = 9</code> ；P90 可旋转键 3；库规模 1；单阶段	<code>var/runs/20260915-174954-6983/report.md:92-109</code>
受体溯源 / 配体告警	<code>dropped_hetatm={"5CM":20}、kept_hetatm={"SF4":8}、unsupported_hetatm=[]</code> ；配体净电荷 -2（请确认质子化态）	<code>docking.json / run.json</code> 的 notes
完成度与调用次数	<code>{"docking":"agent","docked":1,"total":1,"ranking":"ok","report":"ok","positive_control":"skipped"}</code> ；property 5 / pocket 4 / docking 9 / binding 0 / coordinator 5 / intake 1，六者 <code>instance_id</code> 互不相同	<code>var/runs/20260915-174954-6983/run.json</code>

示例 B（歧义 → 零对接）：只写「ROS1」+ 乙醇。检索状态 **ambiguous**（7 个候选并列 **64.0**：P08922 人源 ROS1（RCSB 3ZBF/4UXL/7Z5W/7Z5X/9QEK）、Q78DX7 小鼠、Q63132 大鼠、**Q9SJQ6** 拟南芥、C7IW64 水稻、P08941 鸡、P25655 酵母）。系统行为：**零对接**，最终回复「受体『ROS1』在线解析出现歧义…本次**未执行任何对接计算**，请你从下列候选中选择目标受体」；整段流里只有 2 次 **tool_call**（解析与提问），**没有对接工具调用**（**var/e2e/final_ambig.sse**、**var/e2e/evidence_resolve.out** ②）。

示例 C（查不到 → 零对接 + 列出已尝试检索）：输入「不存在的蛋白 ZZZQQ9」，工具返回 **status=error**、**reason=not_found**（耗时 3.5 s），已尝试 3 条检索（**gene_exact:ZZZQQ9 AND reviewed:true / gene_exact:ZZZQQ9 / ZZZQQ9 protein**）均 0 条命中；系统回复「**本次不执行任何对接计算**，也不得擅自改用系统默认受体。请让用户三选一：① 提供 PDB 编号（如 4HHB）或 UniProt accession；② 写出受体的基因名/蛋白名（中英文均可）；③ 上传受体结构文件」（**var/e2e/evidence_resolve.out** ③；该次实测早于受体纪律收紧，**当时的第三项是「明确同意改用系统默认受体」**，现已删除——系统没有默认受体）。

示例 D（阳性对照缺失的如实标注）：指令未提供阳性对照 → **completeness.positive_control = "skipped"**，报告第 5 节写「未提供阳性对照分子，本次无基线数据」，图表不画对照线（**var/runs/20260915-174954-6983/run.json**、**reporting/charts.py**）。

14. 后续路线

结论：审计给出的总判断是 —— 计算内核与「准备 / 对接 / 评估 / 报告」四段真实且有测试看护；「主管协调」「自动异常处理」「结果可信度」三段缺少代码级强制与兜底，这是下一阶段主战场。以下按`docs/architecture.md § 15`原样归纳，并标注已完成 / 待做。

14.1 P0 / P1（按 docs/architecture.md § 15 原样归纳，标注状态）

表 79 P0 / P1（按 docs/architecture.md § 15 原样归纳，标注状态）

优先级	事项（编号）	状态	工作量	要点 / 验收标准
P0	PAINS / 反应性基团 / 聚集剂标记（3.1）	待做	S	ranking.csv 增 pains_alert 列；报告单列告警分子；只标记不静默剔除
P0	失败可见 + 单分子重试 / 降级（4.4+2.4+4.5）	待做	S+M	报告新增「失败与跳过清单」；CSV 含 status/error；降级写 notes；分发层不掩盖合法 {"status":"error"}
P0	运行级兜底：reaper + 墙钟超时 + Agent 模式可取消（4.1+4.2+4.3）	待做	M	过期 running 标 interrupted 并留产物；点停止 5 s 内 Vina 进程消失；AGENT_RUN_TIMEOUT_SECONDS 生效
P0	报告方法学与可复现性补全（5.1+5.2）	5.1 已完成（封面工具版本、CSV/ 报告 seed/seed_policy）；5.2 待做	S	5.2：失败清单章节进报告
P0	打分可信度：重打分 / 共识 / 位姿 RMSD/聚类（3.2）	待做	M	审计列为 P0，未进「最值得先做 5 件事」验收表
P0	动态重规划 + 失败升级兜底（6.3+6.4）	待做	M	「保证项目正常工作的兜底不存在」
P0	受体质子化态 / pH（1.1）；配体互变异构/质子化态枚举（1.3）	1.1 已完成（pdb2pqr+PROPKA，v0.32 修复 PQR 解析缺陷）；1.3 部分完成（微观态在 pH 窗口内枚举，取一个形式；未做布居/能量与	M	1.1「最影响物理真实性」；1.3 剩余：逐态枚举与打分对比

优先级	事项 (编号)	状态	工作量	要点 / 验收标准
		互变异构比较)		
P1	缺失残基/loop 只做检测 + 披露 (1.2 缩小版) ; 配体多构象/立体化学/大分子阈值 (1.4/1.5/1.6) ; 结构水保留 (1.8)	待做	S/M	结构断链写进报告局限性; 水可保留桥接水/配位水
P1	漏斗参数服务端强制 (2.1) ; 单分子失败重试/降级 + 磁盘护栏与位姿清点 (2.4/2.5)	待做	M/S	不靠提示词自觉; 万级库不写数万小文件
P1	相互作用指纹/接触图/残基能量分解 (3.3) ; 富集与显著性 ROC/AUC/EF/p (3.4) ; ADMET 扩展 QED/SAscore/结构警报 (3.5)	待做	L/M/S	解释盐桥与氢键网络; 判断「优于对照」是信号还是噪声
P1	分发层不掩盖合法 {"status": "error"} ; LLM 传输层重试/退避/限速 (4.5/4.7)	待做	S	端点抖动不直接使运行失败
P1	多受体并列 + 漏斗数据进报告 (5.3+5.4, merge_and_rank 键改 (receptor_key, smiles)) ; 报告补配体准备告警/盐拆分/box_fit_warning (5.6)	待做	M/S	报告按受体分表并列出各自盒子; 写明粗筛范围与 affinity_coarse
P1	计划成为显式产物 (plan/DAG/step, 6.1) ; 预算与步数控制 (6.2) ; 运行图/时间轴落盘 + token 计量 (6.5) ; 前端展示 task_spec/completeness/blackboard_stats (6.6)	待做	M/M/M/S	运行可重放; 可断言「计划被遵守」; 可信度信息用户可见
P1	设置页暴露 POCKET_PADDING/MIN_SIZE/MAX_SIZE 的语义说明与推荐值 (B3) ; 多口袋并行 (B4)	B3 部分完成 (已在 SPECS) ; B4 待做	S/M	用户知道 18/30 从哪来; 未知位点体系 top-1 未必是真实活性位点

14.2 P2（锦上添花）

表 80 P2（锦上添花）

事项（编号）	状态	工作量
实验位点一致性判据硬编码 8.0 Å（1.7）； <code>pass="coarse"</code> 标记丢失（2.2）； <code>top_from_previous</code> 兜底键名错误（2.3）；AD4 缺输入/产物校验（2.6）	待做	各 S
阴性对照/诱饵集（3.6，L）； <code>pockets</code> 工具异常逃逸（4.6，S）；运行保留期/磁盘配额（4.8，M）	待做	L/S/M
<code>similarity_chart.png</code> 生成但未内嵌（5.5，S）；SSE 重连回放/keepalive 与队列背压（6.7，M）	待做	S/M
盒中心对齐到 0.375 Å 网格中心（B5）	待做	S

14.4 「明确不建议做」（docs/architecture.md § 15.8，共 4 条）

表 81 「明确不建议做」（docs/architecture.md § 15.8，共 4 条）

#	不建议做的事	理由	替代做法
1	补「缺失残基/loop 建模 + 柔性受体」（编号 1.2 的完整版，L）	等于内置同源建模 + 诱导契合，成本远超收益	把「结构断链/缺失残基」作为 局限性写进报告 （S）
2	跨运行/多轮批量对比 UI 与大统计（5.7 + 3.4 完整版，L）	定位是单次筛选闭环	先让 单次运行 可复现、失败可归因；确有需求时先支持「导出两个 run 的 <code>ranking.csv</code> 外部比较」
3	继续加码 AD4 与共价/金属对接（2.6 + 1.2 金属部分，M/L）	AD4 与原引擎 不可通约 ，回退分数混进同一排序表本身就是风险；共价/金属需专门引擎与自定义参数	AD4 不可用即诚实失败并写进 `notes` ；金属体系只做「事实上报 + 结论标注」
4	引入分布式队列/集群调度	单机多核是当前瓶颈内的最优形态，运维复杂度不划算	单机多进程 + 内存护栏（现状）

14.5 规范清理计划（对应 docs/architecture.md § 15.9）

表 82 规范清理计划（对应 docs/architecture.md § 15.9）

项	存量（本次实测）	计划
缺返回类型标注	251	集中在 api/app.py、runs.py、blackboard.py；新代码一律要求，存量按模块分批补
函数内 import 未注明原因	457	真正必要的是重依赖（rdkit/vina/meeko/matplotlib/scipy）与破坏（2 处）；标准库延迟导入应上提模块顶部
静默 except: pass	0	已清零（合理解法标了 # 允许静默：<原因>）
print（非 CLI）	0	已清零
裸 os.getenv	0	已清零（白名单内 0 命中）
缺 from __future__ import annotations	0	已清零
>700 行文件	0（门禁口径下）	7 处已登记豁免；新文件不得超限，api/app.py（1,365 行）建议按 router 拆分
依赖单一来源	—	requirements-local.txt 与 pyproject 双份维护（内容一致但会漂移）；计划收敛为 -e .，测试依赖放 [dependency-groups]

15. 附录

15.1 全部环境变量默认值总表

来源：`.env.example` 与 `src/docking_agent/settings.py` 的 `Spec.default` 逐条核对。「界面可改」= 在设置页面可见并可修改；「只读」= 部署保护项（`.env` 优先）。

LLM 与运行

表 83 全部环境变量默认值总表

变量	默认	说明	界面可改	来源
LLM_API_KEY	空	OpenAI 兼容端点密钥（只写不读）	是（secret）	<code>.env.example:13</code> 、 <code>settings.py:105</code>
LLM_BASE_URL	空	任意 OpenAI 兼容端点	是	<code>.env.example:14</code> 、 <code>settings.py:103</code>
LLM_MODEL	<code>doubao-seed-2-0-pro-260215</code>	默认模型	是	<code>.env.example:15</code> 、 <code>settings.py:107</code>
LLM_TEMPERATURE	0.2	采样温度	是	<code>.env.example:16</code> 、 <code>settings.py:109</code>
LLM_TOP_P	0.9	核采样	是	<code>.env.example:17</code> 、 <code>settings.py:111</code>
LLM_TIMEOUT	600	超时（秒）	是	<code>.env.example:18</code> 、 <code>settings.py:112</code>
LLM_MAX_TOKENS	未设（端点默认）	最大输出 tokens	是	<code>.env.example:20</code> 、 <code>settings.py:113</code>
LLM_THINKING	<code>disabled</code>	思考模式（豆包等会发 <code>thinking</code> 字段）	是	<code>.env.example:23</code> 、 <code>settings.py:115</code>
LLM_EXTRA_BODY LLM_EXTRA_HEADERS	/ 空	额外请求体/请求头（JSON）	是	<code>.env.example:24-25</code> 、 <code>settings.py:117-122</code>
LLM_<字段>_<角色>	继承全局	按角色覆盖（角色： <code>INTAKE/COORDINATOR/PROPERTY/POCKET/DOCKING/BINDING</code> ）	是（角色卡片）	<code>.env.example:34-40</code> 、 <code>runtime/llm.py:66</code>
INTAKE_LLM	<code>on</code>	<code>off</code> = 受理层完全走确定性规则	否（env 直读）	<code>.env.example:44</code> 、 <code>intake.py:613</code>
PORT	5000	HTTP 端口	只读	<code>.env.example:47</code> 、 <code>settings.py:318</code>
HOST	<code>127.0.0.1</code>	监听地址	否	<code>.env.example:49</code>
ARTIFACT_BASE_URL	空（由主机 + 端口拼）	旧扁平产物下载地址前缀；需重启	是	<code>.env.example:51</code> 、 <code>settings.py:312</code>

变量	默认	说明	界面可改	来源
RUN_TIMEOUT_SECONDS	900	运行超时（也是 pilot 预算基数）	是	.env.example:52、settings.py:303
RECURSION_LIMIT	120	图递归上限（协调 Agent 与子 Agent 调用都显式带上）	是	.env.example:63、settings.py
LOG_LEVEL	INFO	日志级别	是	.env.example:54、settings.py:307
CHECKPOINT_BACKEND	memory	memory / sqlite；需重启	是	.env.example:161、settings.py:309
ARTIFACT_HOST	127.0.0.1	旧扁平产物 URL 主机	否	.env.example:164
UVICORN_LOG_LEVEL	info	uvicorn 日志级别	否	.env.example:166
DOCKING_WORKSPACE	空（= 项目根）	工作区根目录	否	.env.example:168、paths.py:29
LOCAL_SETTINGS_PATH	空（= config/local_settings.json）	设置文件位置覆盖（多实例/测试）	否	.env.example:170、settings.py:43
COZE_WORKSPACE_PATH	自动设为项目根	兼容遗留	否	.env.example:173、config.py:90
UPLOAD_MAX_MB	200	上传大小上限（MB）	只读	.env.example:178、settings.py:324

对接与并发

表 84 全部环境变量默认值总表

变量	默认	说明	界面可改	来源
DOCKING_MAX_LIGANDS	0（不限）	单次对接分子数上限	只读	.env.example:58、settings.py:321
DOCKING_WORKERS	空（自动规划）	对接并行进程数	是	.env.example:60、settings.py:274
DOCKING_SERIAL_THREADS	8	小库（单进程）线程数	否（settings 未开该 Spec，core/docking.py:101 直读）	.env.example:62、core/docking.py:101
DOCKING_WORKER_MEM_MB	400	每进程内存预算（MB），用于收敛进程数防 OOM	否（core/docking.py:74 直读）	.env.example:64、core/docking.py:74
VINA_CPU	1	Vina 单实例线程数	是	.env.example:66、settings.py:277
POSE_SAVE_MAX	5000	位姿保存上限	是	.env.example:70、settings.py:279
POSE_TOP_N	200	超限时保存最优前 N 个	是	.env.example:72、settings.py:281
POSE_ARTIFACT_MAX	200	位姿作为可下载产物的数量上限	是	.env.example:74、settings.py:283

Agent 上下文与两阶段漏斗

表 85 全部环境变量默认值总表

变量	默认	说明	界面可改	来源
AGENT_TOOL_TOP_N	20	回传给模型的明细行数（只影响模型视图）	是	.env.example:78、runtime/tool_io.py:59
AGENT_FUNNEL_MIN	500	触发两阶段漏斗的候选数阈值（0=关闭）	是	.env.example:80、runtime/tool_io.py:144
AGENT_REFINE_TOP_N	200	精算阶段头部数量	是	.env.example:82、runtime/tool_io.py:145
AGENT_COARSE_EXHAUSTIVENESS	1	粗筛搜索强度	是	.env.example:84、runtime/tool_io.py:146
AGENT_FINE_EXHAUSTIVENESS	16	精算搜索强度	是	.env.example:85、runtime/tool_io.py:147
AGENT_SHARD_MAX	8	分片数上限护栏	是	.env.example:87、runtime/tool_io.py:148

对接参数自动规划（`AUTO_PARAM_*` 全部默认值见 § 8.3 表，此处不重复）

口袋与盒子

表 86 全部环境变量默认值总表

变量	默认	说明	界面可改	来源
POCKET_ENGINE	auto	auto/p2rank/geometric/known_site	是	.env.example:125、settings.py:178
POCKET_TOP_N	10	保存/展示的预测口袋数量	是	.env.example:123、settings.py:183
POCKET_PADDING	4	口袋转盒子的外扩量 (Å)	是	.env.example:127、settings.py:185
POCKET_MIN_SIZE	18	盒边长下限 (Å)	是	.env.example:128、settings.py:187
POCKET_MAX_SIZE	30	盒边长上限 (Å)	是	.env.example:129、settings.py:190
BOX_SPAN_ENABLED	on	库级配体感知下限开关	是	.env.example:132、settings.py:192
BOX_SPAN_SAMPLE	200	库级下限抽样分子数 K（夹 20–1000）	是	.env.example:134、settings.py:196
BOX_GROUP_MARGIN	10	超限分组判据余量 (Å)	是	.env.example:136、settings.py:200
BOX_LARGE_PADDING	12	大配体组盒子余量 (Å)	是	.env.example:138、settings.py:204

变量	默认	说明	界面可改	来源
P2RANK_HOME	空（自动探测）	本地 P2Rank 目录	否	.env.example:140
P2RANK_THREADS	空 (cpu//2)	P2Rank 线程数	否	.env.example:142

报告与流式输出

表 87 全部环境变量默认值总表

变量	默认	说明	界面可改	来源
REPORT_TOP_N	50	报告内嵌表格行数上限	是	.env.example:146、settings.py:287
CHART_TOP_N	20	图表展示前 N 名	是	.env.example:148、settings.py:289
CHART_BAR_MAX	50	柱状图最大条数	是	.env.example:149、settings.py:291
RESULT_INLINE_LIMIT	200	GET /api/runs/{id} 内联明细上限	是	.env.example:151、settings.py:285
STREAM_TOOL_RESULT_CHARS	2000	SSE 单条工具结果回传字符上限	是	.env.example:153、settings.py:301
SSE_BATCH_SIZE	25	实时事件批量大小	是	.env.example:155、settings.py:293
SSE_FLUSH_MS	200	实时事件刷新间隔 (ms)	是	.env.example:156、settings.py:295
SSE_PROGRESS_MS	1000	进度心跳间隔 (ms)	是	.env.example:157、settings.py:297

设置页「对接默认值」（预填表单；不映射环境变量，但引擎项在未指定时决定执行引擎）

表 88 全部环境变量默认值总表

项	默认	说明	来源
docking.engine	auto	对接引擎（auto/vina/autodock/external）。对话/表单都没有指定引擎时由它决定实际执行引擎，解析优先级：工具参数 > 运行请求 > 本项 > auto	settings.py:169、core/params.py::resolve_engine
docking.exhaustiveness	6	搜索强度（留空 = 自动规划）	settings.py:164
docking.n_poses	1	输出位姿数	settings.py:166
docking.save_poses	true	是否保存位姿文件	settings.py:168

项	默认	说明	来源
docking.max_ligands	0	最大分子数（0=不限）	settings.py:169
docking.positive_control	空	默认阳性对照 SMILES（留空 = 不做对照分析）	settings.py:170

系统内置常量（非环境变量）

表 89 全部环境变量默认值总表

常量	值	位置
DEFAULT_BOX_SIZE	[22.0, 22.0, 22.0]	core/receptors.py:24
DEFAULT_EXHAUSTIVENESS	6	core/docking.py:49
DEFAULT_N_POSES	1	core/docking.py:52
DEFAULT_AGREE_RADIUS	8.0 Å	core/pockets.py:46
BOX_LARGE_MAX_SIZE	60.0 Å	core/pockets.py:65
DEFAULT_PADDING / MIN_SIZE / MAX_SIZE / TOP_N	4.0 / 18.0 / 30.0 / 10	core/pockets.py:47-50
DEFAULT_BOX_SPAN_SAMPLE	200（夹 20–1000）	core/pockets.py:57-59
DEFAULT_SPAN_MARGIN	10.0 Å	core/pockets.py:61
PLAN_SEED / PLAN_ENGINE	42 / vina	core/params.py:58-59
FALLBACK_EXHAUSTIVENESS FALLBACK_N_POSES	/ 6 / 1	core/params.py:49-50
_SPAN_SEED	42	core/pockets.py:66
_PREP_ATTEMPTS	altloc A / B / C / 无参数	core/receptors.py:174-175
MAX_MESSAGES	40	agents/coordinator.py:41
黑板快照上限	pockets 20 / molecules • properties • docking • binding 各 50	runtime/blackboard.py:196-202
_SPAN 最近秩百分位	_percentile_nearest_rank（n≤20 时即最大值）	core/pockets.py:205
POSITIVE_CONTROL_NAME	__positive_control__	core/docking.py:47

15.2 关键文件索引（哪个文件负责什么）

表 90 关键文件索引（哪个文件负责什么）

想知道…	看这个文件
命令行入口（ <code>--check/pipeline/agent/runs/serve</code> ）	<code>src/docking_agent/cli.py</code>
全部 HTTP/SSE 接口（31 个路由）	<code>src/docking_agent/api/app.py</code>
请求体模型	<code>src/docking_agent/api/schemas.py</code>
任务受理层（ <code>task_spec</code> / 三态决策 / 多轮继承 / 受体三态）	<code>src/docking_agent/intake.py</code>
确定性流水线（零 LLM）	<code>src/docking_agent/pipeline.py</code>
主管 Agent 构建与提示词加载	<code>src/docking_agent/agents/coordinator.py</code>
4 个子 Agent 构建（独立 LLM 实例 + 独立 checkpointer）	<code>src/docking_agent/agents/workers.py</code>
子 Agent 系统提示词（含 <code>OUTPUT_ECONOMY</code> 输出经济性）	<code>src/docking_agent/agents/prompts.py</code>
子 Agent 结构化输出契约（ <code>agent_note</code> 一句/≤80 字由校验器强制）	<code>src/docking_agent/agents/reports.py</code>
受体溯源记账（条件纪律段与报告 § 1.2 共用）	<code>src/docking_agent/runtime/run_facts.py</code>
主管提示词正文（12 条纪律，其中 4 段按运行条件注入）	<code>config/agent_llm_config.json</code> 的 <code>sp</code>
条件纪律段（按运行事实抽取提示词子集）	<code>src/docking_agent/agents/prompt_blocks.py</code>
运行级小事实记账（条件段的判断依据）	<code>src/docking_agent/runtime/run_facts.py</code>
共享黑板（横向协作）	<code>src/docking_agent/runtime/blackboard.py</code>
工具原始返回落盘 + 模型视图截断 + 漏斗参数	<code>src/docking_agent/runtime/tool_io.py</code>
唯一的结果合并/落盘入口（ <code>_PROVENANCE_KEYS</code> ）	<code>src/docking_agent/agents/persistence.py</code>
主管级分发工具与边界校验 / 受体护栏	<code>src/docking_agent/agents/dispatch.py</code>
子 Agent 对接工具	<code>src/docking_agent/tools/docking.py</code>
口袋工具（预测/比对/提交盒子）	<code>src/docking_agent/tools/pockets.py</code>
性质工具	<code>src/docking_agent/tools/properties.py</code>
结合模式工具	<code>src/docking_agent/tools/binding.py</code>
报告工具（触发落盘 + 产物登记）	<code>src/docking_agent/tools/report.py</code>
在线解析工具（RCSB/UniProt/PubChem）	<code>src/docking_agent/tools/online.py</code>
<code>choices</code> 结构化选项与失败回执	<code>src/docking_agent/tools/choices.py</code>
Vina/AD4 引擎封装、并发规划、LPT、large	<code>src/docking_agent/core/docking.py</code>

想知道…	看这个文件
分组	
受体准备（去水/杂原子/altloc/内容哈希缓存）	src/docking_agent/core/receptors.py
配体读取、3D 构象、化学体检	src/docking_agent/core/ligands.py
口袋预测与对接盒决策（含库级下限）	src/docking_agent/core/pockets.py
对接参数自动规划（公式/漏斗/pilot 降级）	src/docking_agent/core/params.py
性质/指纹/药效团/相似度	src/docking_agent/core/chemistry.py
结果融合与排序	src/docking_agent/core/ranking.py
名称归一化 + UniProt 检索打分	src/docking_agent/core/resolve.py
报告 Markdown 模板（固定 9 章）	src/docking_agent/reporting/report.py
报告内容纯函数层（格式化/版本/失败统计/参数规划表）	src/docking_agent/reporting/content.py
PDF 生成（封面/页脚/图片内嵌）	src/docking_agent/reporting/pdf.py
图表	src/docking_agent/reporting/charts.py
ranking.csv 列定义	src/docking_agent/reporting/tables.py
产物登记与 CHART_SPECS	src/docking_agent/reporting/artifacts.py
运行目录、产物清单、下载命名、分页/聚合/zip	src/docking_agent/runs.py
设置页字段表与优先级	src/docking_agent/settings.py
环境变量读取与运行时环境准备	src/docking_agent/config.py
路径解析	src/docking_agent/paths.py
按角色模型解析与登记表	src/docking_agent/runtime/llm.py
SSE 事件封装与节流	src/docking_agent/runtime/streaming.py
取消（协作式）	src/docking_agent/cancellation.py
受体注册表与已知位点	config/receptors.json
前端全部交互（SSE/卡片/设置页/编排时间轴）	web/app.js
前端结构（DOM id 被门禁看护）	web/index.html
设计令牌与布局	web/styles.css
简易模式（默认首页 /；只有对话 + 结果；参数走默认与自动规划）	web/simple.html / simple.js / simple.css
高级模式入口（参数表单 / 历史 / 报告 / 中间数据 / 设置页）	GET /advanced → web/index.html
简易模式的静态契约（无参数表单 / 标准协议 / 背景动效可关闭 / 无孤儿 id）	scripts/check_web.py 第 9 节

想知道…	看这个文件
一键启动与自检	start.sh
真实性取证（独立复算 + 反证控制）	scripts/verify_docking.py
Fake LLM 多 Agent 冒烟	scripts/smoke_test.py
受理层用例集	scripts/intake_eval.py
前端静态校验 / DOM id 快照 / jsdom 端到端 / 真浏览器体检	scripts/check_web.py / snapshot_ids.py / ui_e2e.js / ui_shot.py
静态规范增量门禁	scripts/lint_local.py + scripts/lint_baseline.json
残留进程清理	scripts/kill_leftovers.sh

15.3 术语表

表 91 术语表

术语	含义
Vina	AutoDock Vina ， 本 系 统 的 主 对 接 引 擎（1.2.7），提供打分函数与采样；结果为近似结合自由能（kcal/mol），越负表示越强
AutoDock4 (AD4)	经 典 AutoDock4 CPU 引 擎（autodock4/autogrid4），作为 Vina 不可用时的回退；其分数与原引擎 不可通约
PDBQT	AutoDock 系列的受体/配体输入格式，在 PDB 基础上增加原子类型、电荷与可旋转键（TORSDOF）信息
HETATM	PDB 中非标准残基记录（金属离子、辅因子、水、共晶配体、结晶添加剂等）
apo	不含辅因子/共晶配体的受体结构；本系统在丢弃杂原子时会在报告中标明结论是「去辅因子（apo）条件下的结果」
apex / 药效团锚定基团	与靶点关键亚口袋（如丝氨酸蛋白酶 S1 口袋）形成特征性相互作用的官能团，本系统用 SMARTS 匹配脘基/胍基/羧酸/磺酰胺/芳环
funnel（两阶段漏斗）	大库打法：先全库低强度粗筛，再对头部高强度精算；合并时精度优先，粗筛分保留在 affinity_coarse
pass（阶段）	漏斗的一轮（coarse / fine）；同一 pass 内参数与盒子完全一致

术语	含义
shard (分片)	把分子库切成多片分别调度；本系统 不推荐 （LLM 往返开销），且片数受 AGENT_SHARD_MAX=8 约束
box_group	对接盒分组： main （主组，全库共用主盒）与 large （超限大配体，同一中心的更大盒子）； 不跨组比较分数
pocket (口袋)	受体表面可能结合小分子的空腔；由 P2Rank（机器学习）或内置几何法预测，或由共晶配体/注册表给出实验位点
candidate (候选)	① 候选分子（待筛选的小分子）；② 候选口袋；③ 在线解析时的候选受体条目（accession 级）。上下文会明确是哪一种
task_spec (任务规约)	受理层产出的结构化任务描述，含 decision/authority/receptor/ligands/site/params 等只读字段
decision	run / ask / reject 三态，是受理层对「该跑还是该问」的显式判定，编排层无条件服从
blackboard (共享黑板)	运行级进程内共享工作区，子 Agent 通过它横向交换分子/性质/口袋/位点/对接/结合模式
tool_io 视图	工具原始返回全量落盘，只把 summary + top-N 回给模型； AGENT_TOOL_TOP_N 只控制视图大小
choices	解析出现多个候选或置信度不足时下发的结构化选项（前端渲染成按钮），点选后以同一 conversation_id 继续
unresolved	点名受体在线解析失败 / 歧义时写回的 receptor.source ；此时 run_docking 代码级护栏在调用任何引擎前拦下
param_plan	参数自动规划的可追溯产物： exhaustiveness/n_poses/漏斗/pilot/每条决策理由
pilot	预算护栏的探针：用最贵的少数分子以 exhaustiveness=1 真实试跑并外推总耗时； 只测量 ，不写位姿、不进排序
completeness	运行记录里「本次实际完成了什么」的字段； 仅信息展示，不驱动行为 （服务端不事后接管/不自动补齐）
seed_policy	种子策略；当前只有 session （Vina 每次 dock() 按构造种子重新采样，故分数与批次无关）

术语	含义
内容寻址缓存	以「源文件内容哈希 + 影响结果的参数哈希」为键的缓存（而非时间戳），避免并发互相覆盖与缓存恒失效
LPT 调度	Longest Processing Time first：按成本（可旋转键×4 + 重原子）从大到小提交对接，减少尾部拖尾
P95（最近秩法）	百分位取法： <code>vals[min(rank, n)-1]</code> ， $n \leq 20$ 时即最大值；用于库级跨度下限，抗单个异常值
C 方案	对接盒的「库级配体感知下限 + 超限分子分组」实现（ docs/architecture.md § 15.7b ）
门禁	改完代码必须全绿的可执行检查集合（ <code>pytest / smoke / verify_docking / check_web / snapshot_ids / ui_e2e / ui_shot / intake_eval / lint_local</code> ）
取证（反证控制）	不信任自身封装的验证方法：独立复算 + 「参数一变结果必须变」的反向证明

报告结束。 本报告的全部数值均来自仓库内代码、既有文档与本次实测运行；未实测的量已明确标注为「未实测」，未做任何推算冒充实测。